



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Bitácora S09

Mensajería con RabbitMQ en Mexi Banco

Asignatura: Sistemas Distribuidos
Profesor: Oscar Manuel Ruiz Hurtado
Grupo: 02
Semestre: 2027-1
Integrante: Arroyo Onofre Leonardo

Objetivo

Cambiar los avisos de las transferencias de una llamada HTTP directa a mensajes en RabbitMQ. La idea es comprobar que una transferencia pueda terminar aunque notificacion esté detenido, y después observar qué pasa con los mensajes repetidos y con uno que ni siquiera se puede interpretar. Para esto se continuó con Mexi Banco y se usaron cuentas distintas en cada escenario.

0 El problema de depender de una respuesta inmediata

Primero se levantó el entorno con Docker Compose y se crearon las cuentas de terminación 911 y 912 con 1000 y 500. Antes de hacer la transferencia se consultaron ambos saldos por el gateway; así se tuvo una referencia de lo que debía cambiar.



Figura 1: Levantamiento del entorno de Mexi Banco con Docker Compose.

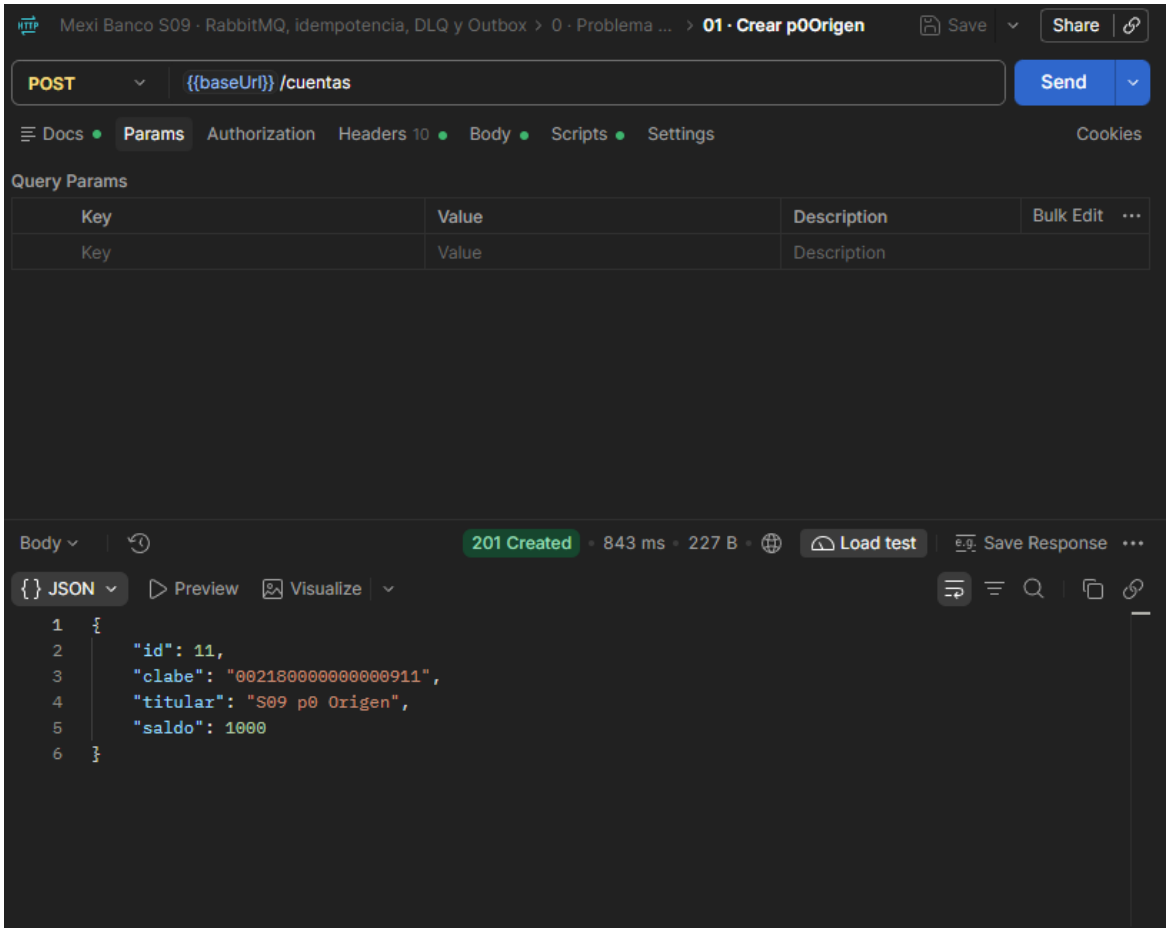


Figura 2: Creación del origen 911 con saldo de 1000.

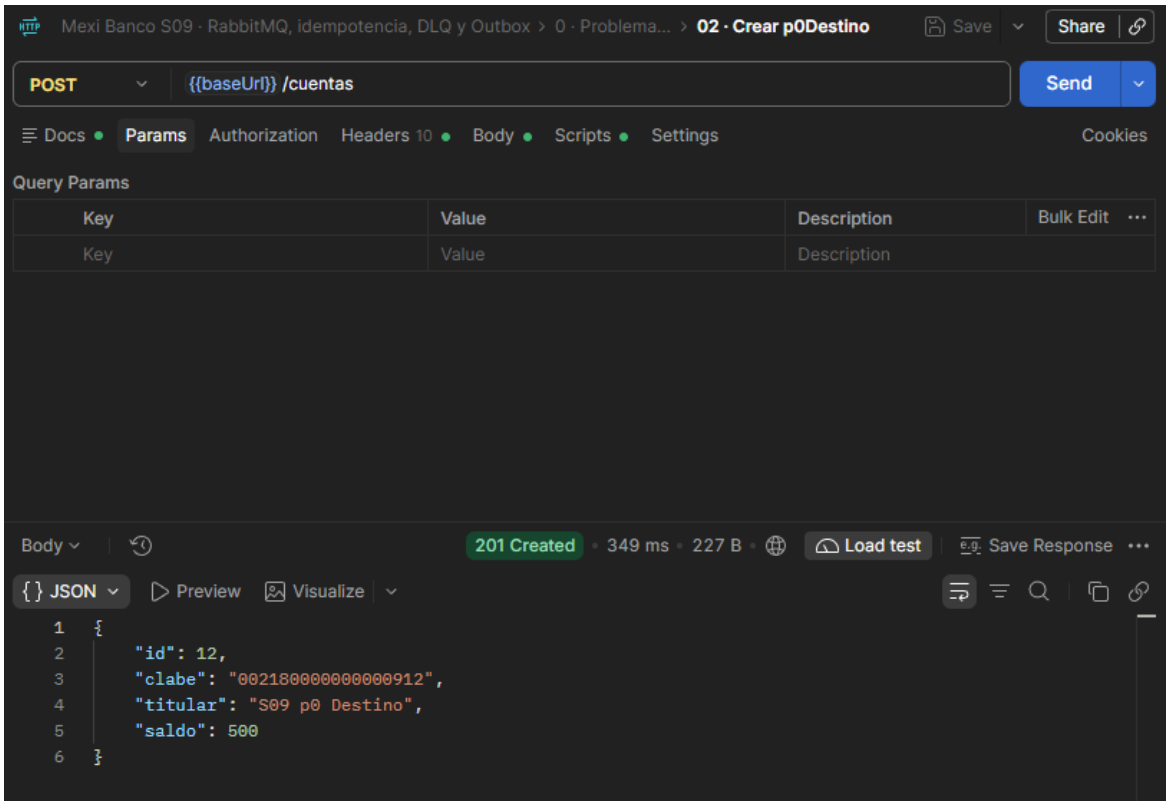


Figura 3: Creación del destino 912 con saldo de 500.

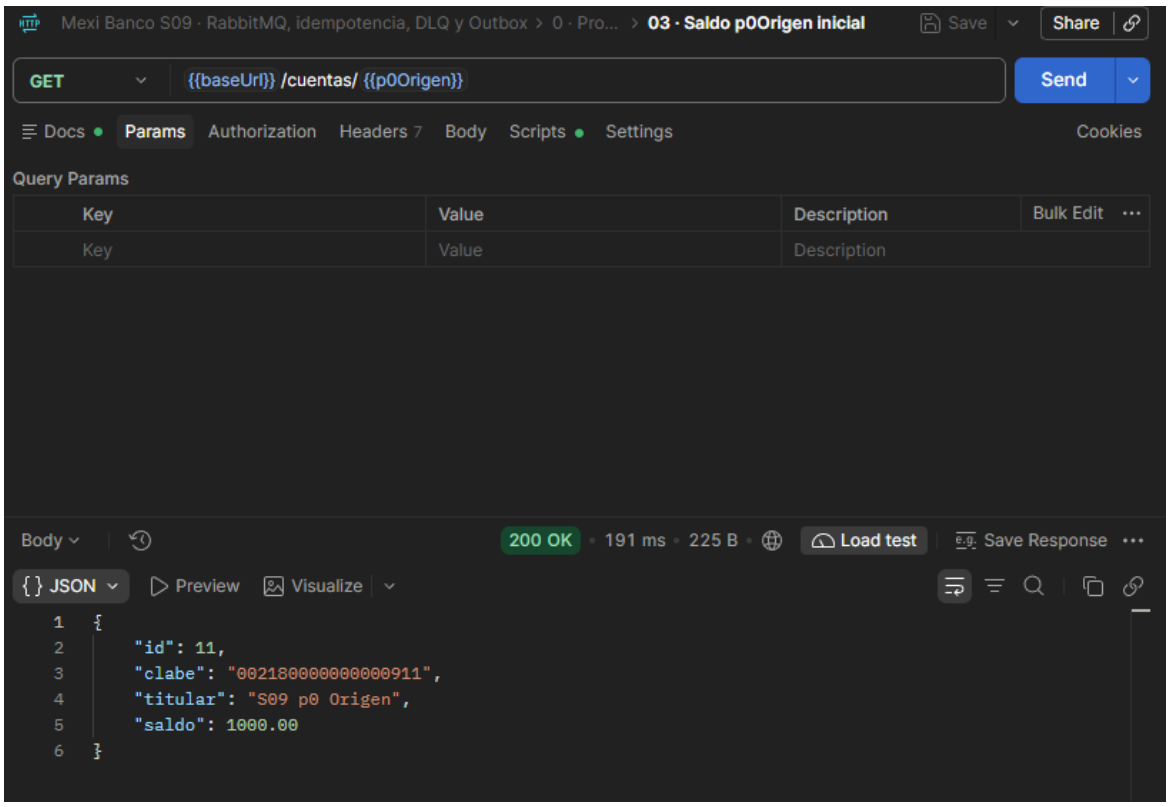


Figura 4: Consulta inicial del origen: 1000.

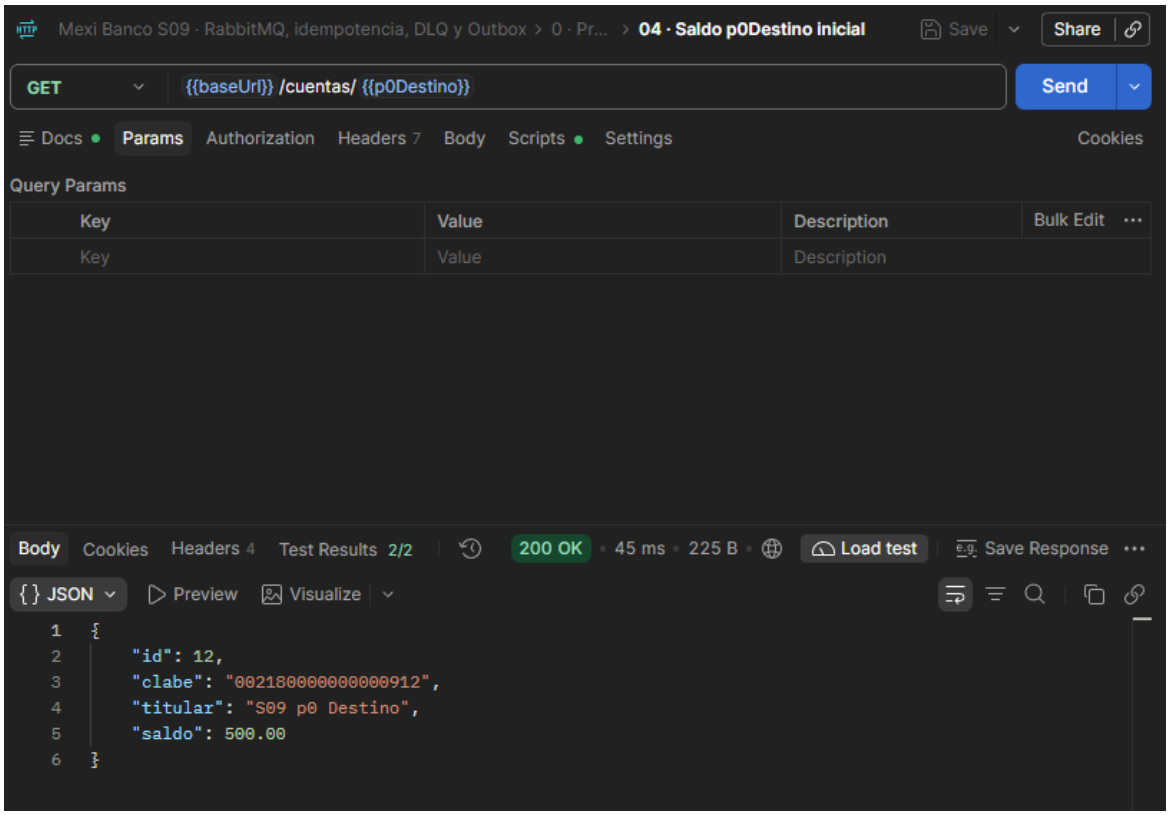


Figura 5: Consulta inicial del destino: 500.

Después se detuvo notificacion y se envió una transferencia de 50. La respuesta fue 503, pero el cuerpo indicó que los saldos y los movimientos ya estaban completos y que el aviso estaba pendiente. La transferencia 25 quedó en NOTIFICACION_PENDIENTE. Es importante distinguir las dos cosas: el error al responder no significa que el dinero no se haya movido.

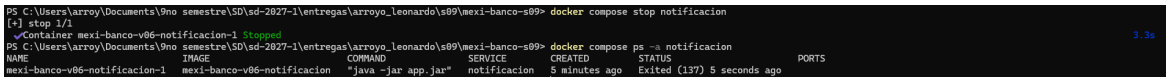


Figura 6: Detención de notificacion antes de transferir.

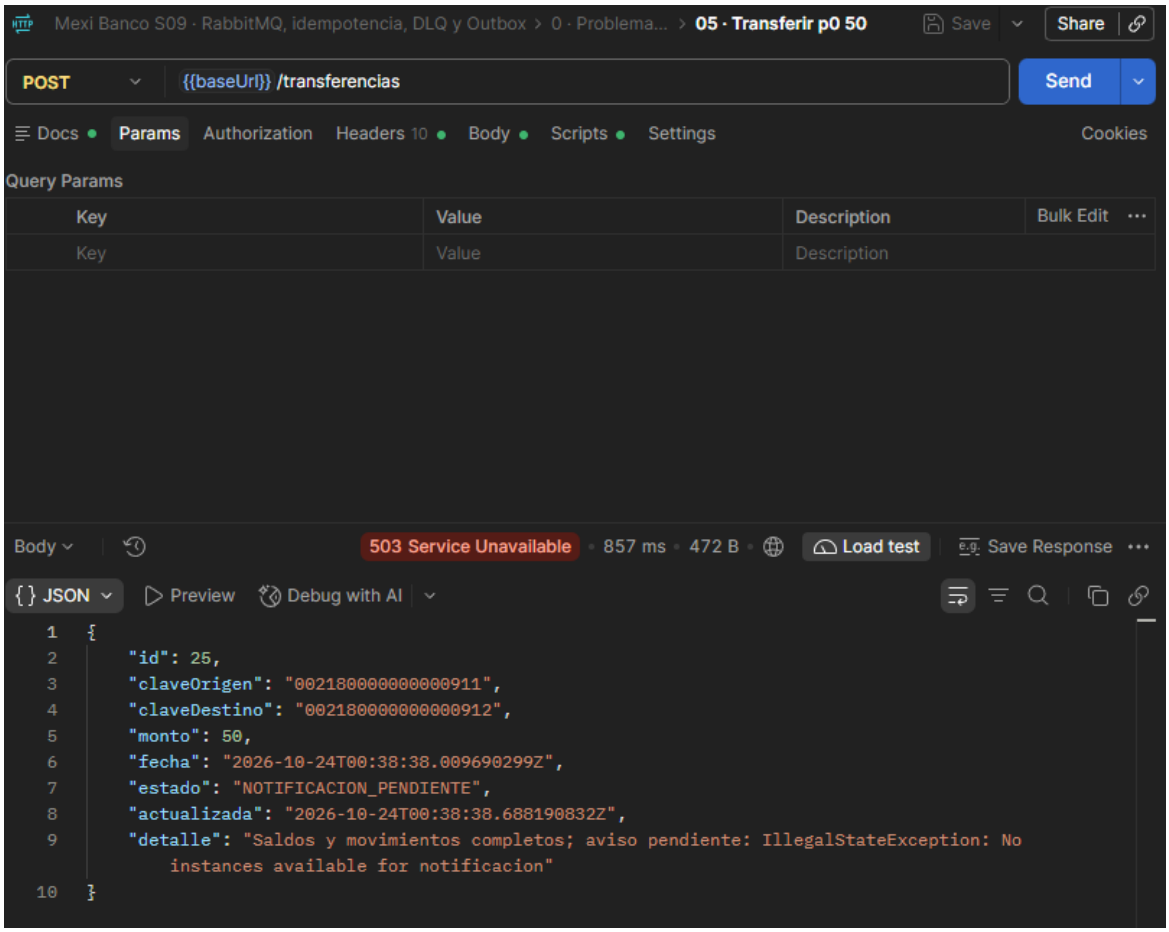


Figura 7: Respuesta 503 de la transferencia 25, con aviso pendiente.

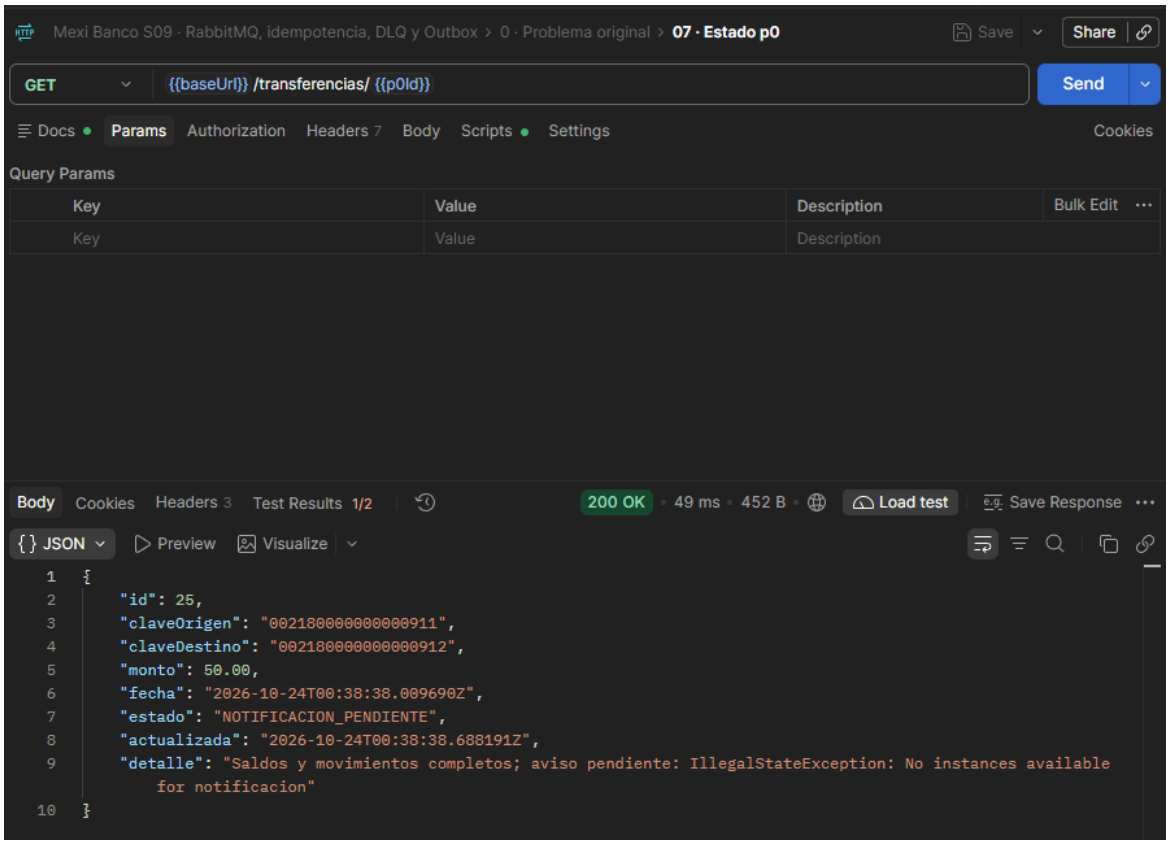


Figura 8: Consulta del estado guardado de la transferencia 25.

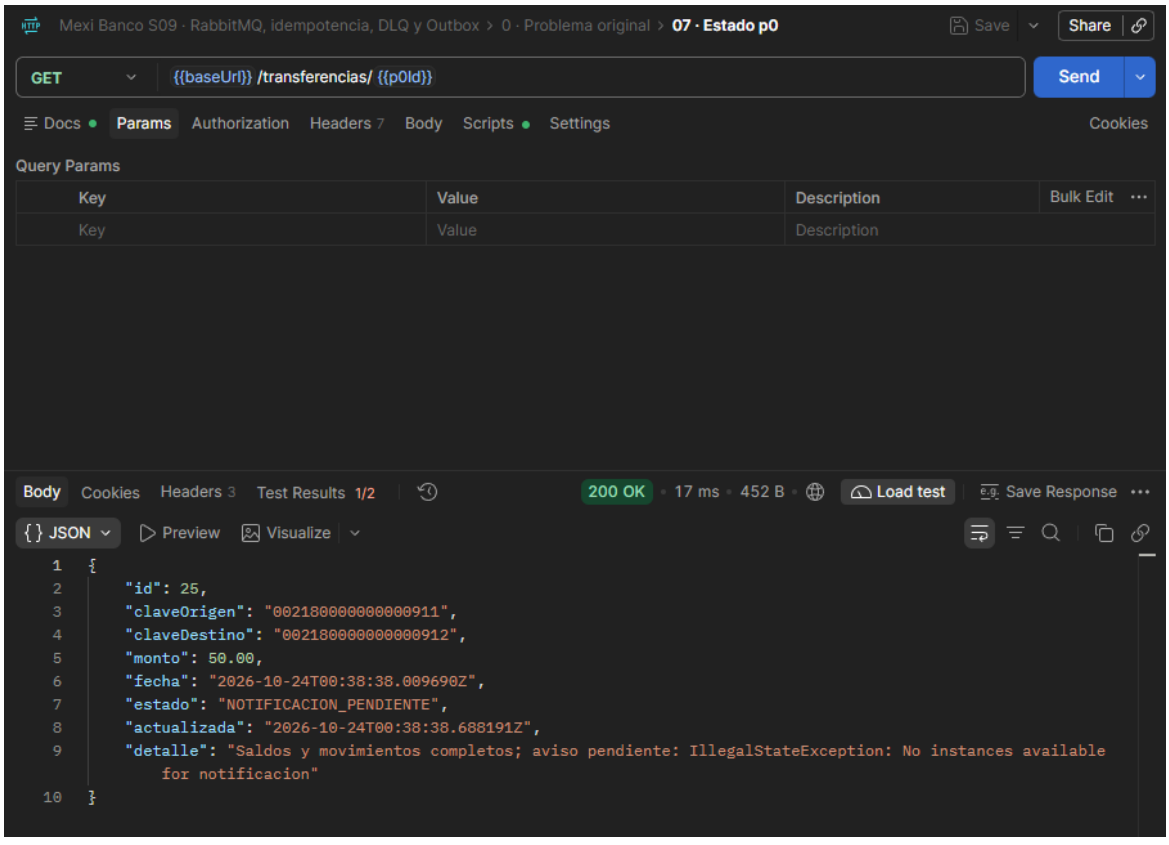


Figura 9: Nueva consulta que conserva NOTIFICACION_PENDIENTE.

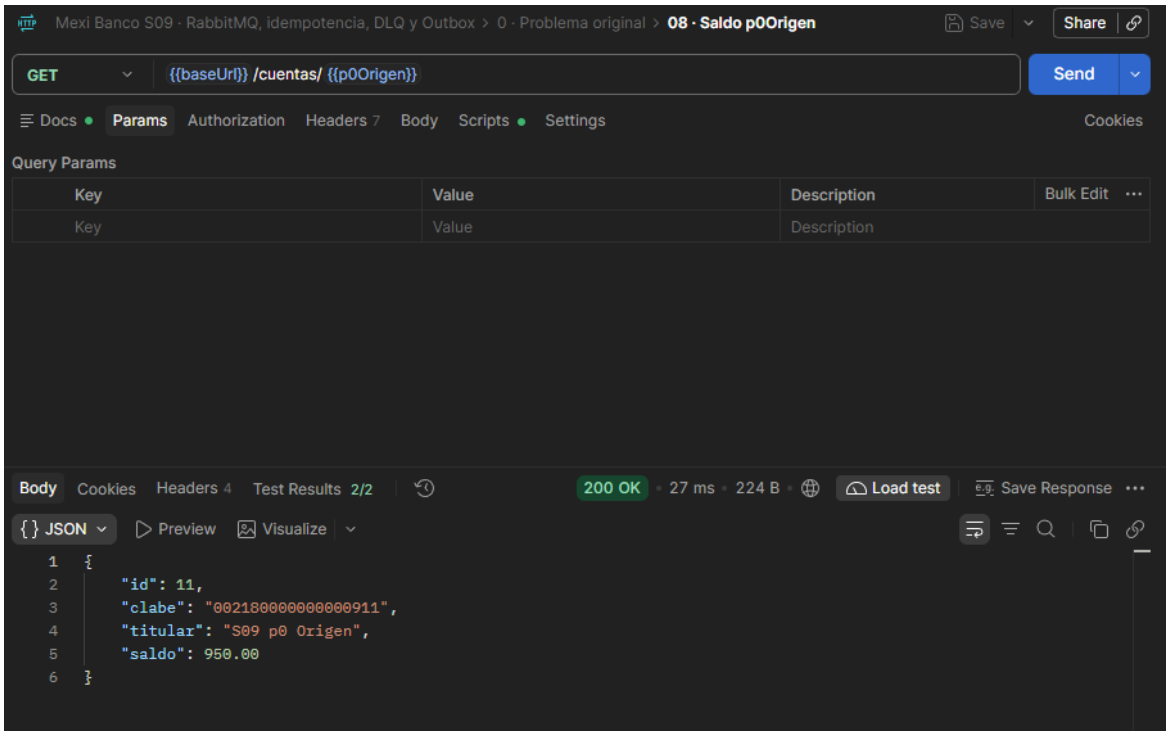


Figura 10: Saldo del origen después del cargo: 950.

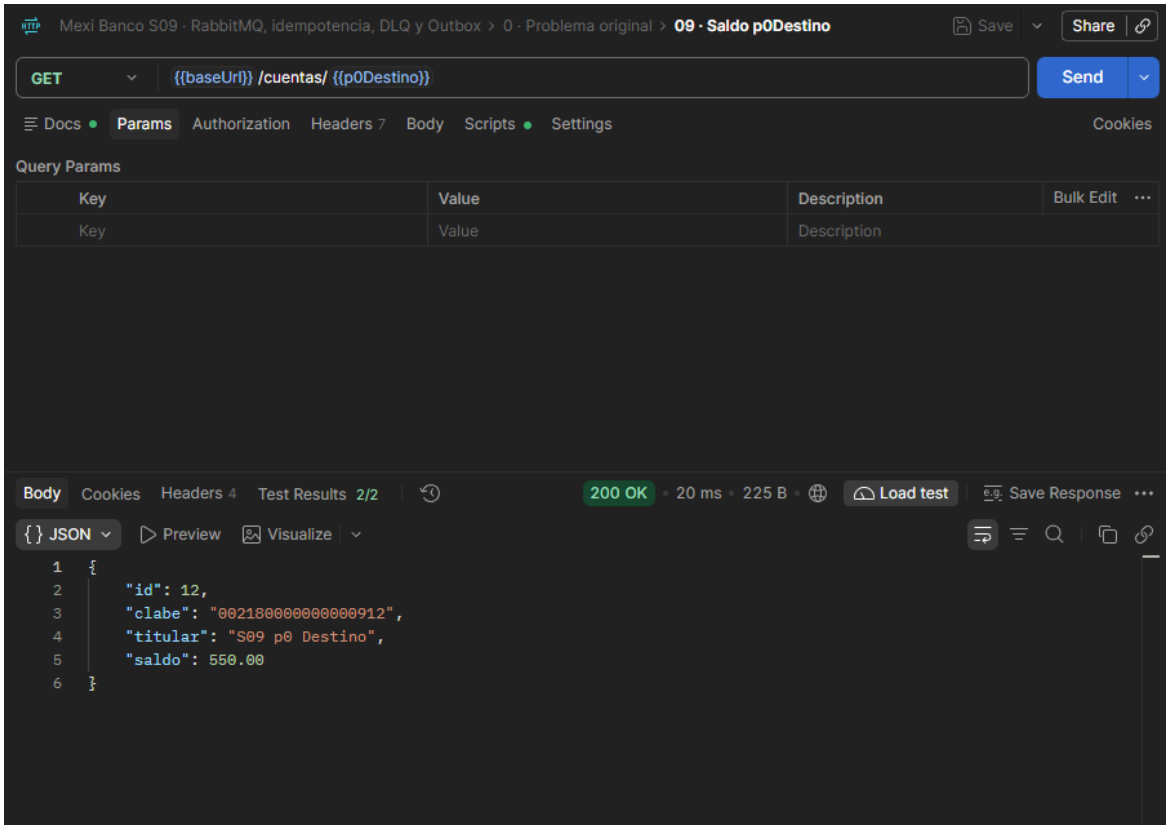


Figura 11: Saldo del destino después del abono: 550.

Se volvió a iniciar notificacion y se consultó otra vez la transferencia. El estado siguió pendiente. Encender el servicio no repite por sí solo la petición HTTP que falló. En esta

versión el resultado que se observa es NOTIFICACION_PENDIENTE, y los saldos 950 y 550 confirman que el problema estuvo en el aviso, no en el traslado de los 50.

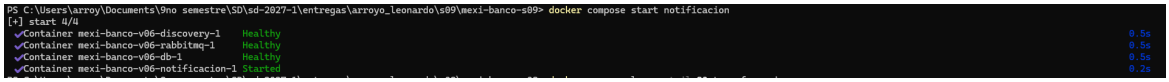


Figura 12: Reanudación de notificacion.

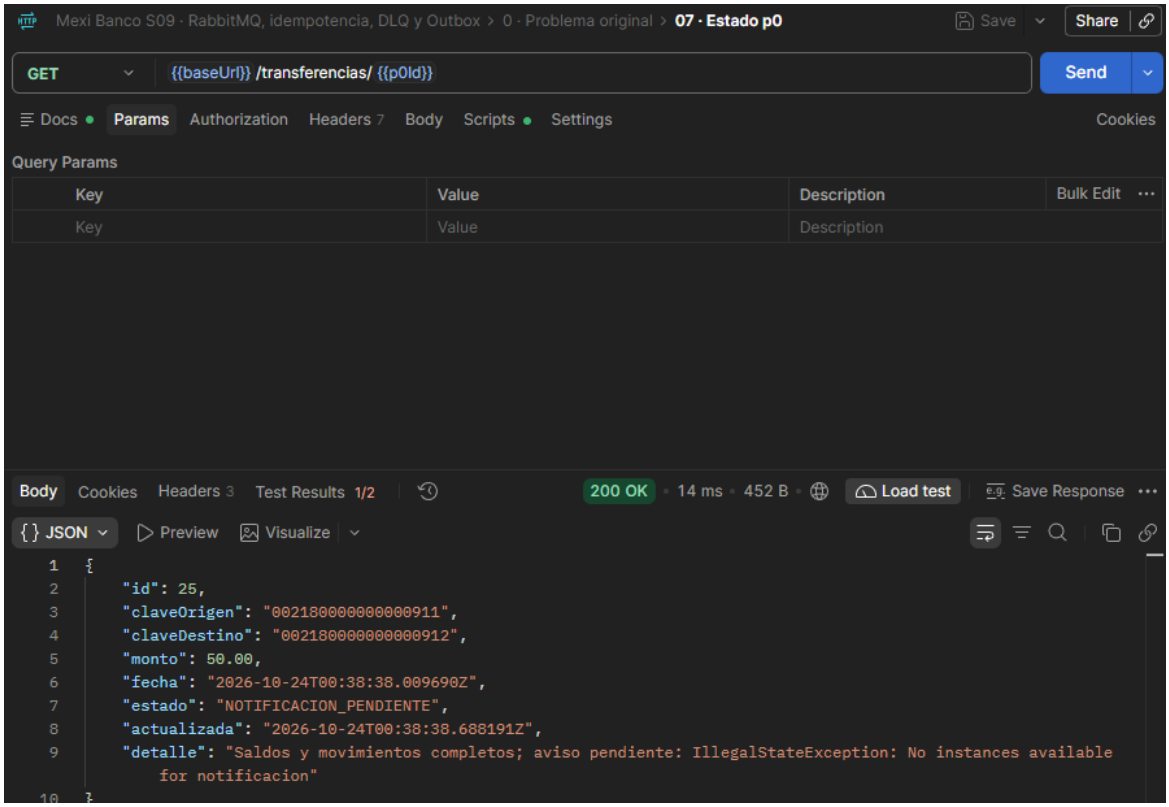


Figura 13: El pendiente se conserva después de recuperar el servicio.

1 Publicación de eventos en RabbitMQ

Ahora el servicio transferencia publica un evento y notificacion lo consume por separado. RabbitMQ es el intermediario: el exchange recibe el evento y lo dirige a las colas que coincidan con su llave de ruteo. La cola conserva el trabajo hasta que el consumidor lo procese y confirme. Esto separa el momento de transferir del momento de crear los avisos.

El exchange se llama transferencias, es de tipo topic y es durable. Una transferencia COMPLETADA publica con la llave transferencia.realizada; una COMPENSADA usa transferencia.revertida. La cola durable notificacion.transferencias se enlaza con transferencia.*, por lo que recibe ambos tipos de evento. Que un exchange sea durable conserva su definición al reiniciar el broker, pero no lo convierte en una cola que almacene eventos sin destinatario.

El evento contiene eventoId, tipo, transferenciaId, clabeOrigen, clabeDestino, monto, saldoOrigen, saldoDestino y fecha. eventoId es un UUID estable que identifica el hecho de negocio. Si se vuelve a entregar el mismo evento, se conserva ese identificador; generar otro UUID en cada reintento impediría reconocerlo como repetido.

En PublicadorDeEventos se envía el mensaje como persistente, se usa eventoId como messageId y se espera el publisher confirm del broker. El siguiente fragmento resume las operaciones del publicador; los confirms correlacionados permiten relacionar el resultado con este envío.

```
String id = evento.eventoId().toString();
CorrelationData correlacion = new CorrelationData(id);
rabbit.convertAndSend(EXCHANGE, evento.routingKey(), evento, msg -> {
    msg.getMessageProperties().setMessageId(id);
    msg.getMessageProperties().setDeliveryMode(MessageDeliveryMode.PERSISTENT);
    return msg;
}, correlacion);
CorrelationData.Confirm confirm =
    correlacion.getFuture().get(esperaMs, TimeUnit.MILLISECONDS);
```

Además se comprueba si el confirm fue negativo o si el mensaje fue devuelto por no encontrar una cola. La publicación confirmada no significa que notificacion ya haya guardado un aviso: solamente describe la etapa del broker. El procesamiento y su confirmación ocurren después.

Para la primera prueba se reconstruyó transferencia y se mantuvo notificacion detenido. Con las cuentas 921 y 922 se enviaron 50. La transferencia 26 respondió 201 y terminó COMPLETADA; los saldos quedaron en 950 y 550 sin esperar una respuesta HTTP del consumidor.



Figura 14: Reconstrucción de transferencia y detención de notificacion.

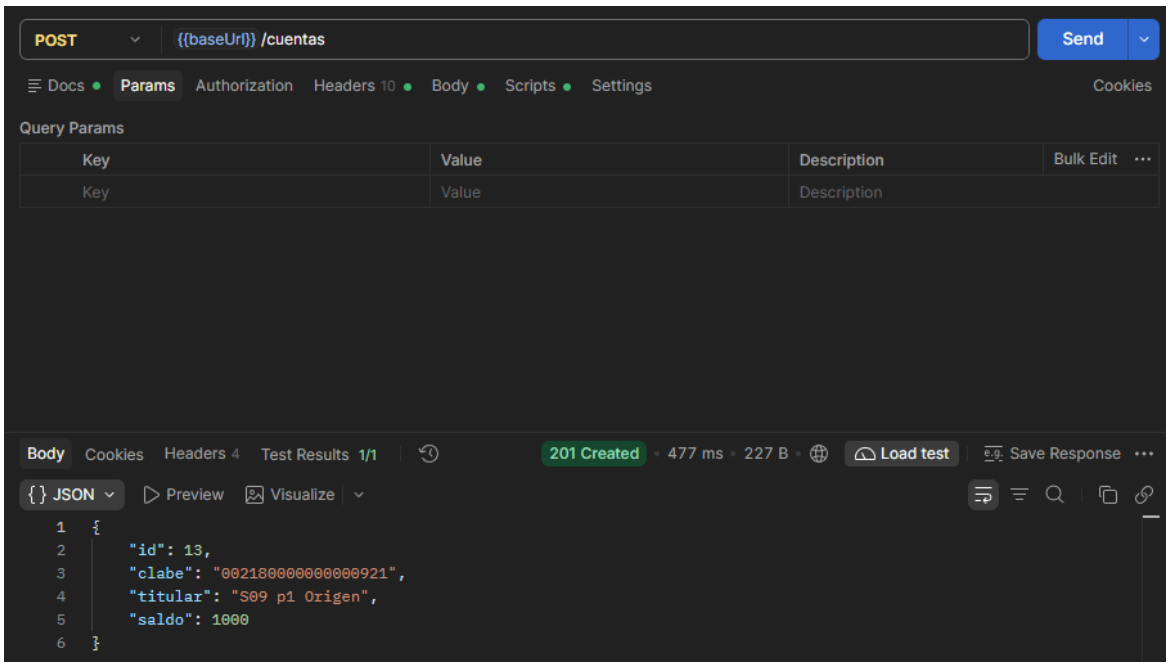


Figura 15: Origen 921 con saldo inicial de 1000.

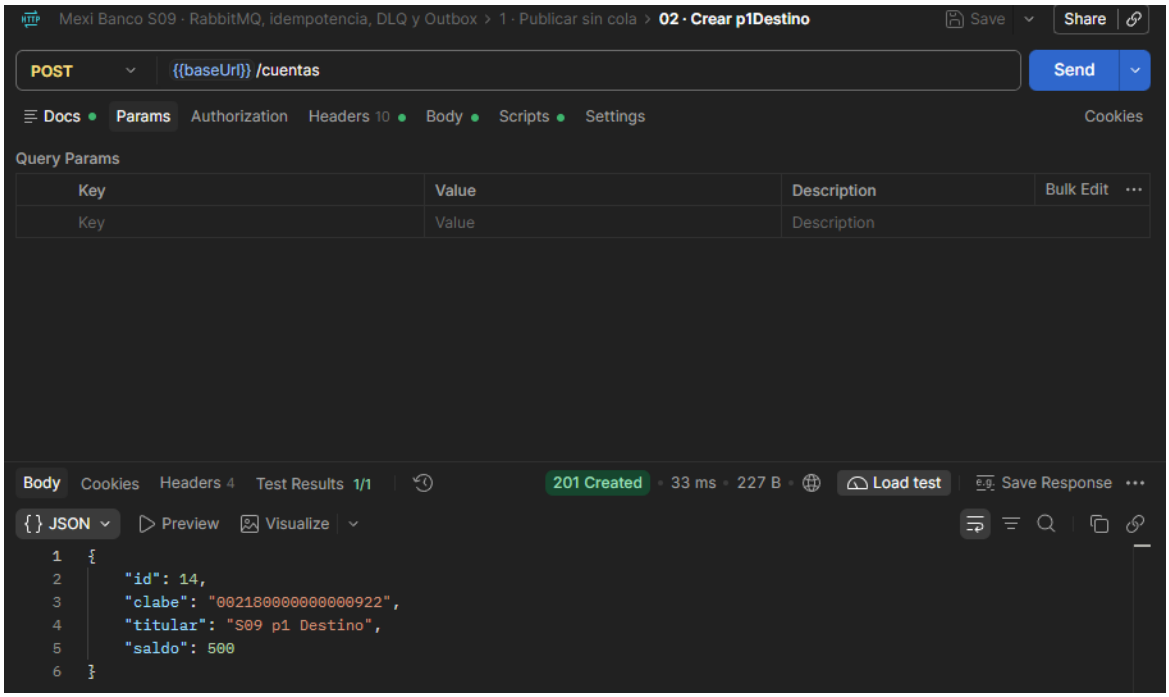


Figura 16: Destino 922 con saldo inicial de 500.

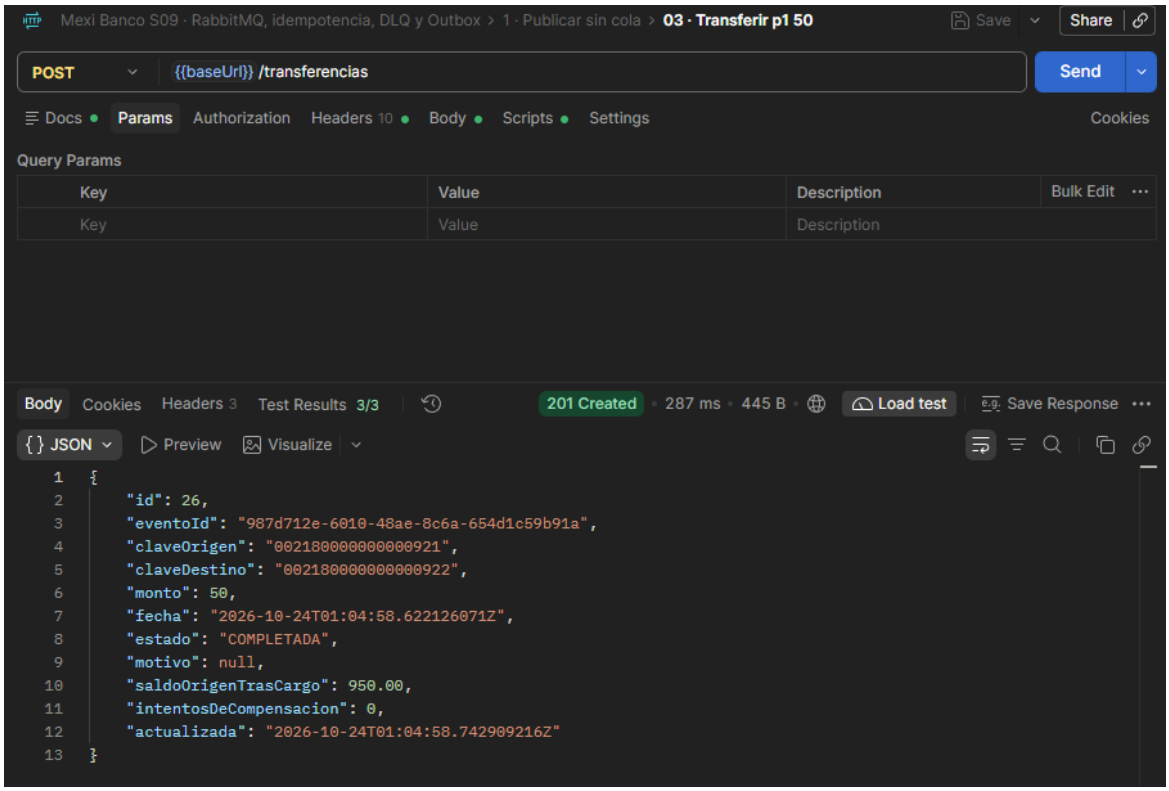


Figura 17: Transferencia 26 completada con respuesta 201 y eventoId.

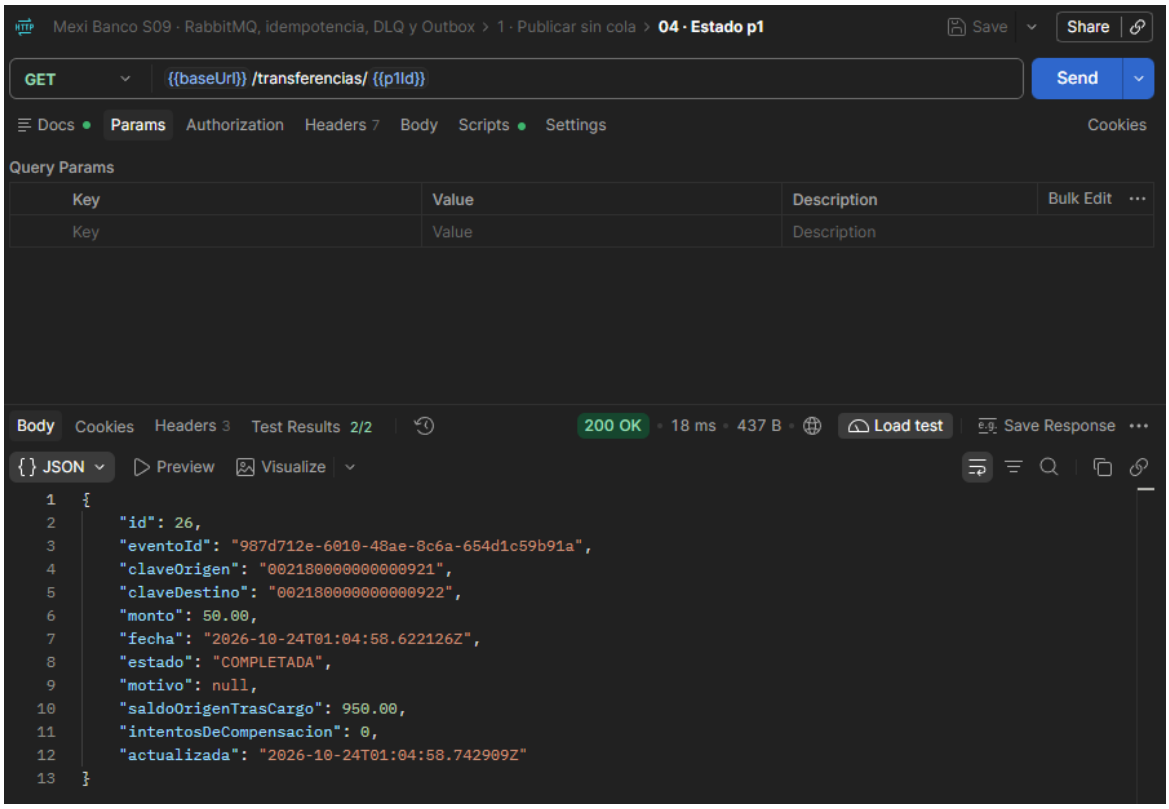


Figura 18: Consulta posterior de la transferencia COMPLETADA.

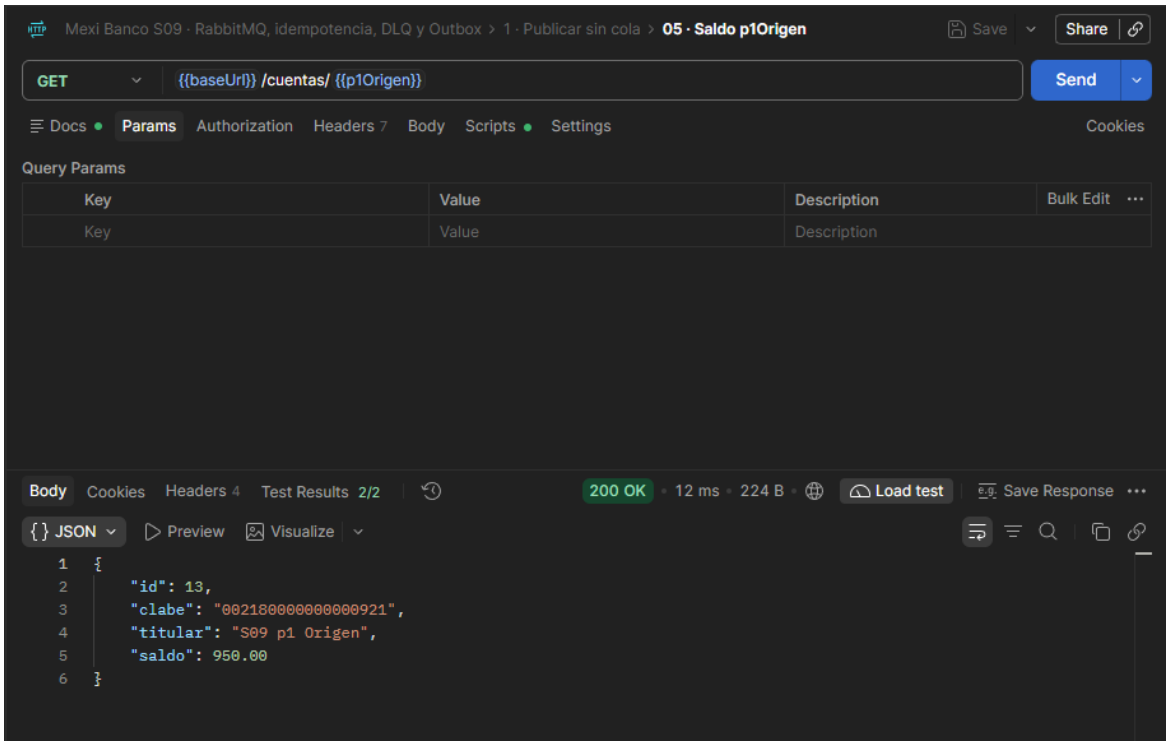


Figura 19: Saldo del origen: 950.

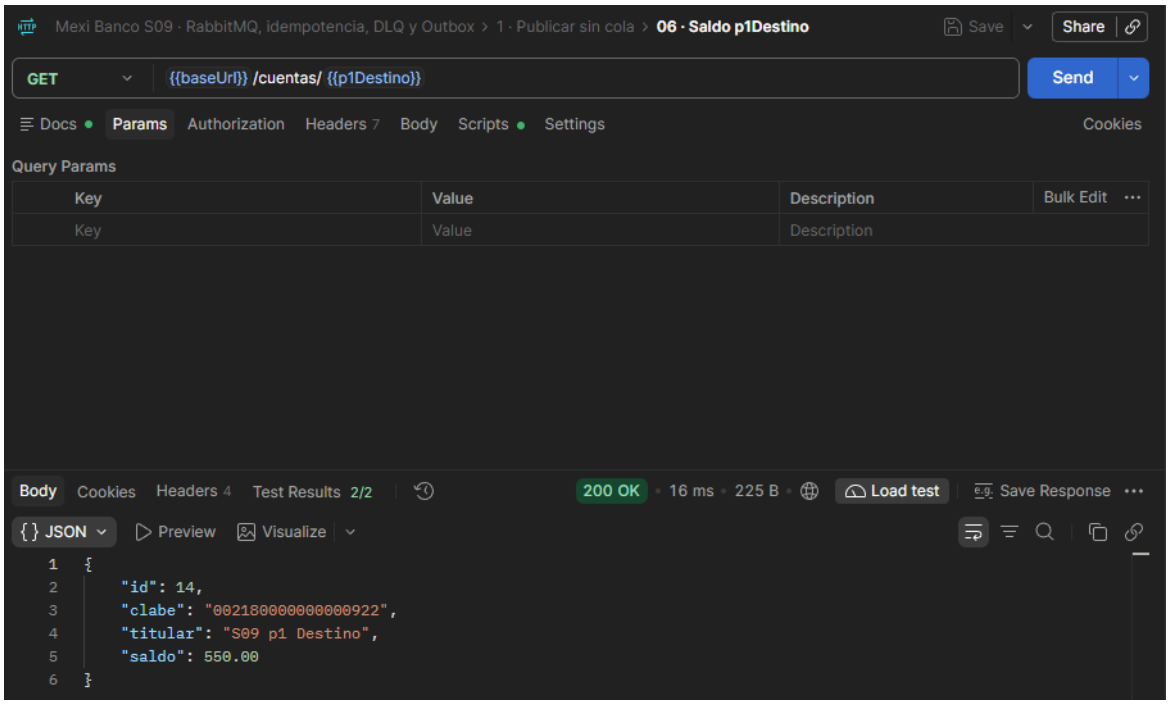


Figura 20: Saldo del destino: 550.

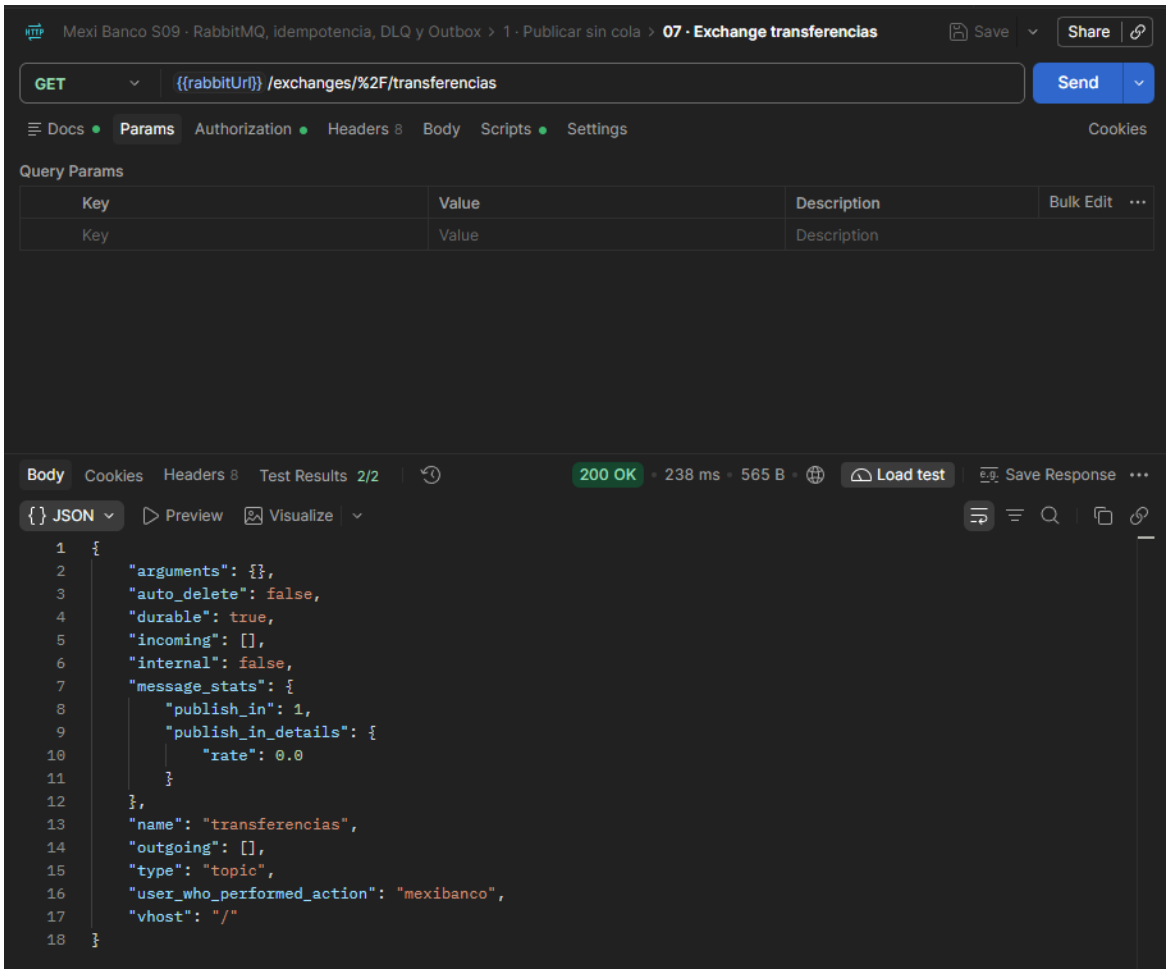


Figura 21: Exchange transferencias de tipo topic y durable.

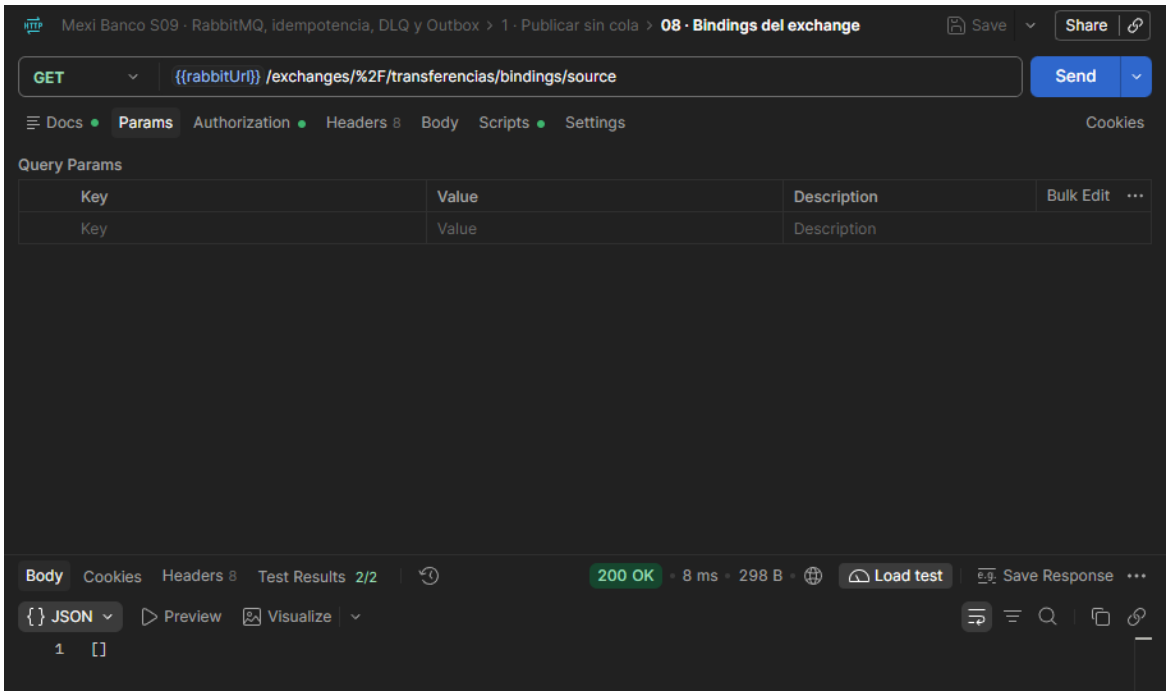


Figura 22: Consulta de bindings que devuelve una lista vacía en esta primera publicación.

Aquí apareció un detalle importante: la consulta de bindings devolvió una lista vacía. Aunque el exchange existía, todavía no había una cola enlazada para conservar el mensaje de esa publicación. Por eso antes de probar la acumulación se inició notificacion para que declarara la cola y su enlace. La disponibilidad del consumidor y la existencia de la cola son condiciones distintas.

2 Desacoplamiento temporal y recuperación del consumidor

Se inició notificacion y se prepararon las cuentas 931 y 932 para las transferencias normales, además de 941 y 942 para la compensación. Todas comenzaron con 1000 en el origen y 500 en el destino. Se revisó que la cola ya existiera y que estuviera vacía antes de detener al consumidor.



Figura 23: Inicio de notificacion para declarar la topología.

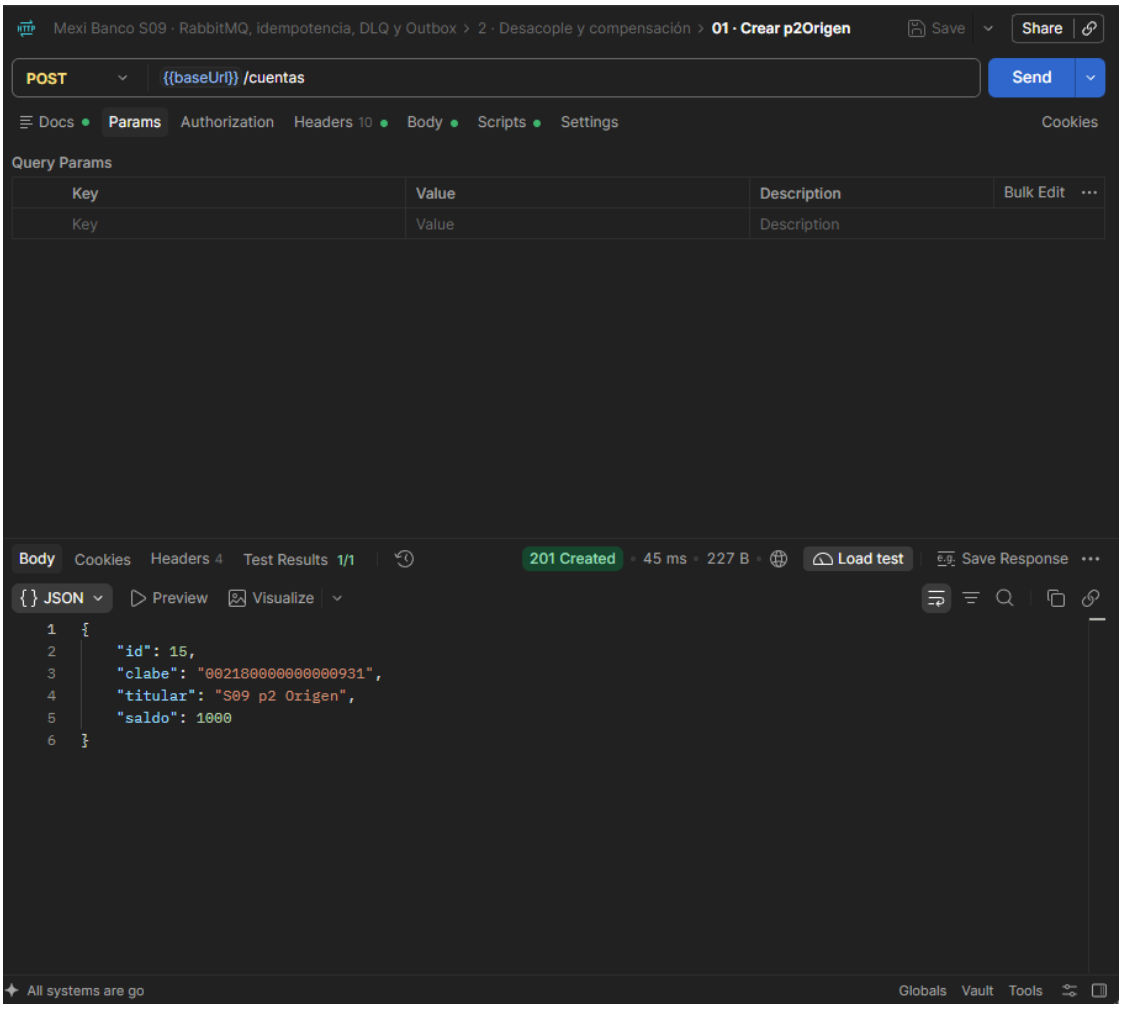


Figura 24: Origen 931 del escenario de acumulación.

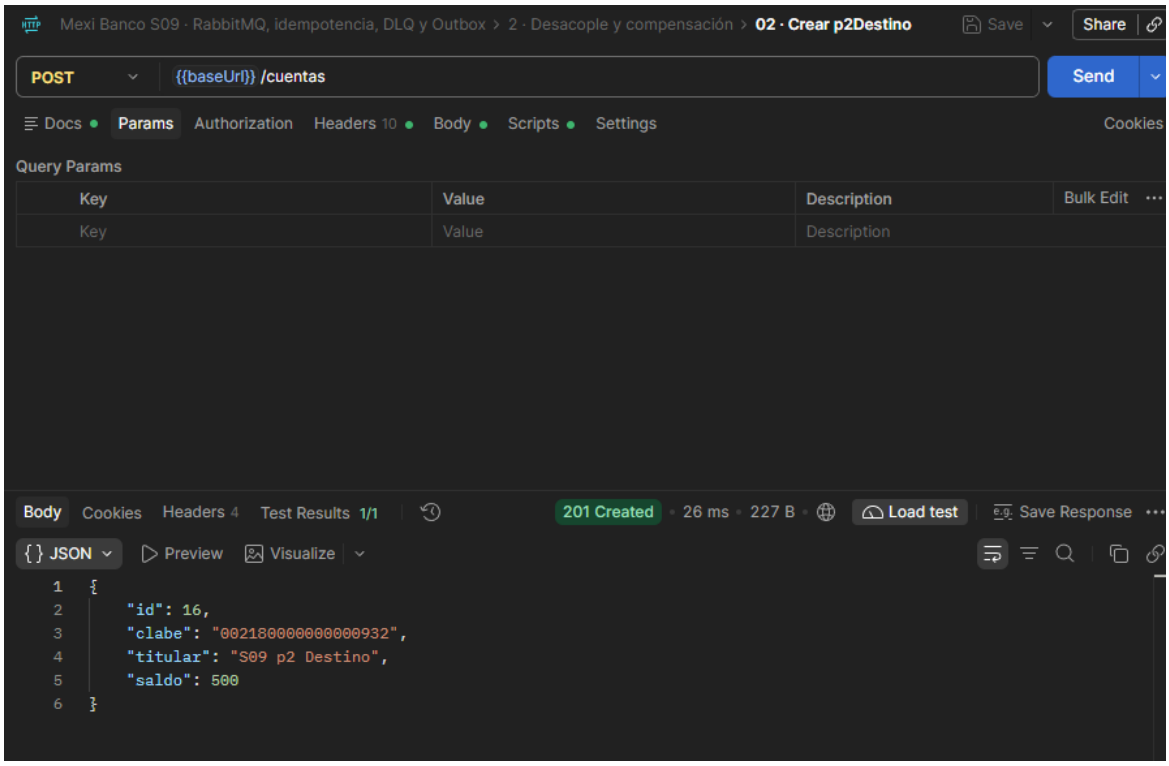


Figura 25: Destino 932 del escenario de acumulación.

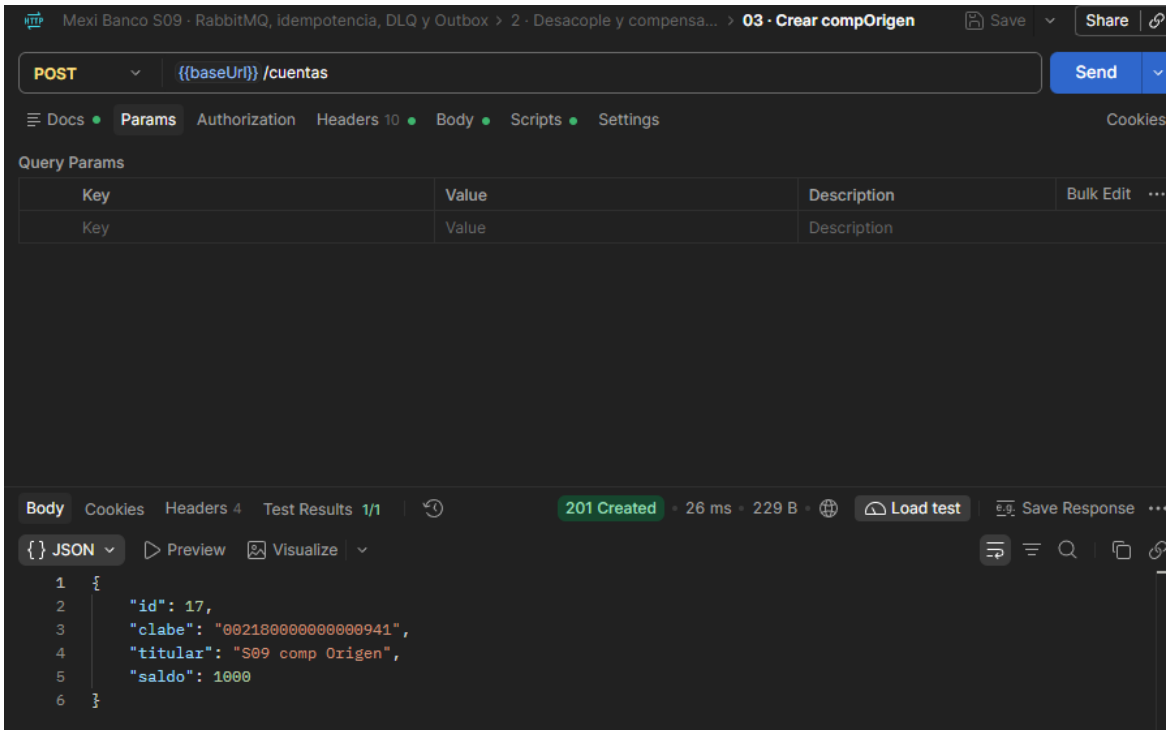


Figura 26: Origen 941 del escenario de compensación.

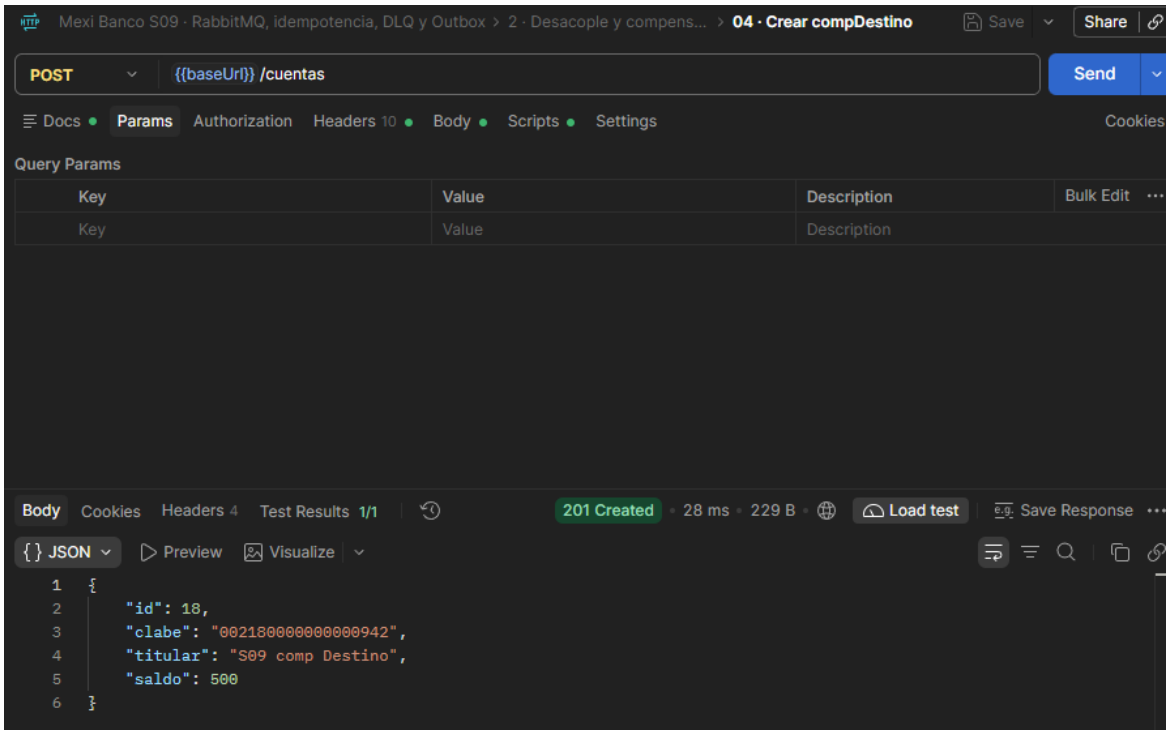


Figura 27: Destino 942 del escenario de compensación.

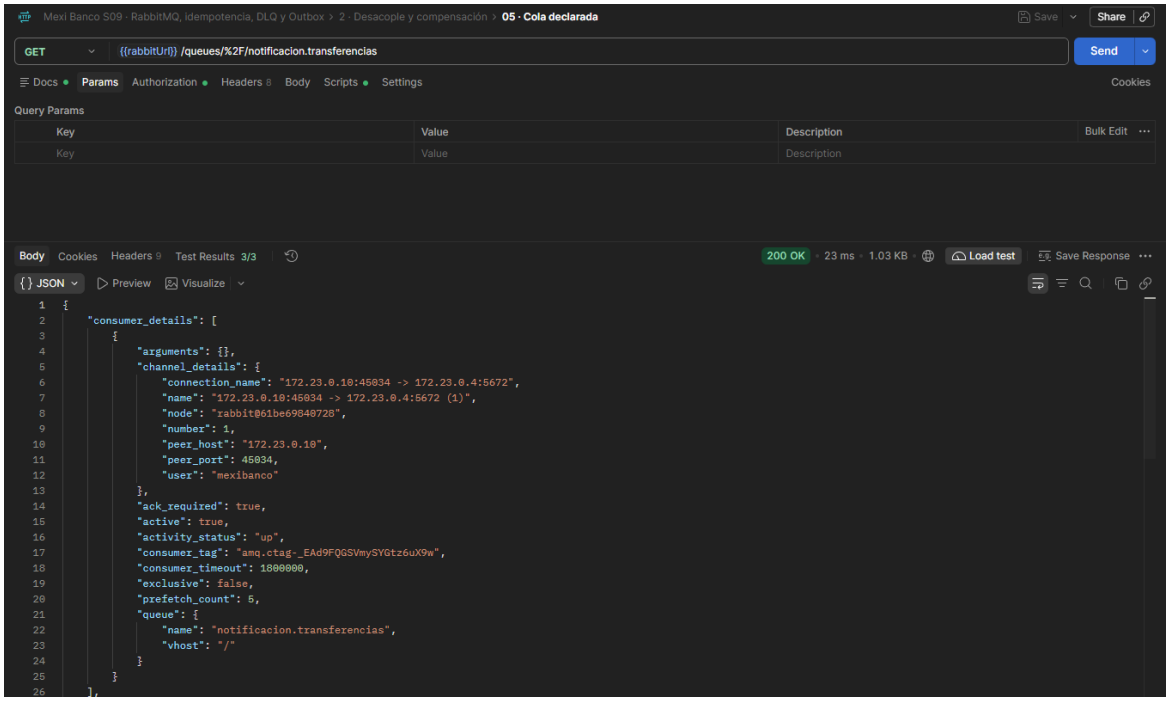


Figura 28: Cola declarada y datos de su consumidor.

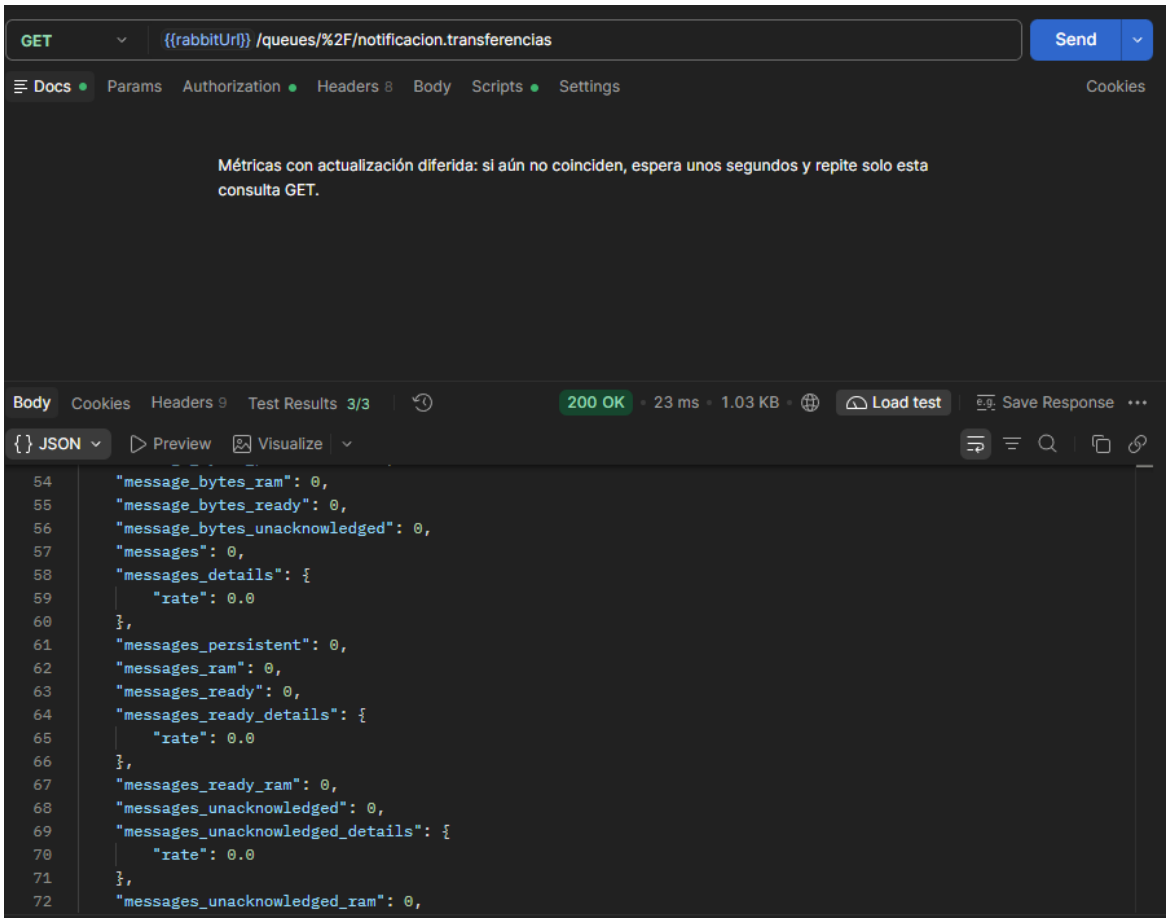


Figura 29: Cola inicialmente sin mensajes pendientes.



Figura 30: Detención del consumidor con la cola ya existente.

Con notificacion detenido se hicieron tres transferencias de 11, 12 y 13. Las tres respondieron 201 y quedaron COMPLETADAS, con identificadores 27, 28 y 29. La cola mostró tres mensajes Ready y ningún consumidor. Ready significa que el mensaje espera ser entregado; no significa que el aviso ya se haya guardado.

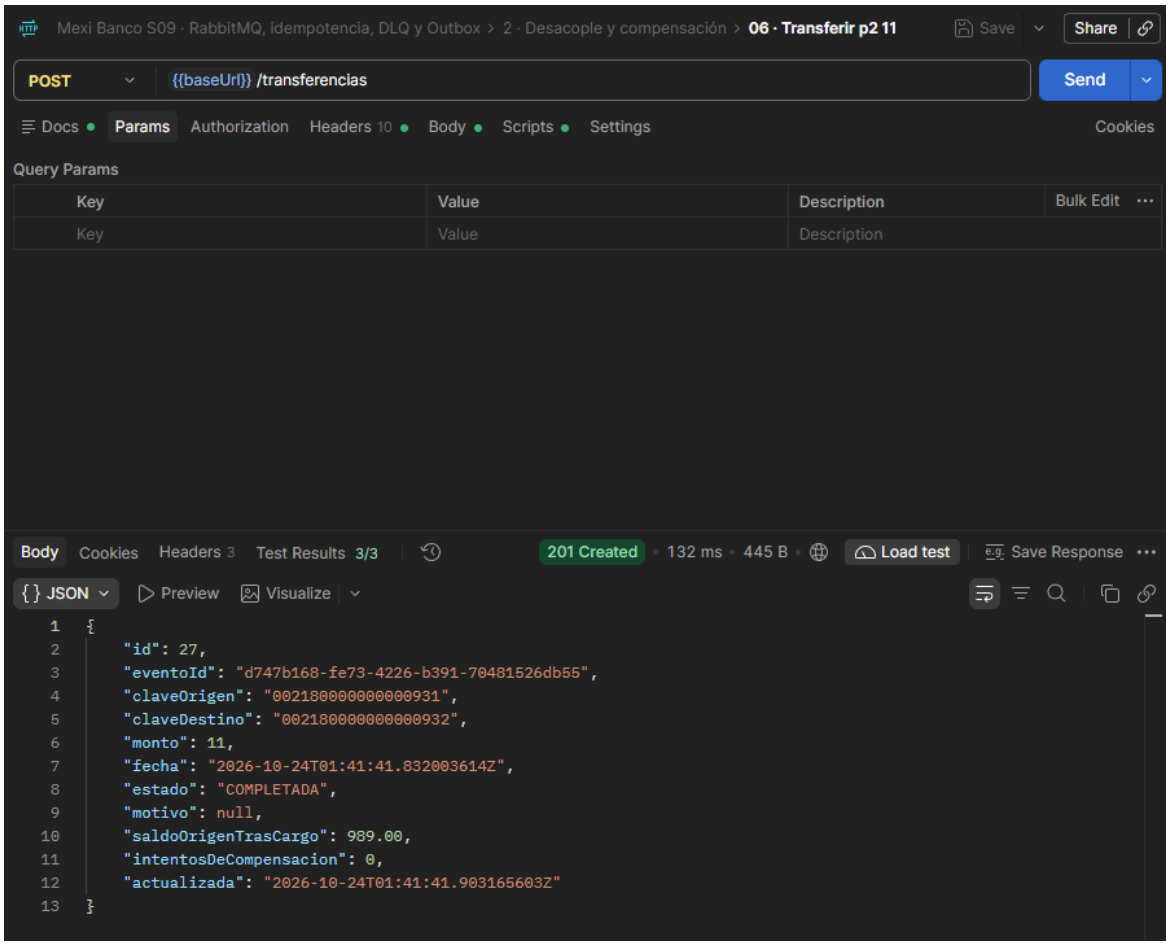


Figura 31: Transferencia de 11 completada, con identificador 27.

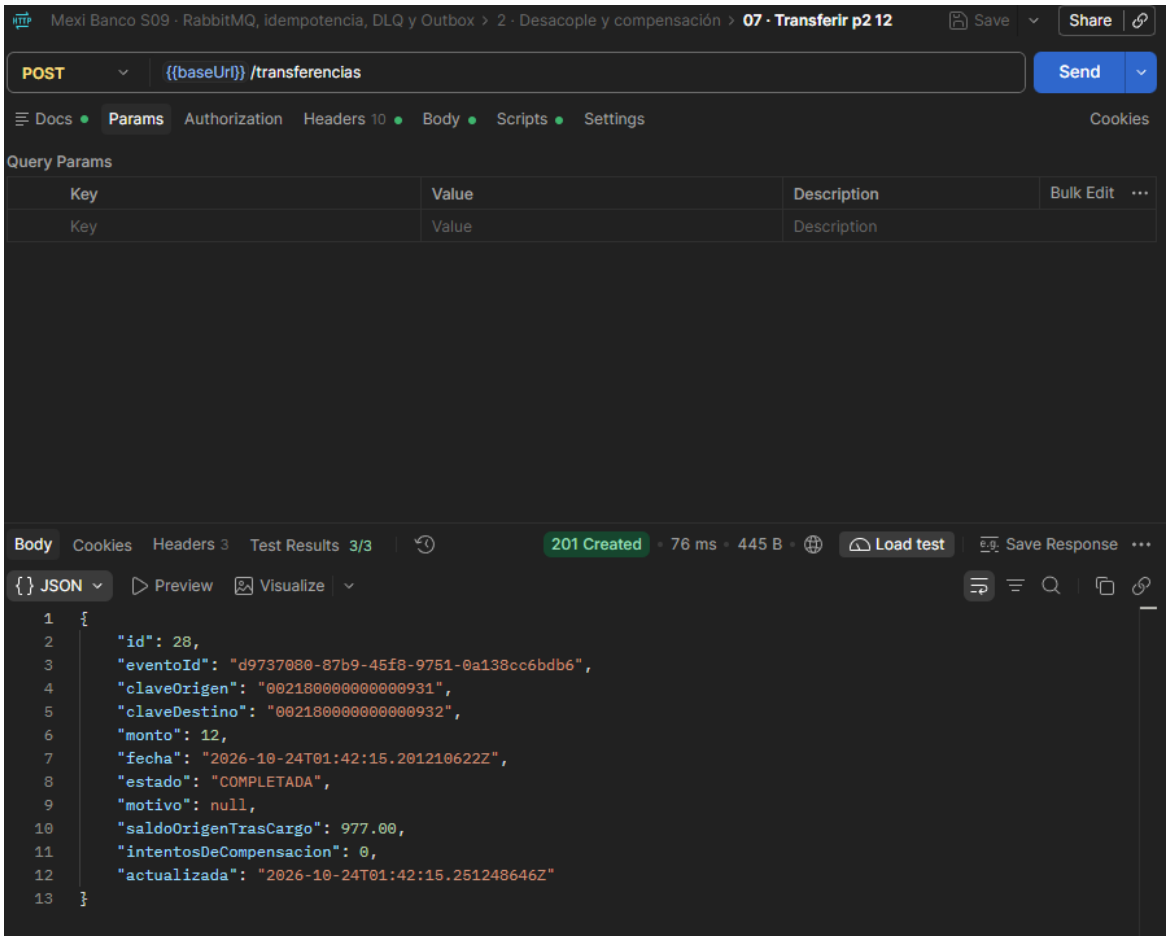


Figura 32: Transferencia de 12 completada, con identificador 28.

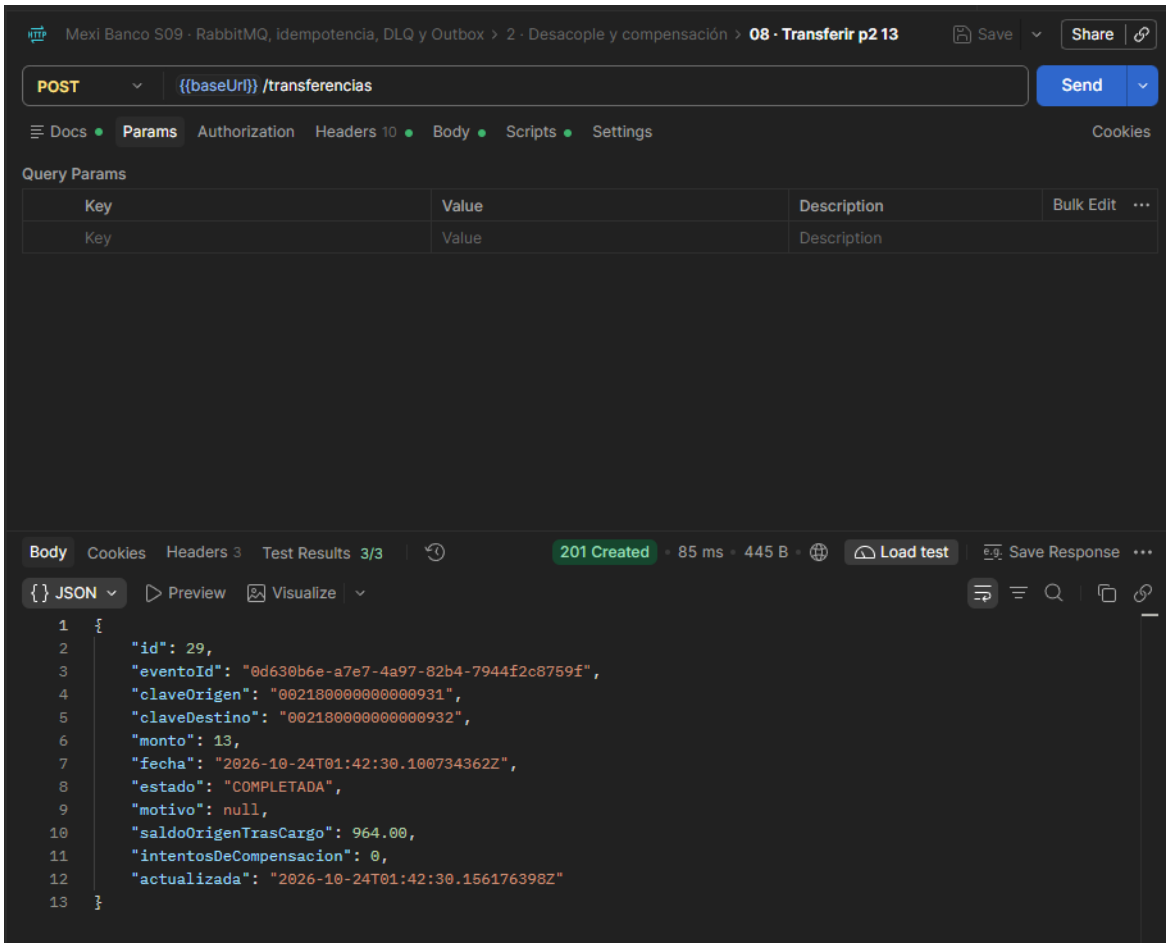


Figura 33: Transferencia de 13 completada, con identificador 29.

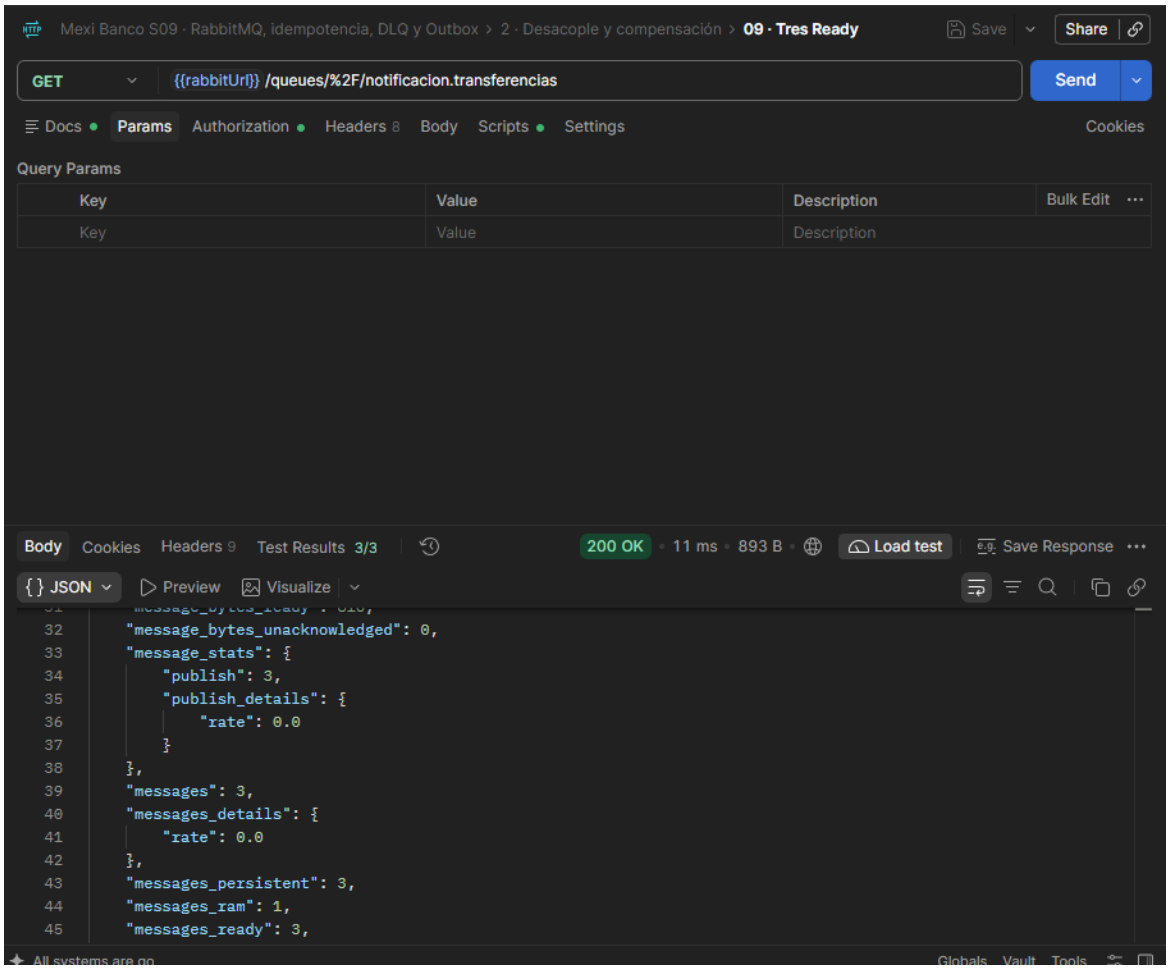


Figura 34: Tres mensajes Ready acumulados en la cola.

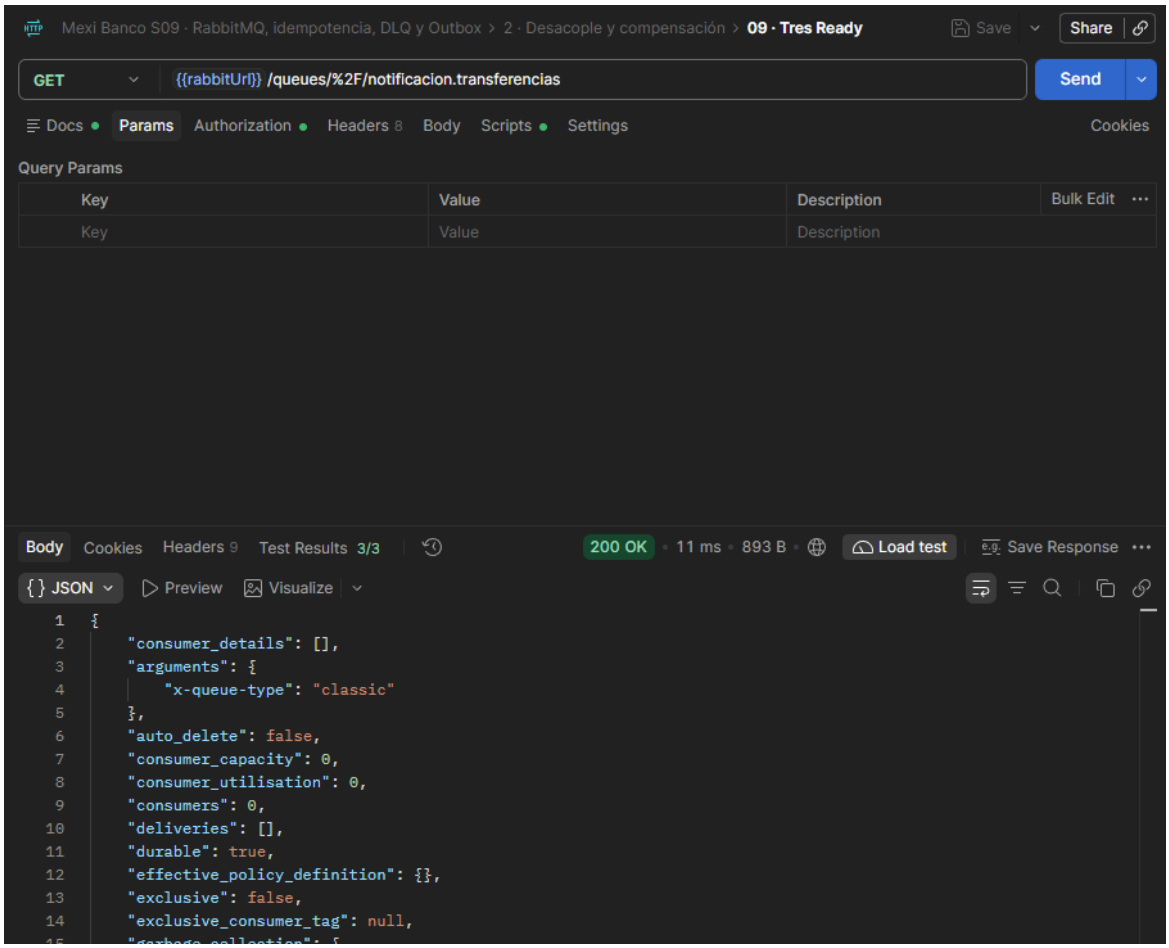


Figura 35: La cola no tiene consumidores mientras notificacion está detenido.

Después se reanudó notificacion y el total de mensajes pendientes bajó a cero. Al consultar los avisos aparecieron tres para el origen y tres para el destino. Los saldos finales fueron 964 y 536, que corresponden a mover 36 en total. Con esto se ve el desacoplamiento temporal: las transferencias terminaron primero y los avisos se procesaron al recuperar el consumidor.



Figura 36: Reanudación del consumidor.

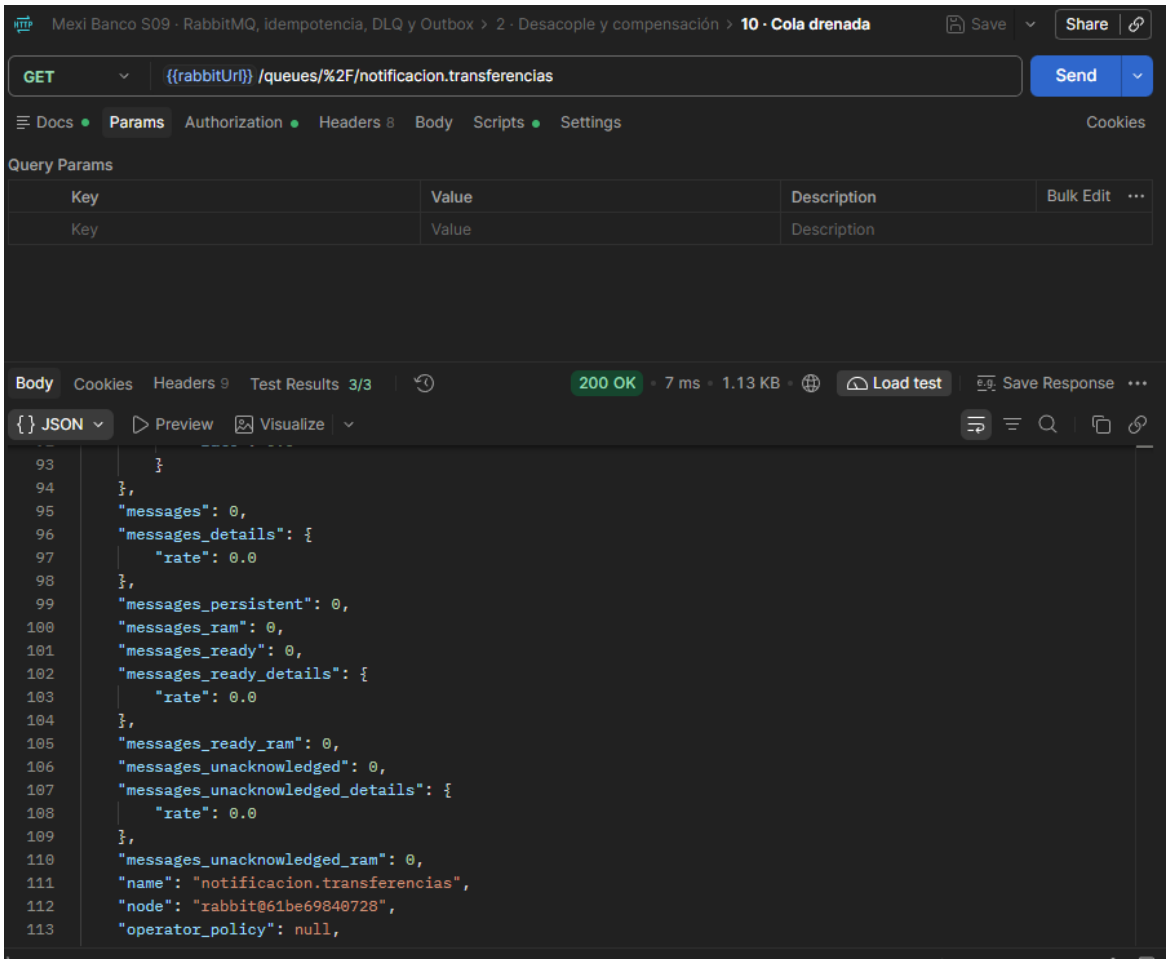


Figura 37: La cola queda vacía después del consumo.

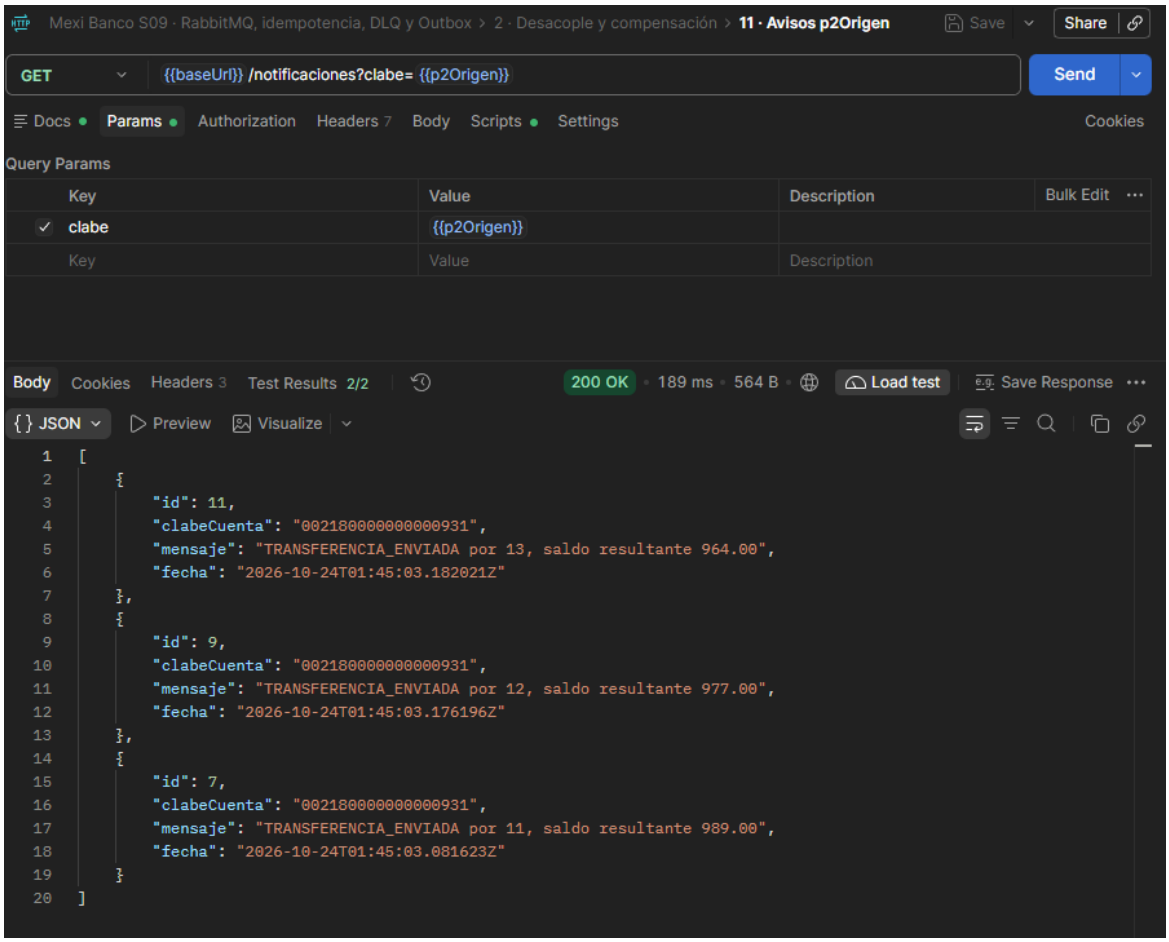


Figura 38: Tres avisos del origen por 11, 12 y 13.

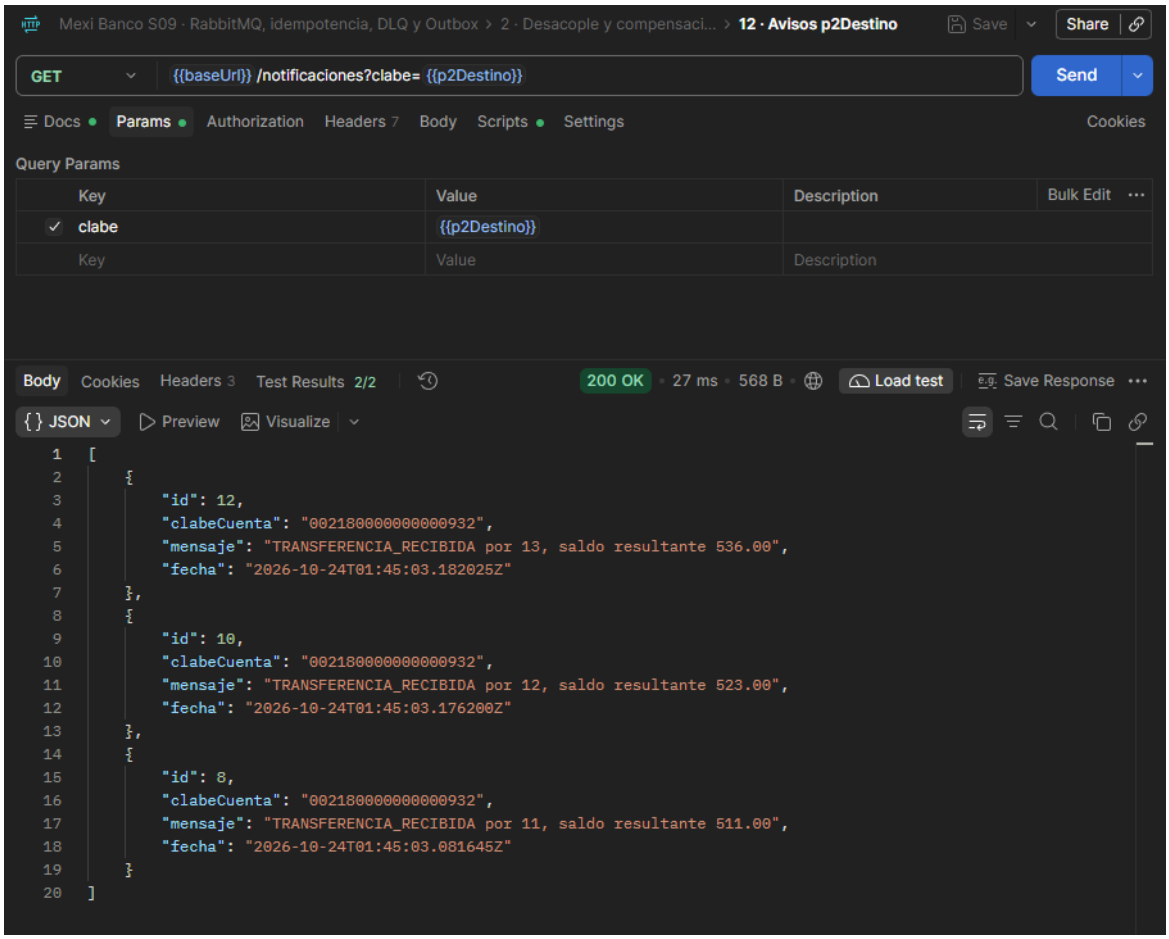


Figura 39: Tres avisos del destino por los mismos montos.

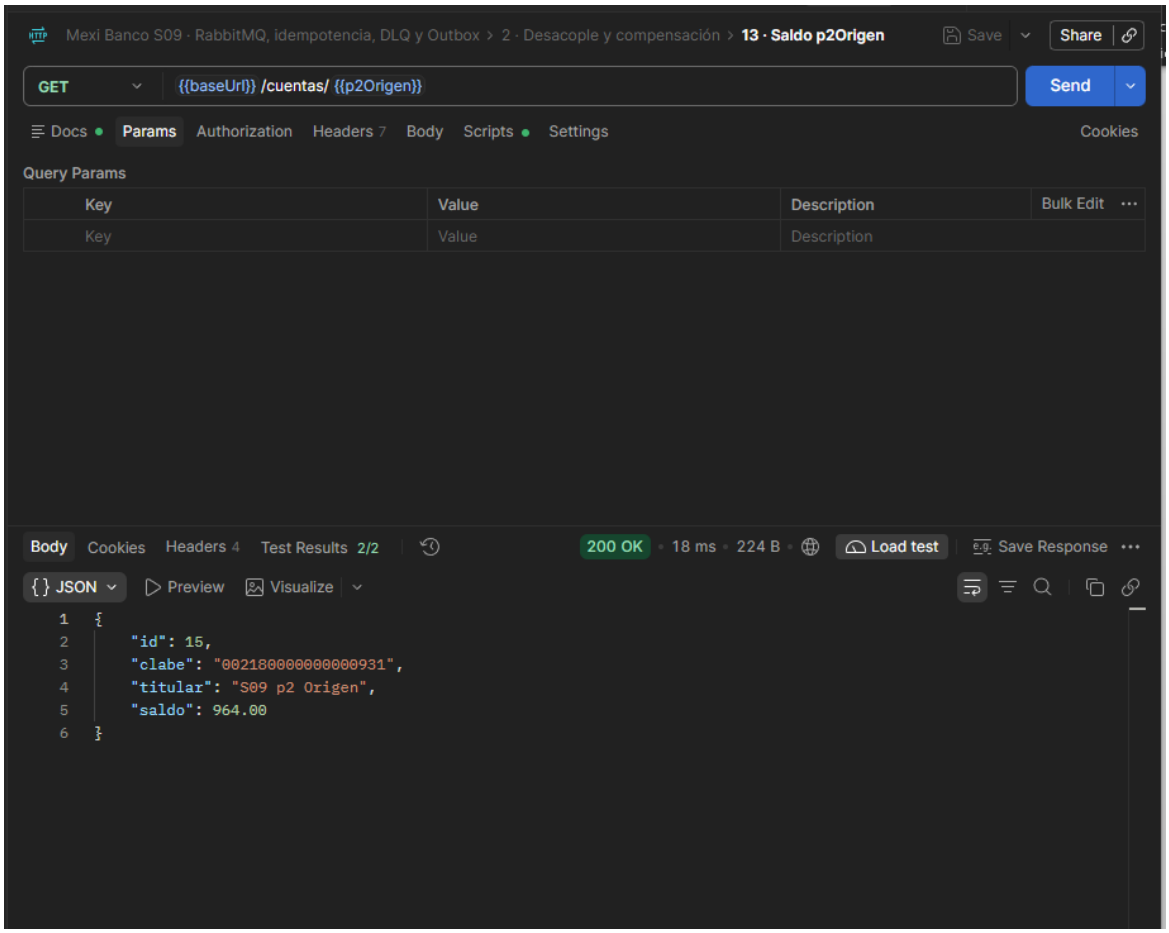


Figura 40: Saldo final del origen: 964.

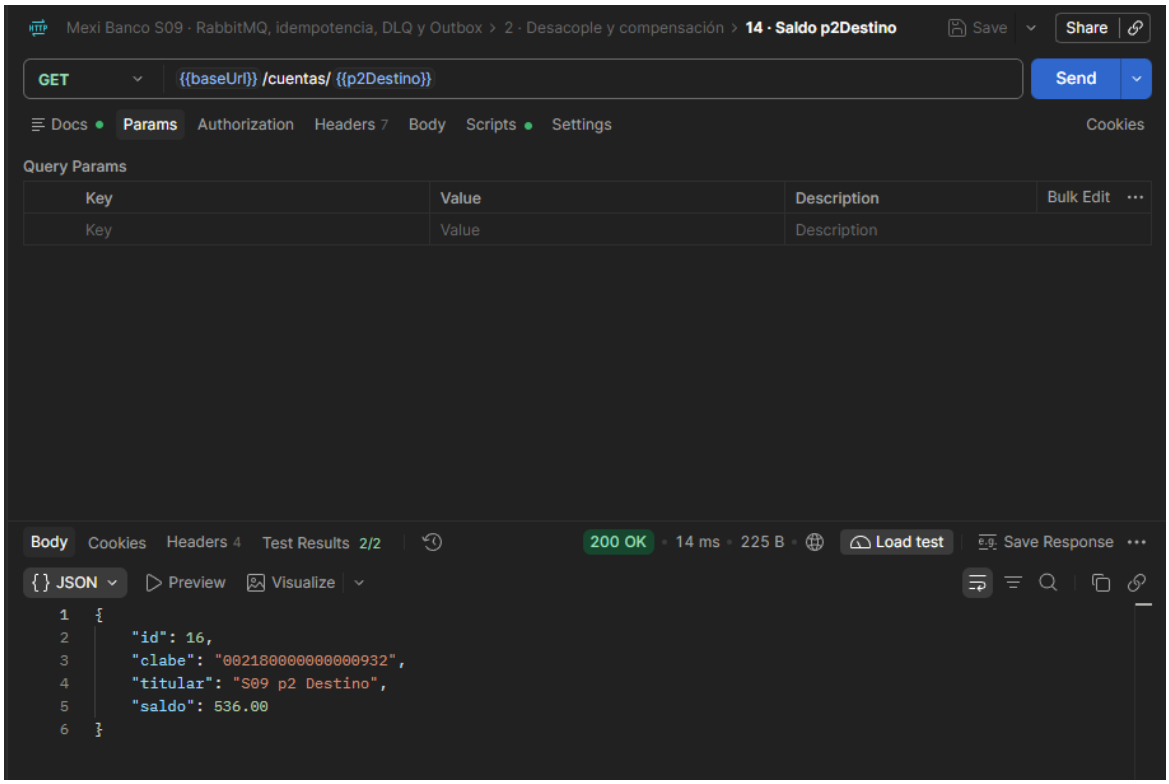


Figura 41: Saldo final del destino: 536.

2.1 Evento de una transferencia compensada

Para probar el otro tipo de evento se configuró FALLAR_ABONO_A_CLABE hacia la cuenta 942 y se recreó transferencia. La operación fue de 150. Inicialmente respondió 502 con COMPENSACION_PENDIENTE, porque todavía no se confirmaba la devolución. Durante la corrección y los reintentos, la consulta posterior terminó mostrando COMPENSADA; el contador registrado fue de 73 intentos.

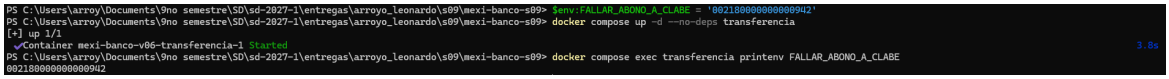


Figura 42: Configuración de la falla controlada para el destino 942.

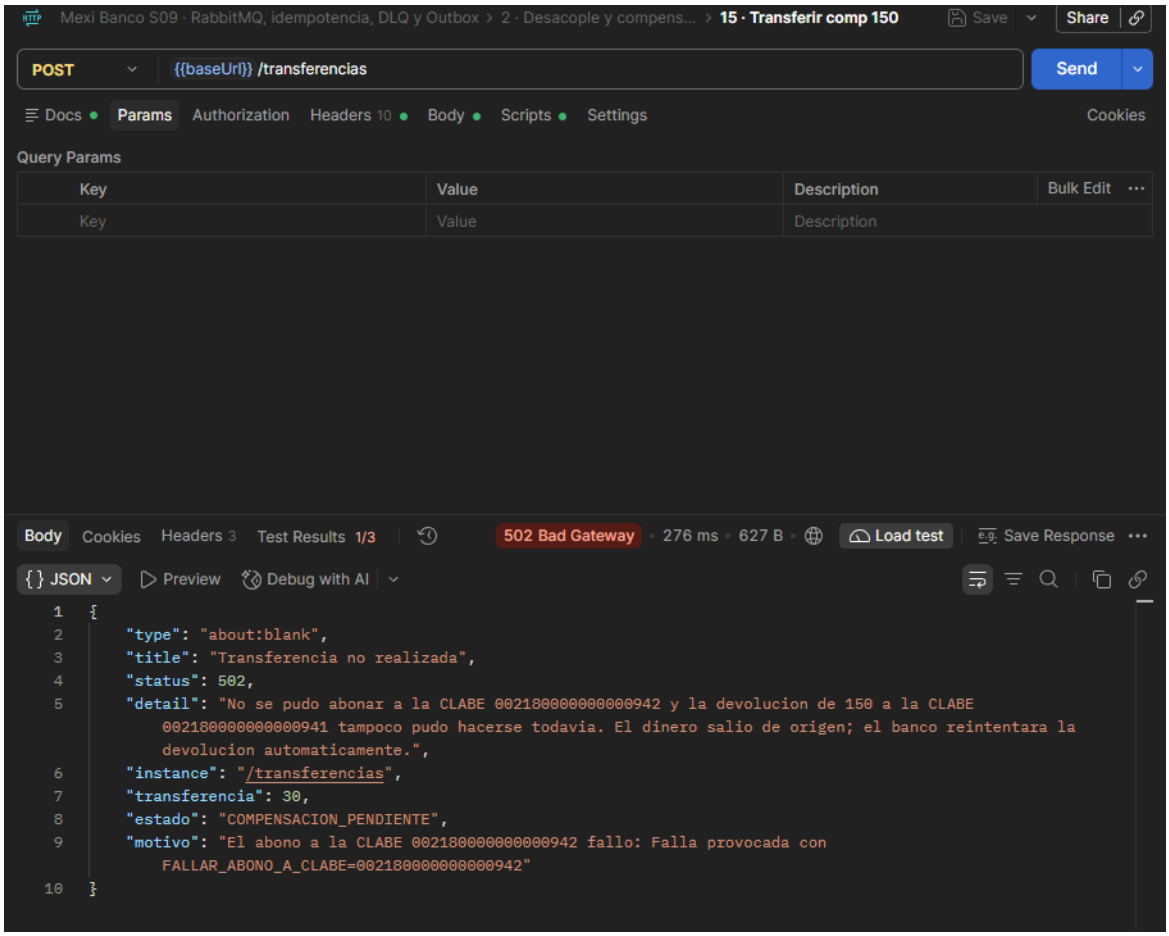


Figura 43: Respuesta inicial con COMPENSACION_PENDIENTE para la transferencia 30.

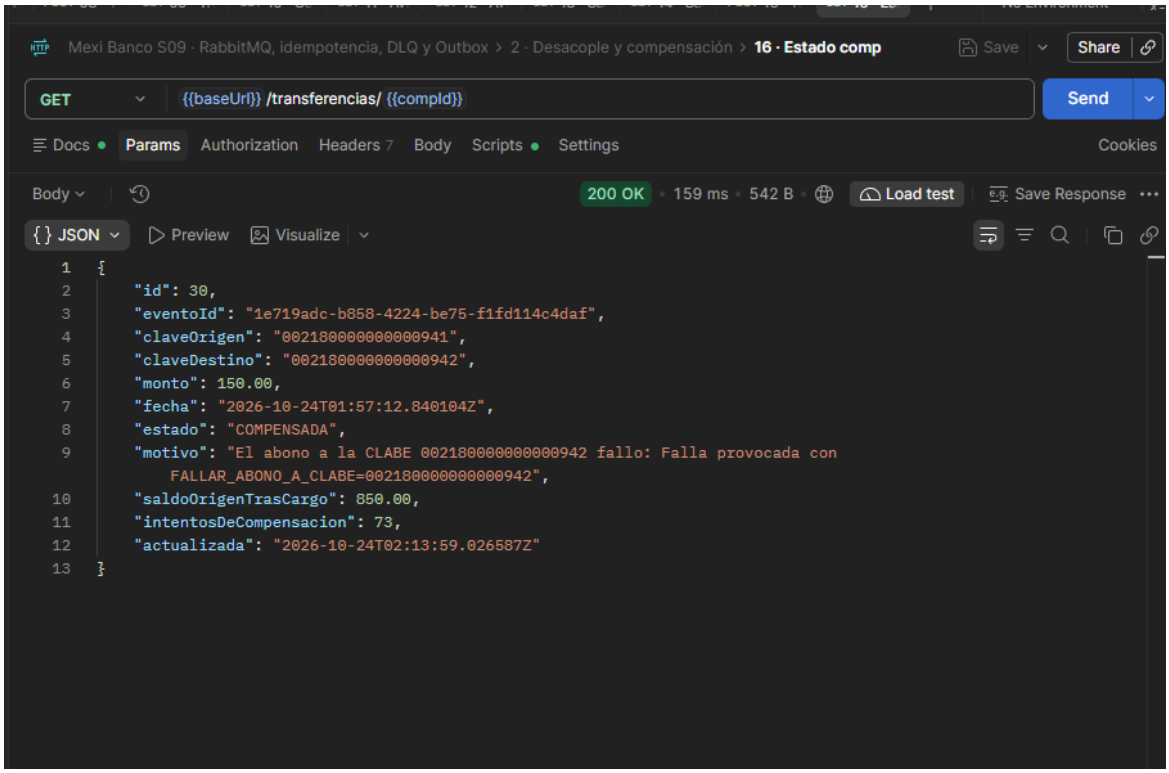


Figura 44: Consulta posterior de la transferencia 30 en COMPENSADA.

La notificación del origen fue de DEVOLUCION_TRANSFERENCIA por 150 y no se generó un aviso para el destino, que continuó en 500. Sin embargo, el origen aparece con 6400, no con sus 1000 iniciales. Este resultado no permite dar por correcta la conservación del dinero: el estado COMPENSADA y la existencia del aviso no bastan para validar el saldo. La cifra obliga a revisar que la devolución y sus reintentos apliquen una sola vez el mismo efecto; estas capturas por sí solas no identifican la causa exacta del incremento.

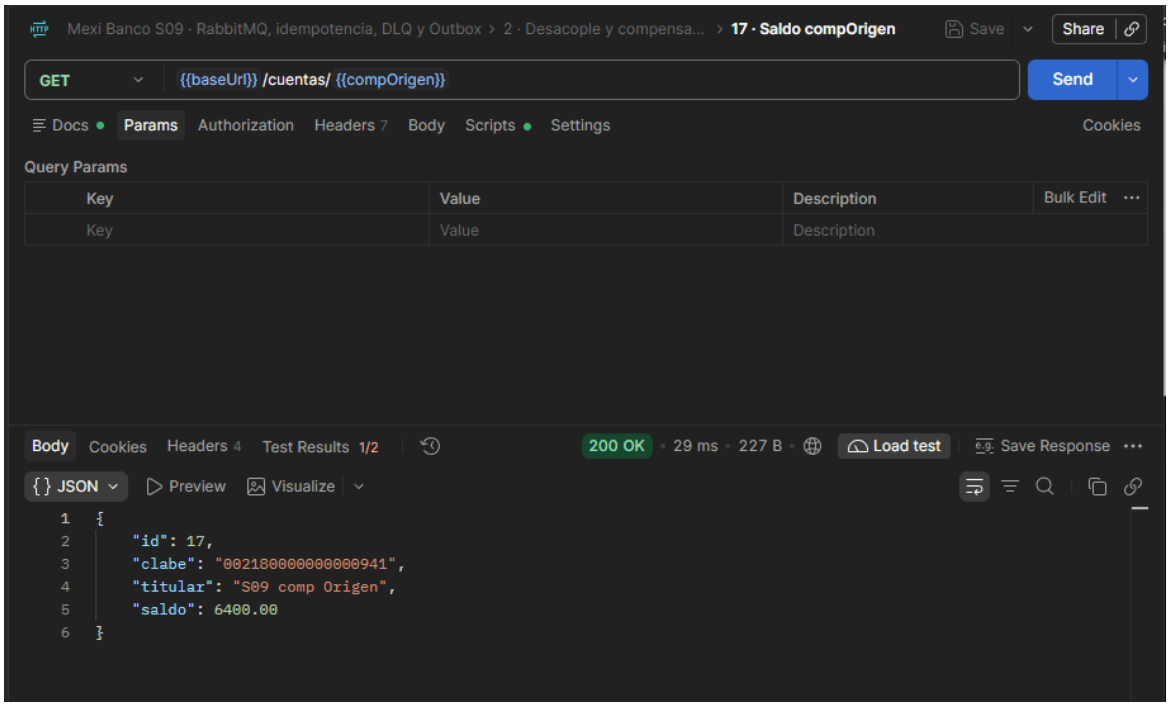


Figura 45: Saldo observado del origen de compensación: 6400.

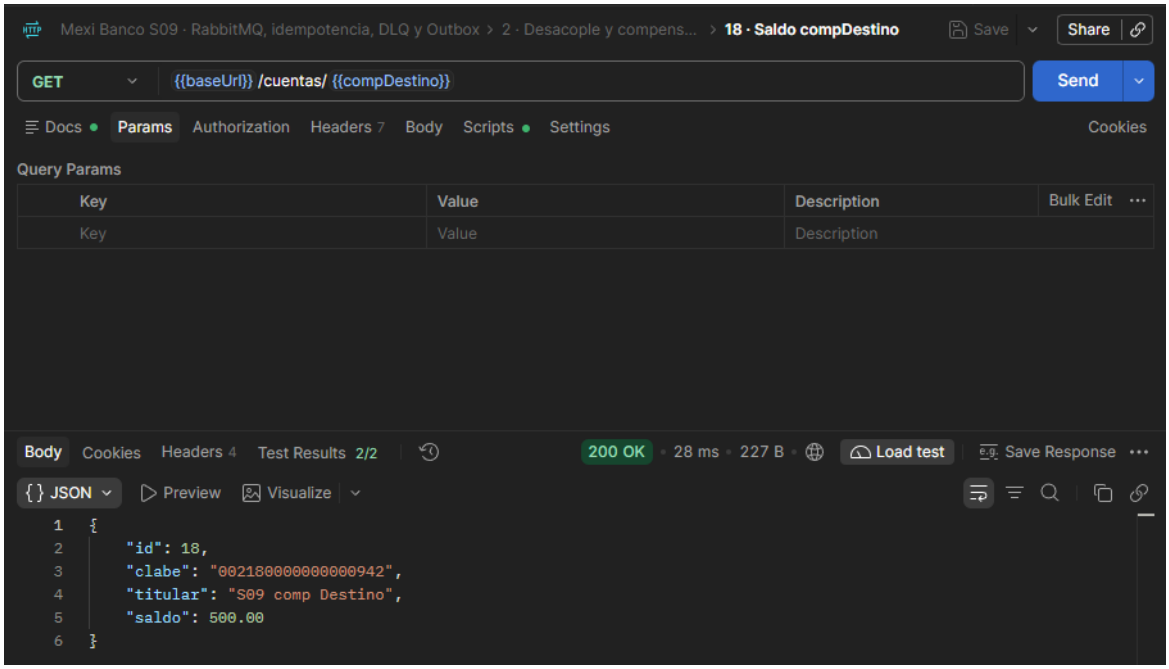


Figura 46: El destino de compensación conserva 500.

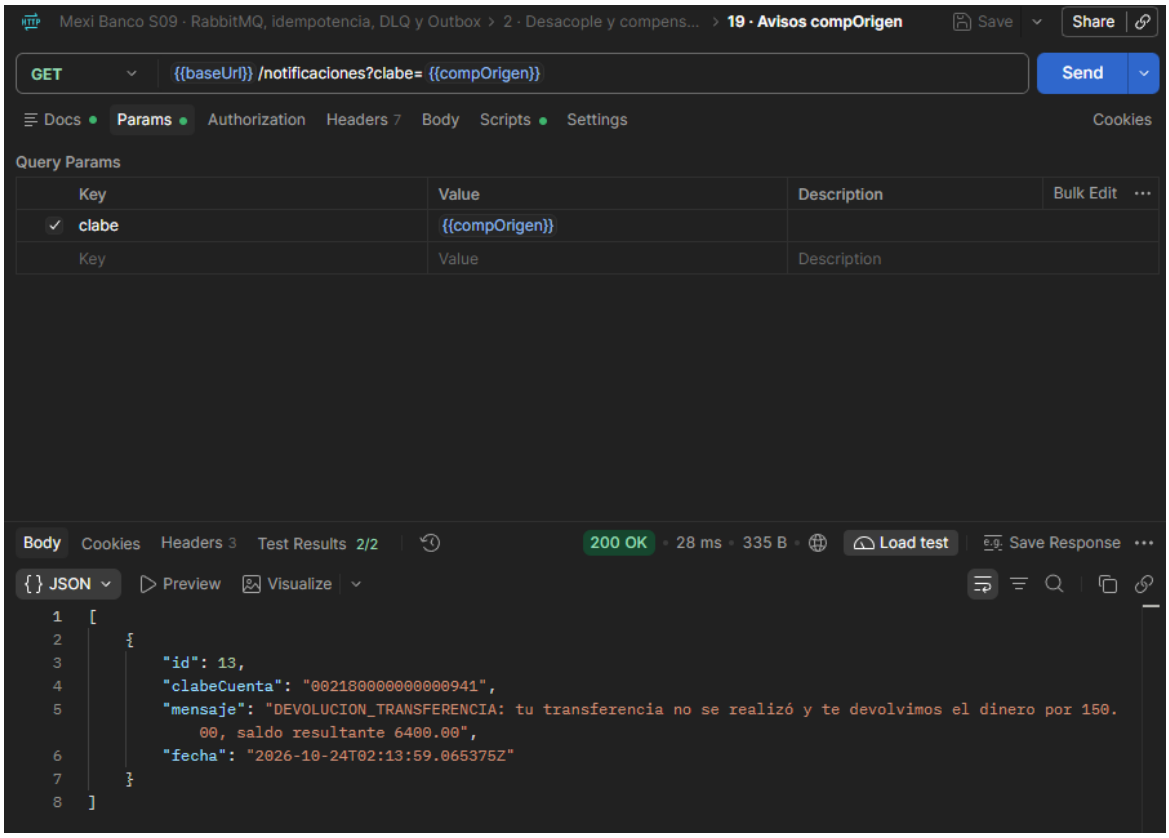


Figura 47: Aviso de devolución por 150 con saldo resultante 6400.

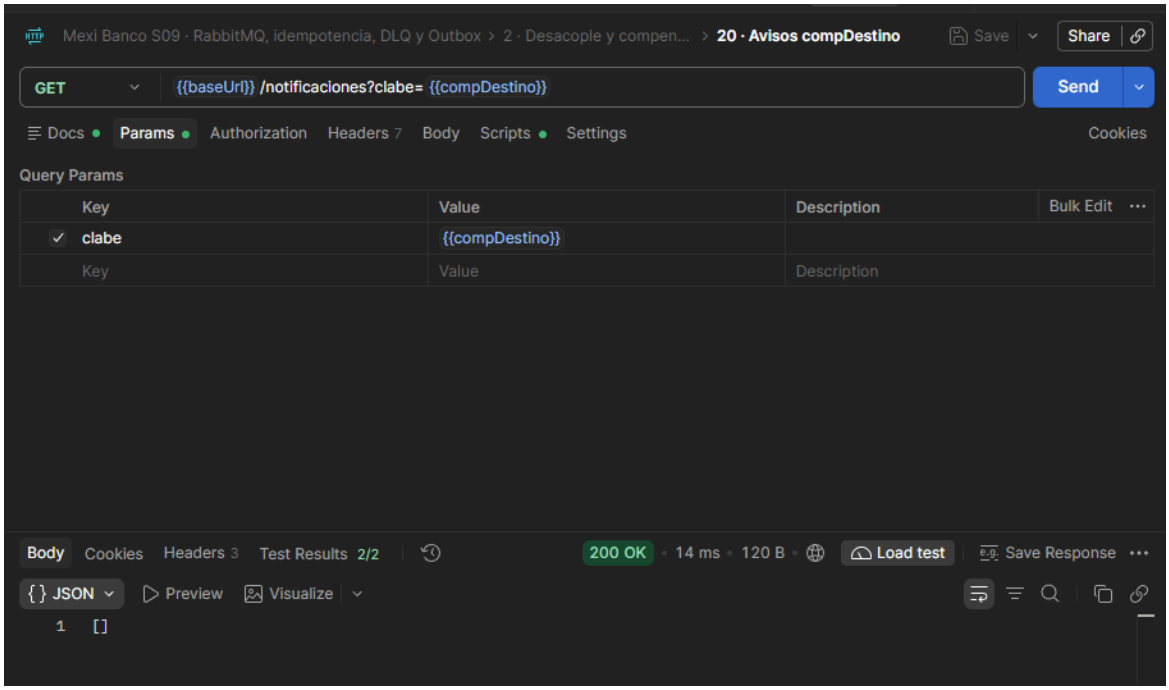


Figura 48: El destino no tiene avisos de una transferencia recibida.

En el código, el evento de reversión se publica después de guardar COMPENSADA. Su propósito es avisar de una devolución, no pedir al consumidor que vuelva a devolver

dinero. notificacion solamente escribe avisos; la corrección de los saldos pertenece al flujo de transferencia y cuenta.

3 Mensajes duplicados e idempotencia

Un broker puede volver a entregar un mensaje. Por ejemplo, el consumidor puede confirmar su base de datos y caer antes de enviar el ACK. Para RabbitMQ todavía falta la confirmación y corresponde reentregar. La solución no consiste en asumir que nunca habrá repetidos, sino en hacer que repetir un evento no repita el efecto de negocio.

3.1 Duplicado antes de la protección

Se crearon las cuentas 951 y 952 con 1000 y 500. Con el consumidor detenido se transfirieron 17 y se obtuvo la transferencia 32. Después se leyó el evento con requeue, para devolverlo a la cola, y se publicó otra copia del mismo contenido conservando eventoId. La cola pasó de un mensaje a dos. Esta segunda publicación no ejecutó otra transferencia: solamente duplicó su aviso.

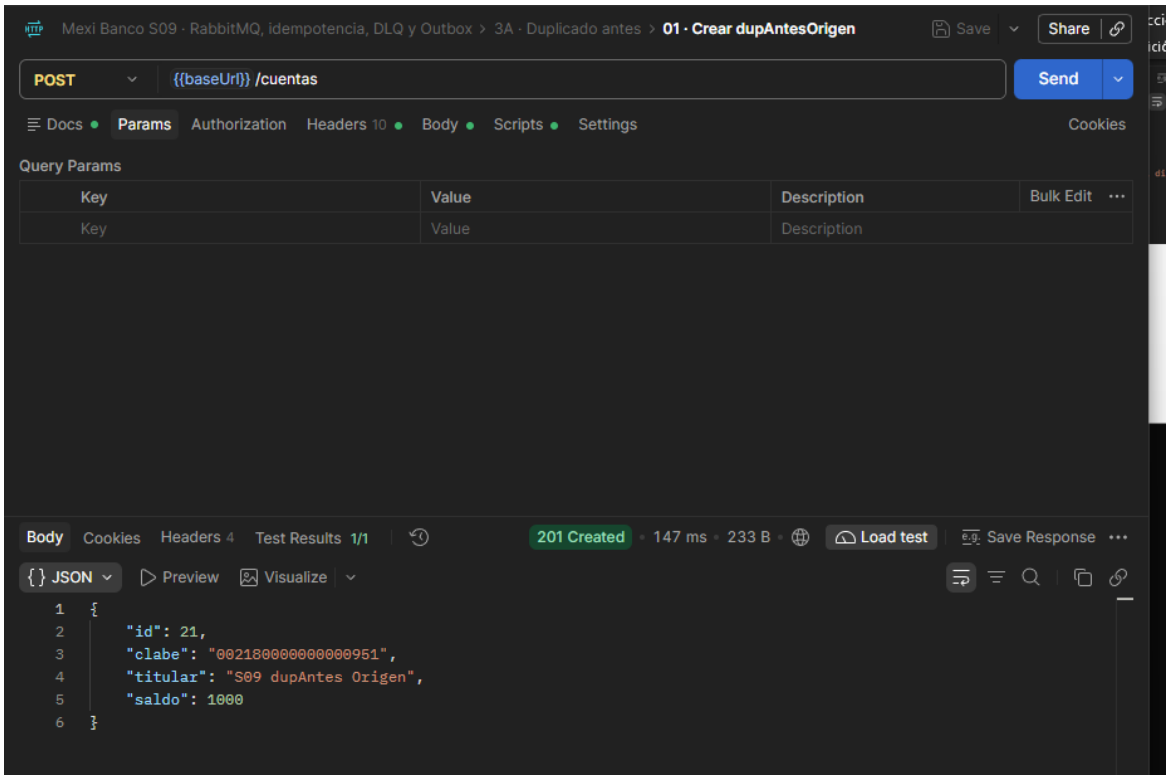


Figura 49: Creación del origen 951 para probar duplicados.

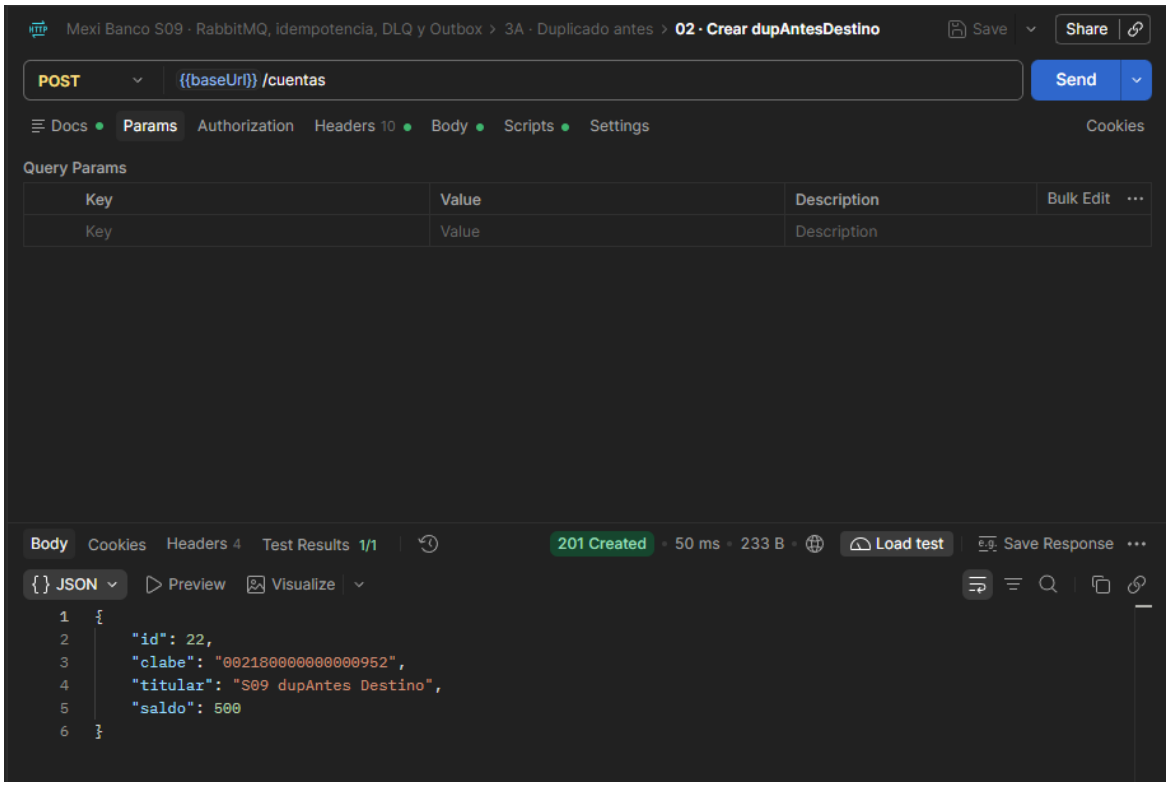


Figura 50: Creación del destino 952 para probar duplicados.



Figura 51: Consumidor detenido antes de duplicar el evento.

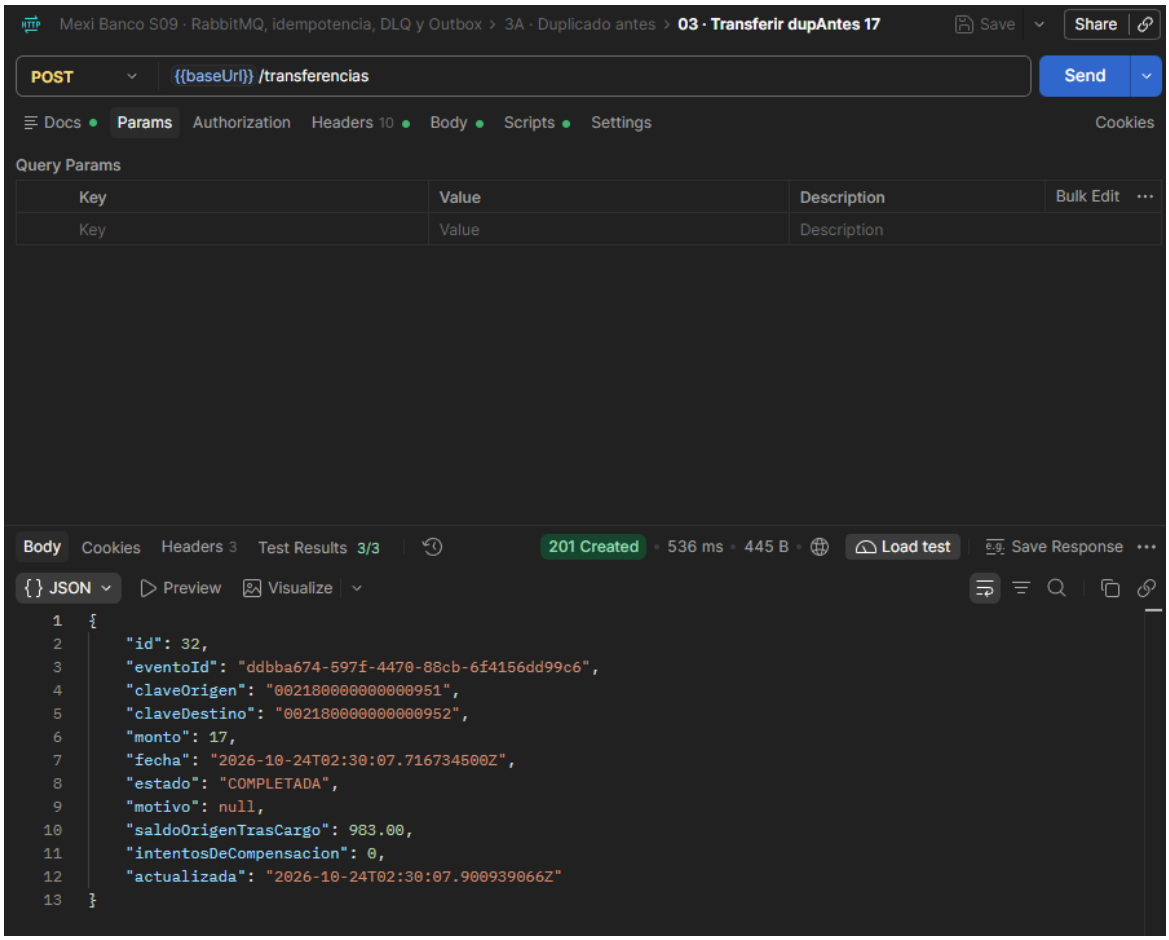


Figura 52: Transferencia 32 completada por 17.

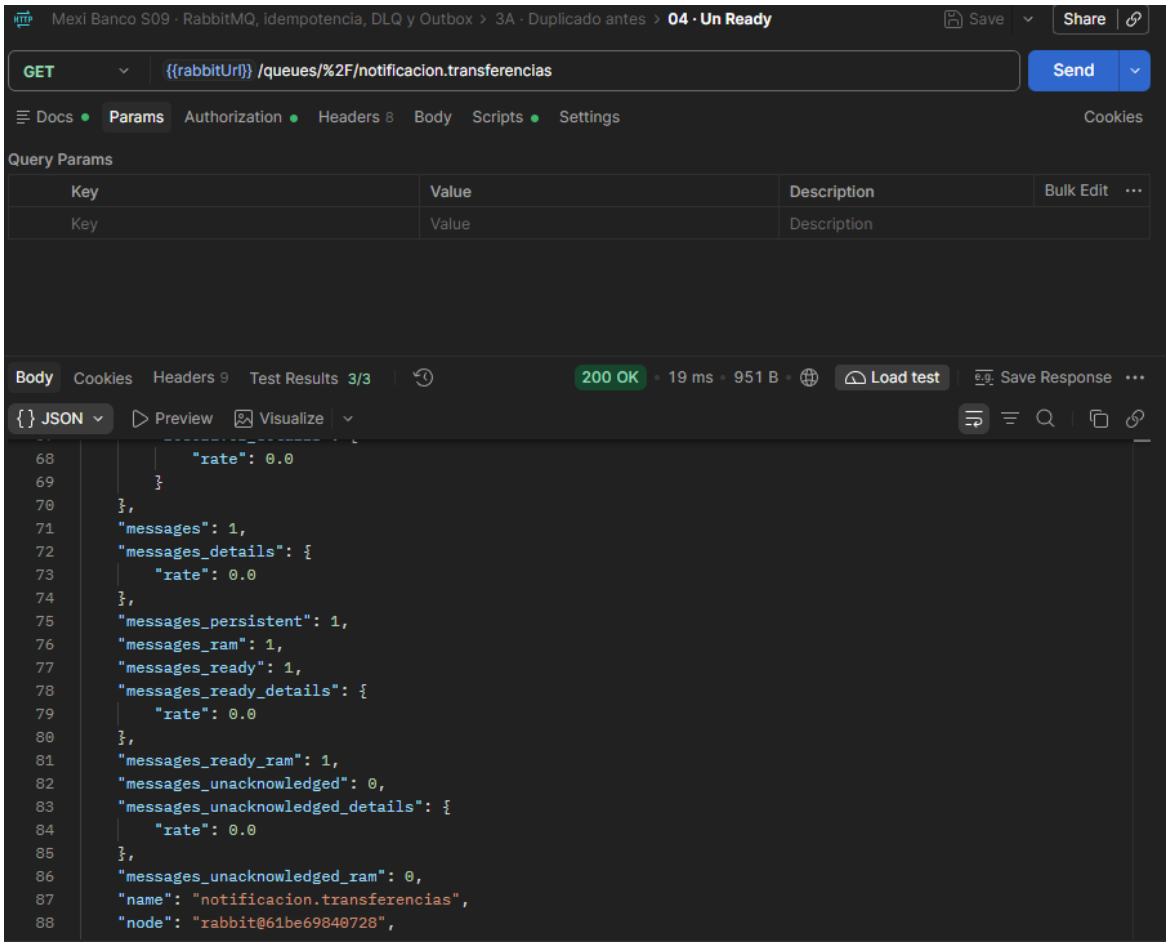


Figura 53: Un mensaje Ready antes de republicar.

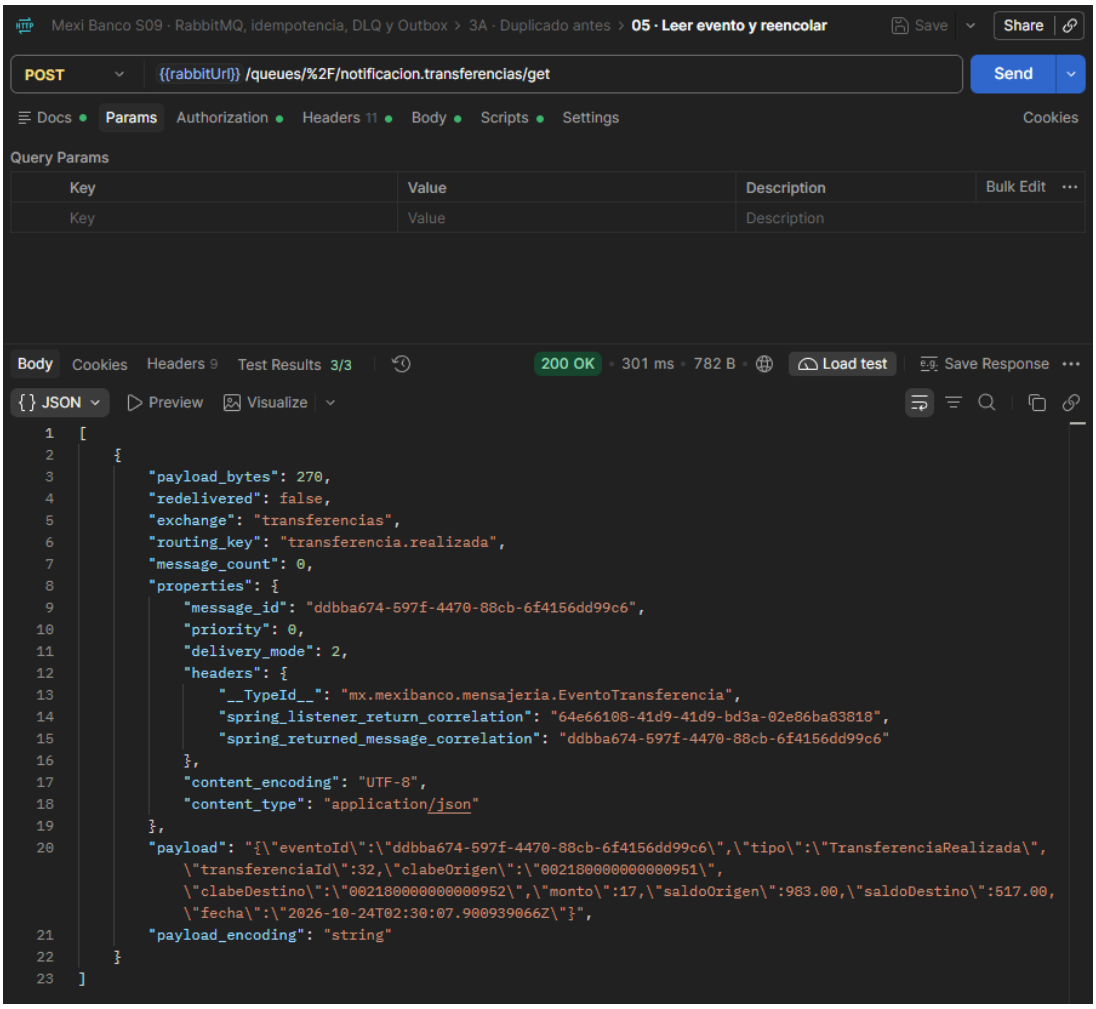


Figura 54: Lectura del evento con requeue para conservar el original.

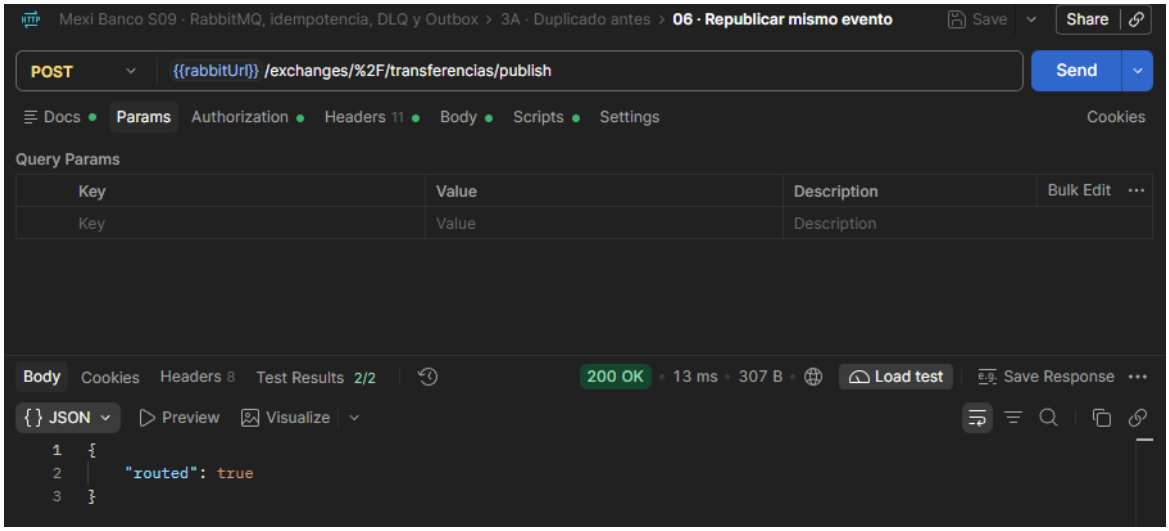


Figura 55: Publicación de una copia del mismo evento con ruteo exitoso.

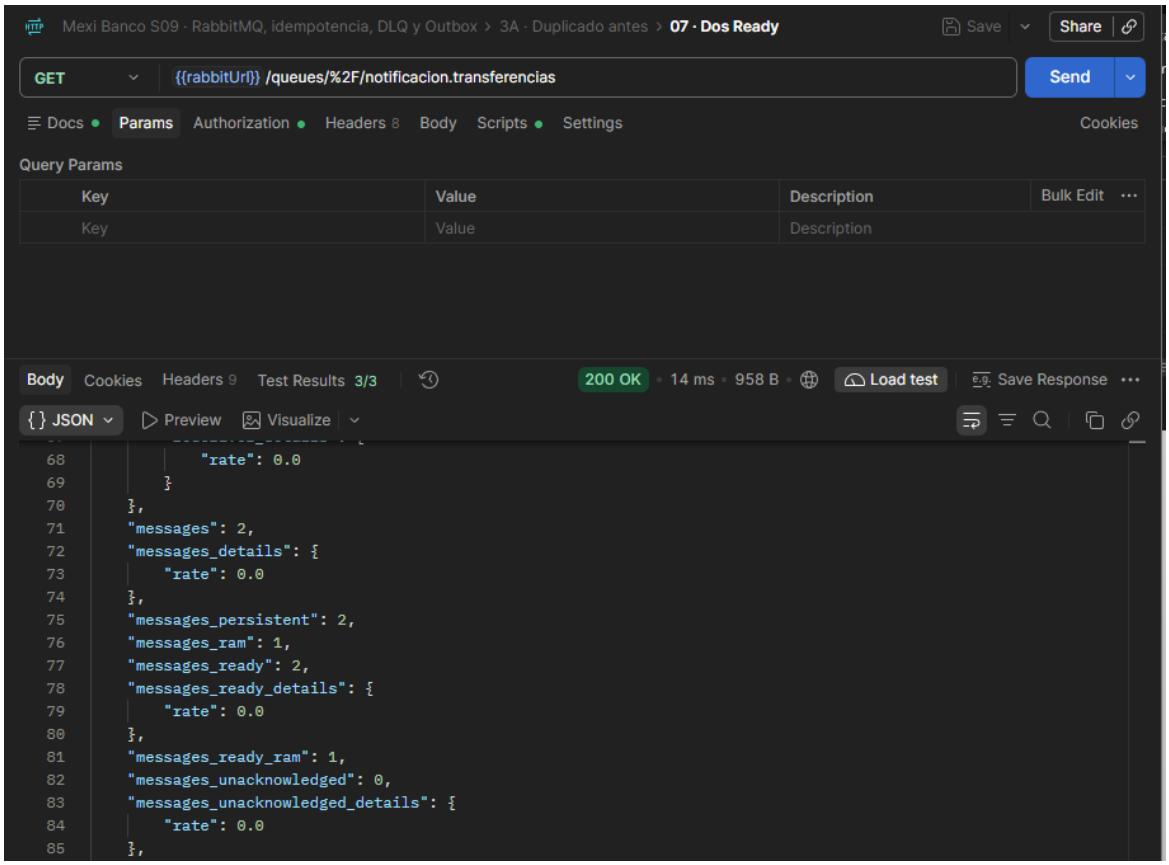


Figura 56: Dos mensajes Ready correspondientes al mismo evento.



Figura 57: Inicio del consumidor para procesar ambas entregas.

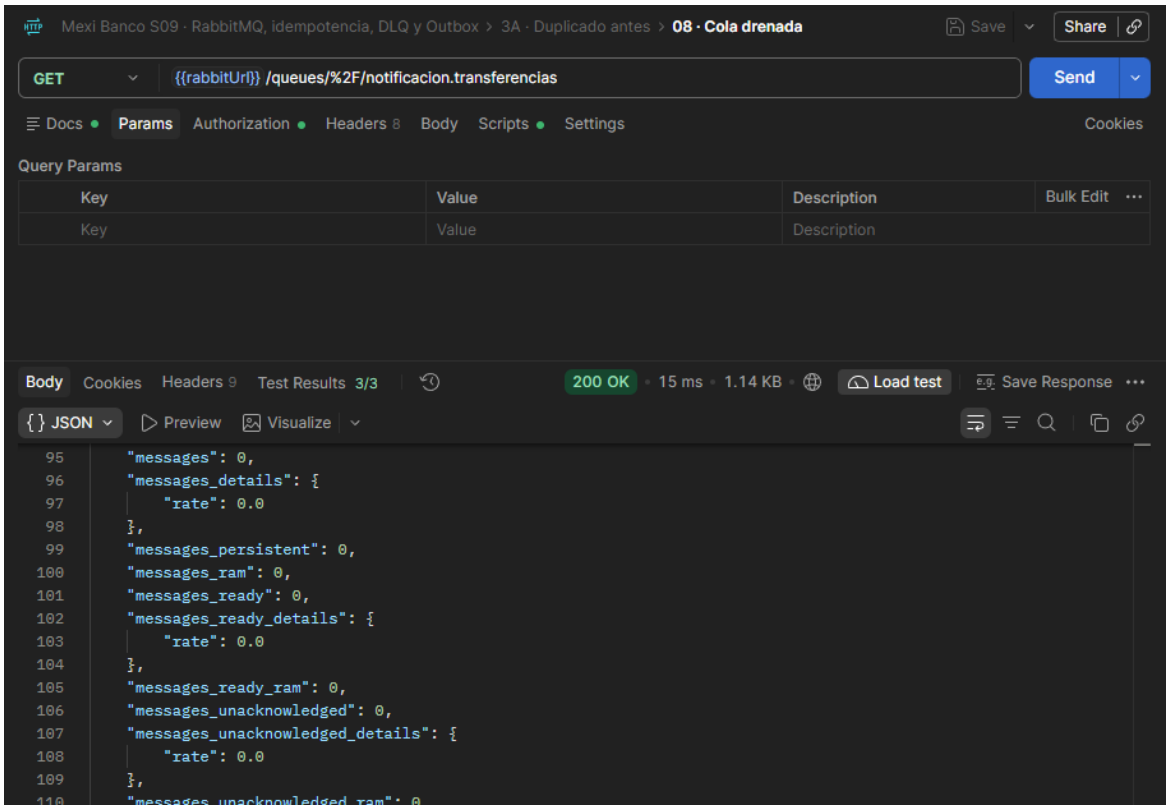


Figura 58: La cola queda vacía después del procesamiento.

Sin deduplicación aparecieron dos avisos iguales en el origen y dos en el destino. Los saldos, en cambio, fueron 983 y 517: el dinero se movió una sola vez. El efecto repetido estuvo en las notificaciones, que se generaban por cada entrega del mensaje.

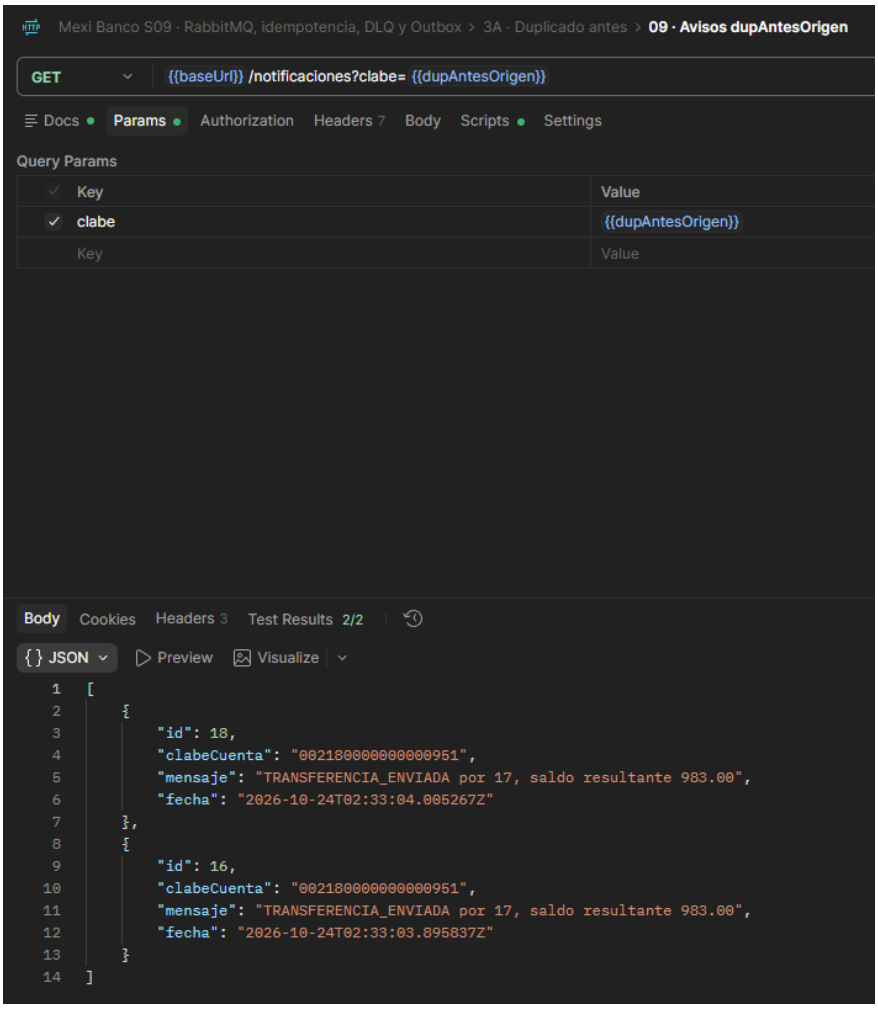


Figura 59: Dos notificaciones iguales en el origen antes de deduplicar.

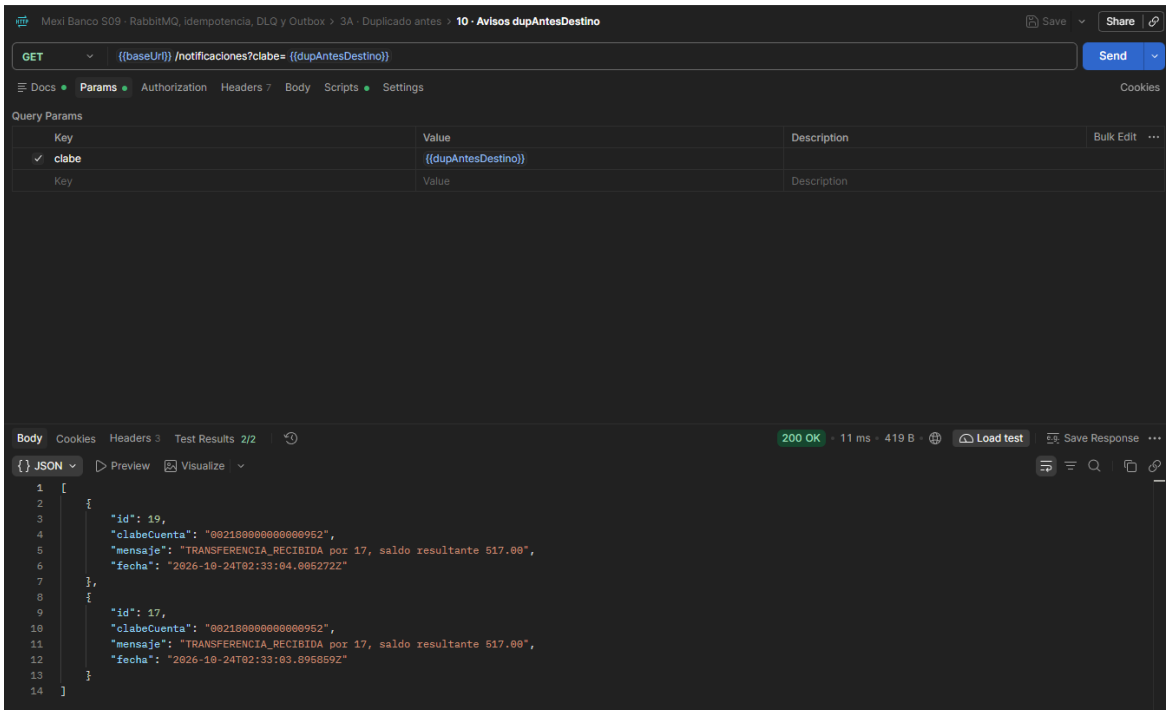


Figura 60: Dos notificaciones iguales en el destino antes de deduplicar.

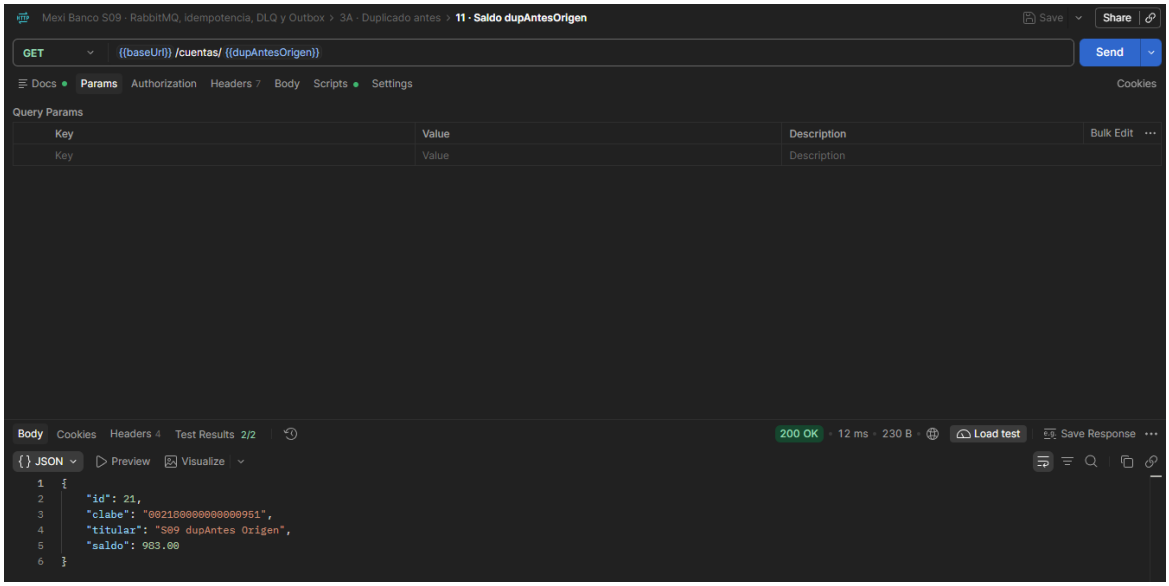


Figura 61: Saldo del origen después de una sola transferencia: 983.

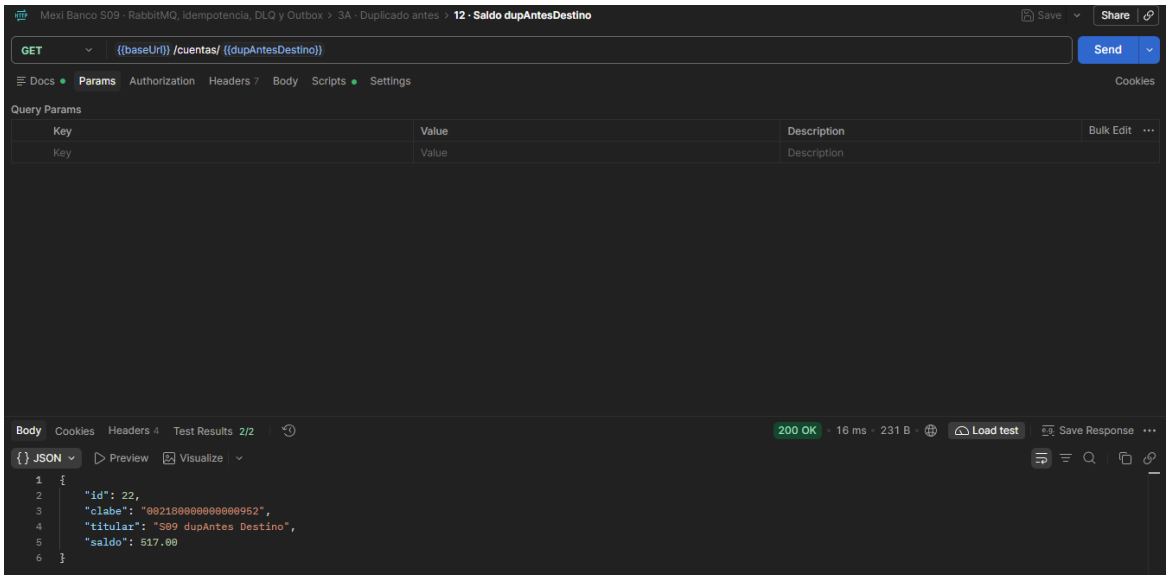


Figura 62: Saldo del destino después de una sola transferencia: 517.

3.2 Registrar el evento una sola vez

Ahora se agregó un registro de eventos procesados y se reconstruyó notificacion. La base de datos inserta eventoId con una restricción de unicidad y ON CONFLICT DO NOTHING. Si la inserción afecta cero filas, ese evento ya fue procesado y el servicio devuelve una lista vacía en lugar de crear más avisos.

```
INSERT INTO eventos_procesados
    (evento_id, tipo, transferencia_id, procesado_en)
VALUES (:id, :tipo, :transferenciaId, CURRENT_TIMESTAMP)
ON CONFLICT (evento_id) DO NOTHING
```

La marca del evento y las notificaciones se guardan dentro del mismo método con @Transactional. Si falla el guardado, se revierte también la marca; de otro modo un reintento podría encontrarse como procesado aunque los avisos nunca se hubieran guardado. Una consulta previa sin restricción única no bastaría, porque dos consumidores podrían revisar al mismo tiempo y ambos creer que el evento es nuevo.

```
@Transactional
public List<Notificacion> avisarDeTransferencia(EventoTransferencia evento) {
    if (procesados.marcarSiEsNuevo(evento.eventoId(),
            evento.tipo(), evento.transferenciaId()) == 0) {
        return List.of();
    }
    // Construir los avisos correspondientes al tipo de evento.
    return notificaciones.saveAll(avisos);
}
```

El fragmento anterior es un extracto del flujo: se omite solamente la construcción de los avisos para destacar la transacción y la deduplicación. En una transferencia realizada se crea un aviso para cada cuenta; en una revertida, uno de devolución para el origen.



Figura 63: Reconstrucción del consumidor con la protección de idempotencia.

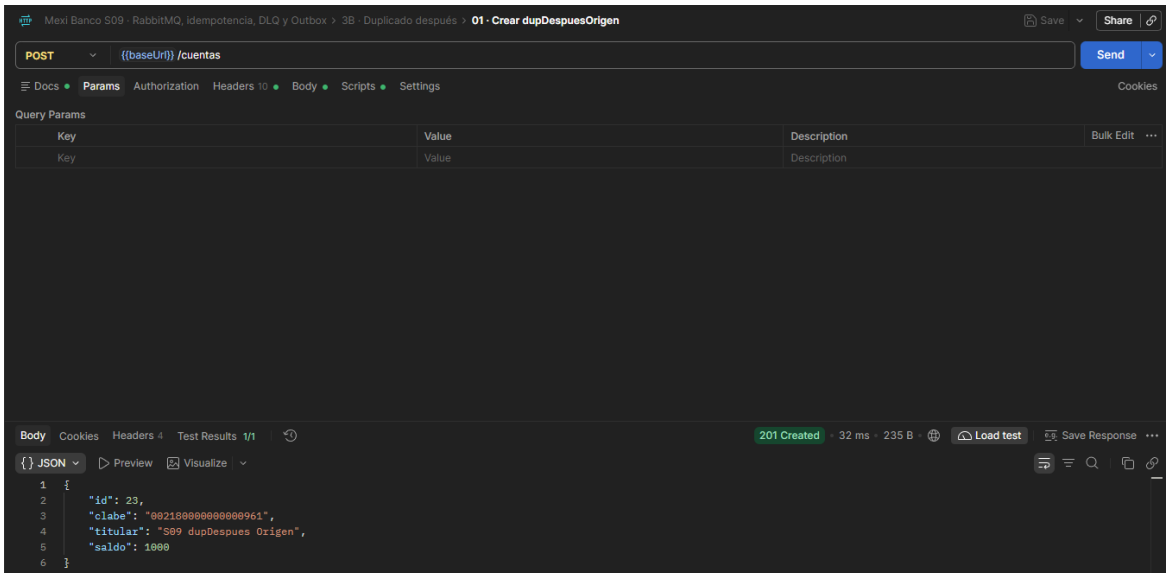


Figura 64: Origen 961 con saldo inicial de 1000.

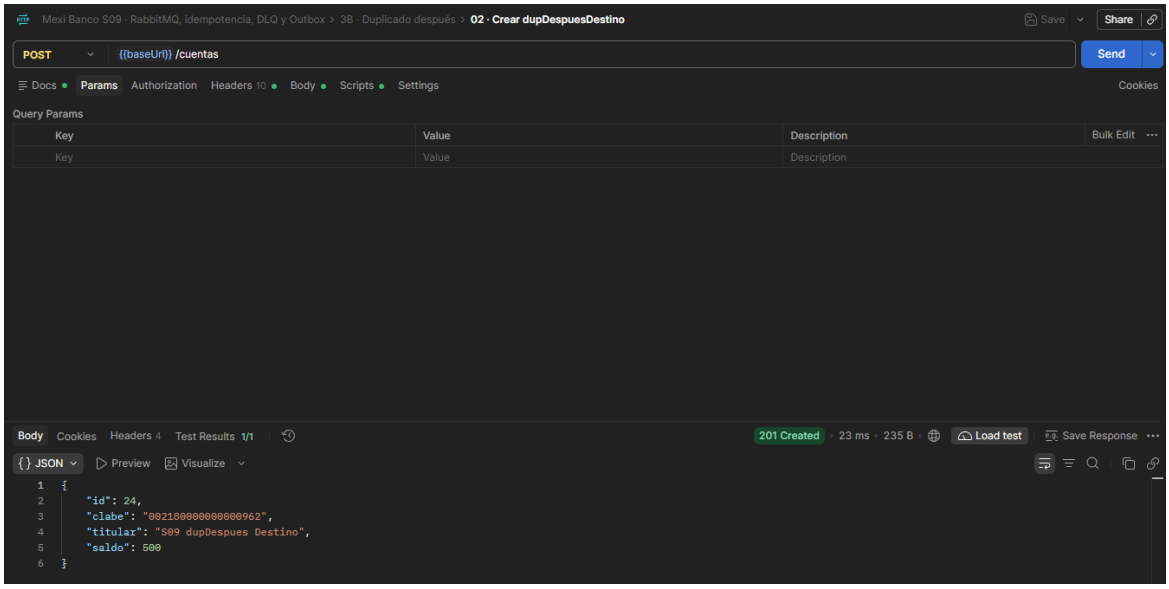


Figura 65: Destino 962 con saldo inicial de 500.

Se repitió la prueba con las cuentas 961 y 962: consumidor detenido, transferencia de 17, lectura con requeue y republicación del mismo evento. La transferencia fue la 33. Otra vez había dos mensajes Ready antes de arrancar el consumidor, de modo que la diferencia no se explica por haber enviado una sola copia.

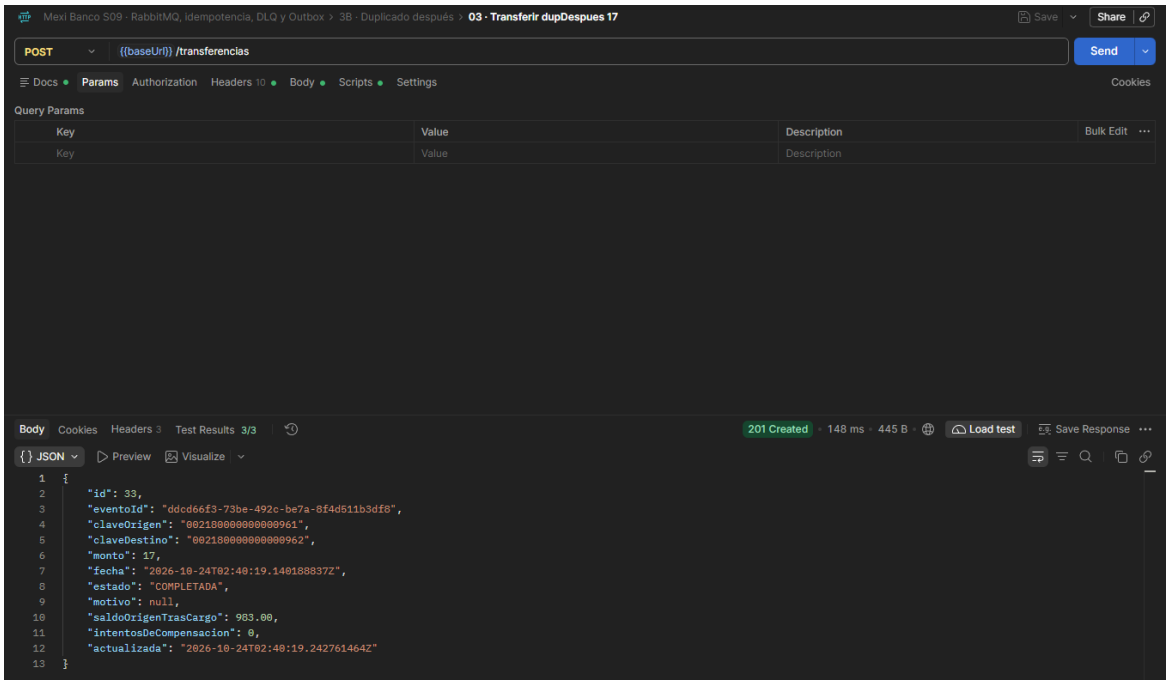


Figura 66: Transferencia 33 completada por 17.

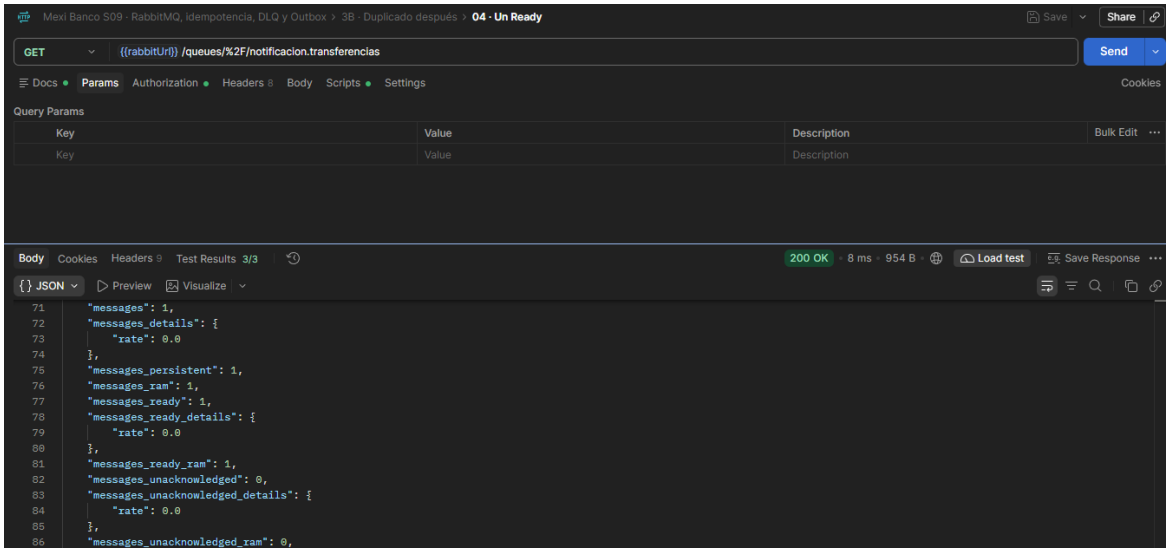


Figura 67: Un evento pendiente antes de duplicarlo.

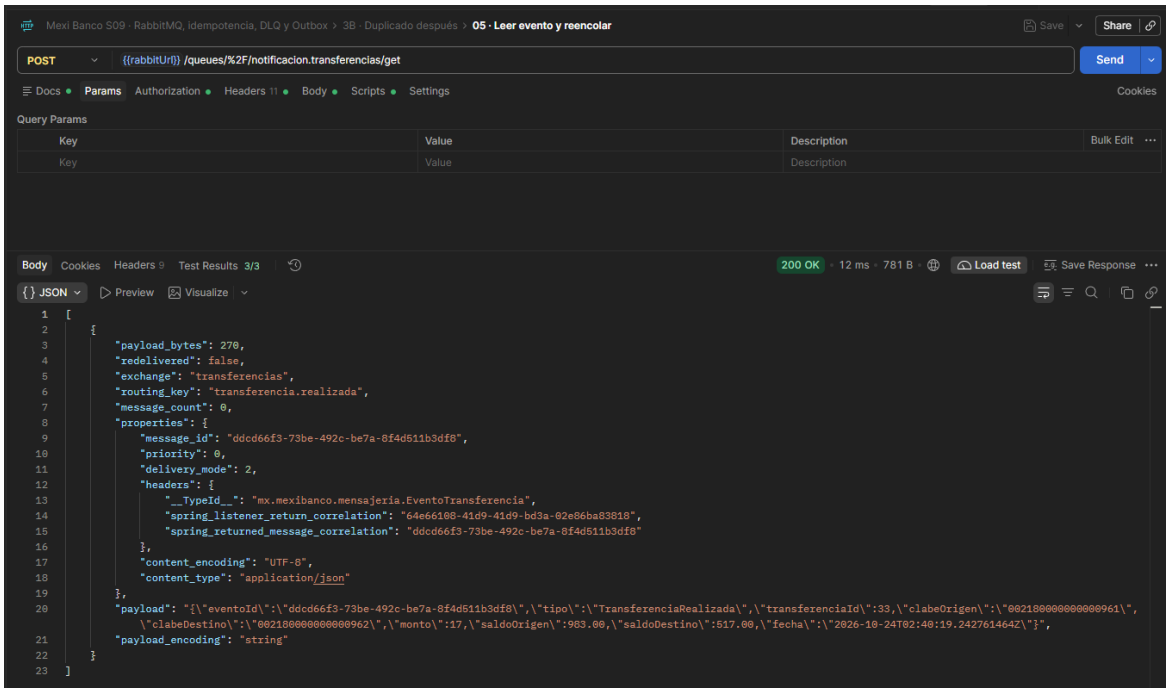


Figura 68: Lectura con requeue del evento de la transferencia 33.

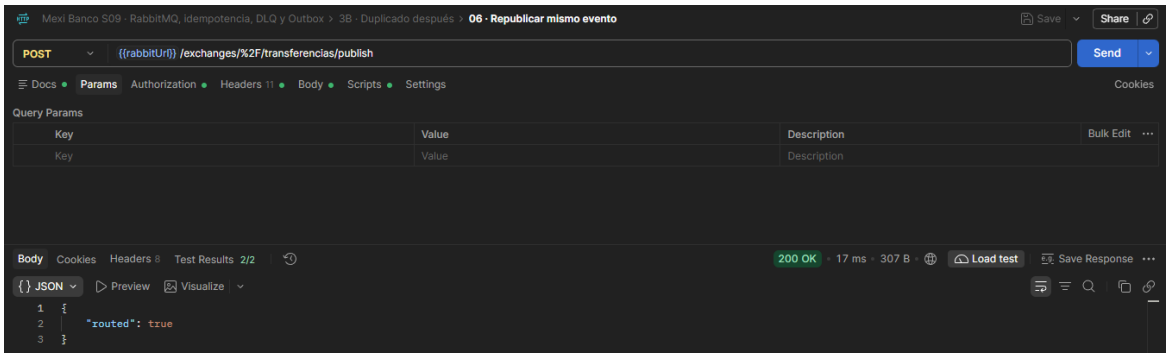


Figura 69: Republicación del mismo evento.

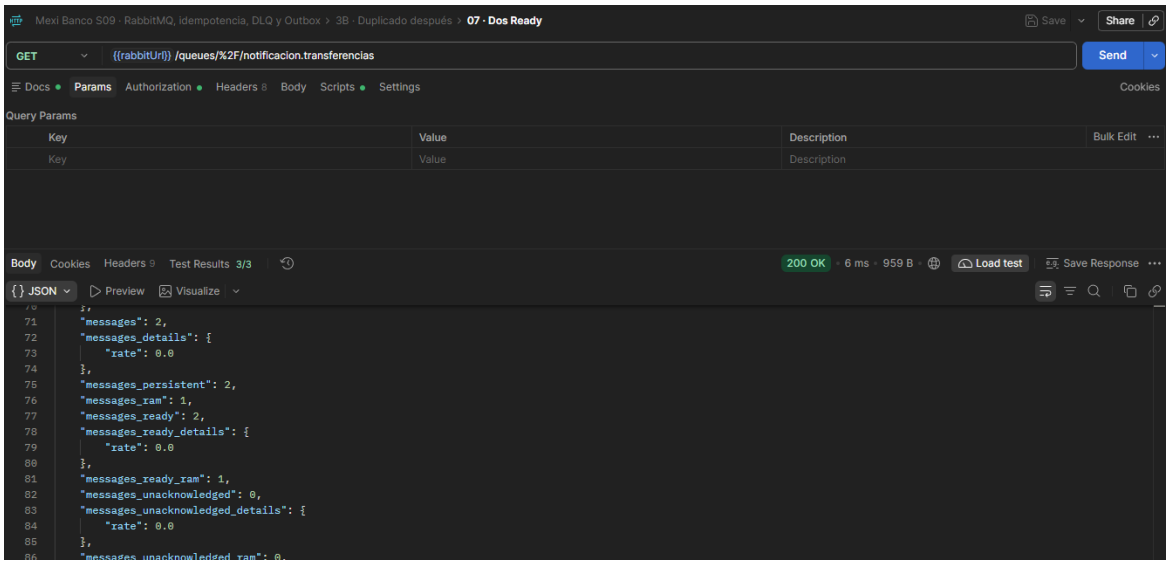


Figura 70: Dos mensajes Ready antes de reanudar el consumo.

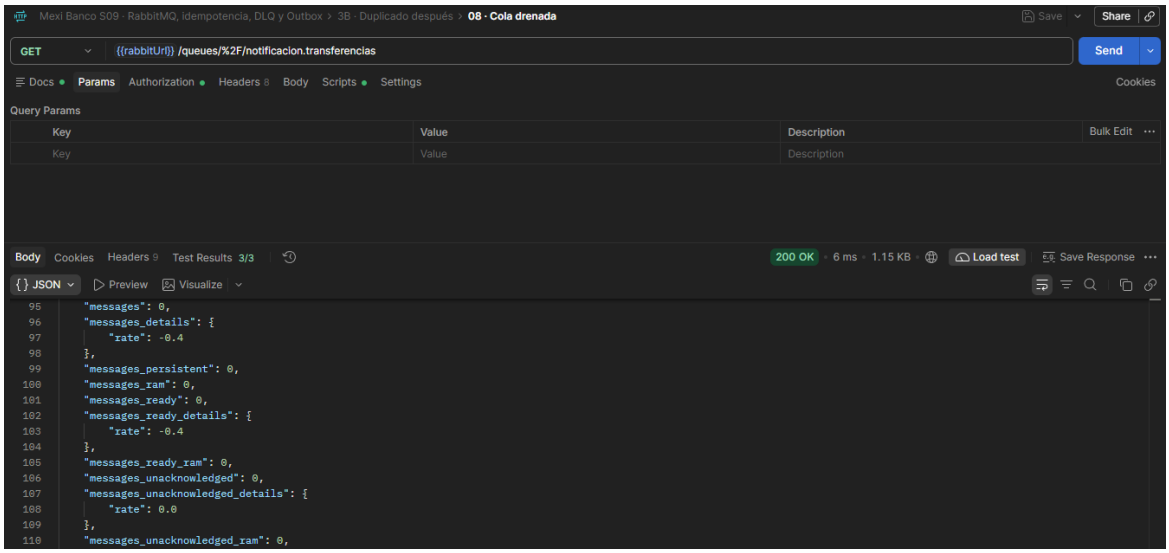


Figura 71: Cola vacía después de procesar las dos entregas.

Al reanudar notificacion quedó un solo aviso en cada cuenta. Los saldos volvieron a ser 983 y 517. Se aceptaron las dos entregas, pero sólo la primera produjo avisos nuevos; la segunda se reconoció por eventoId y se confirmó sin repetir el efecto.

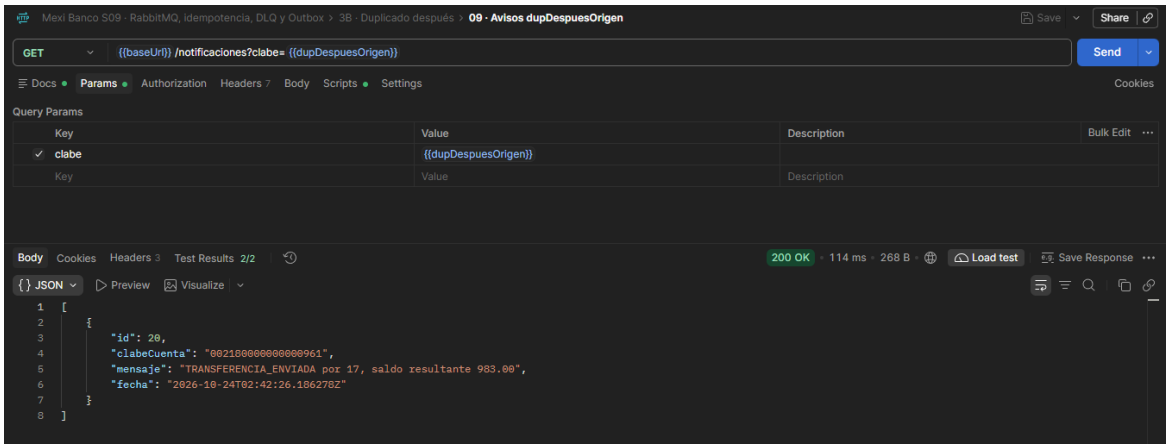


Figura 72: Un solo aviso en el origen después de deduplicar.

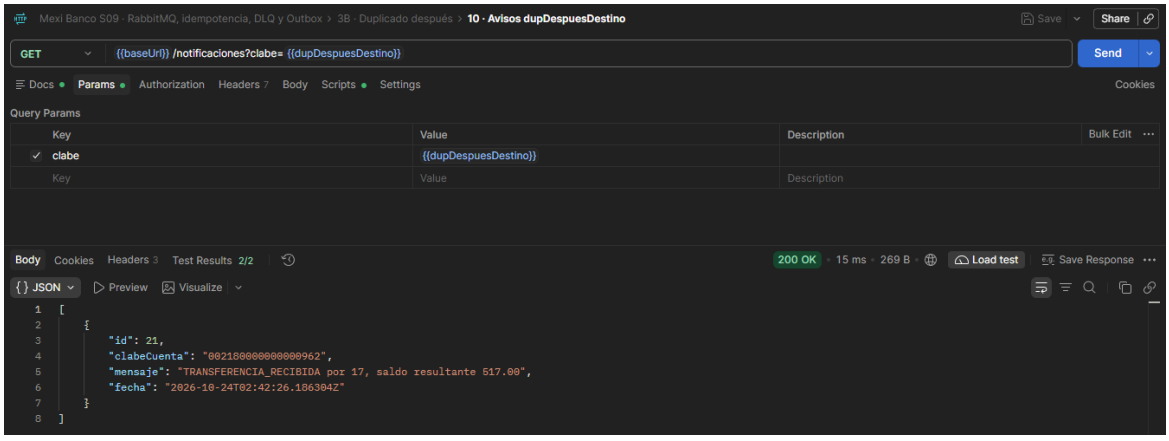


Figura 73: Un solo aviso en el destino después de deduplicar.

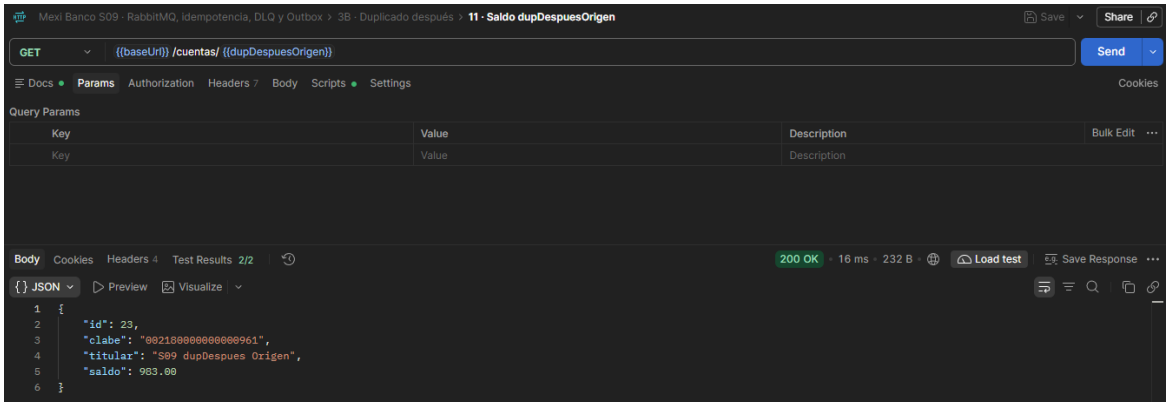


Figura 74: Saldo final del origen: 983.

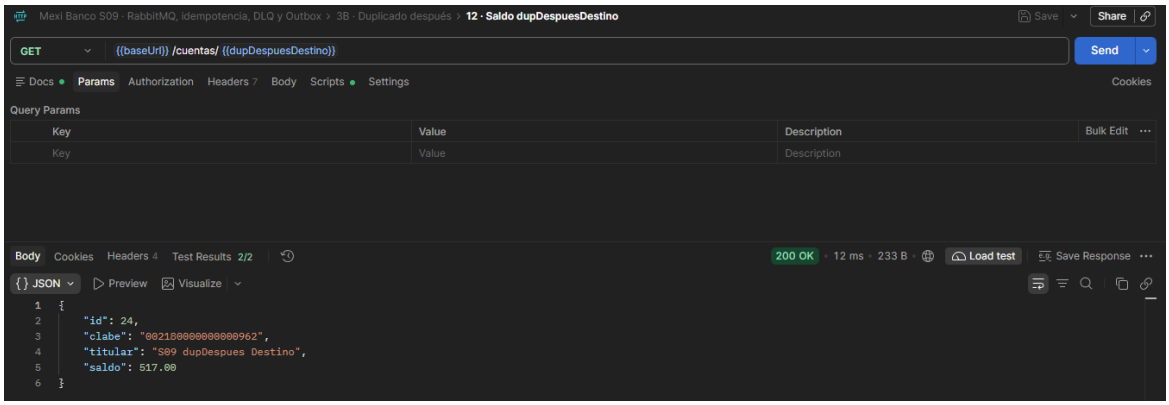


Figura 75: Saldo final del destino: 517.

4 Mensaje inválido y cola de mensajes muertos

Se prepararon las cuentas 971 y 972 para una última prueba. Primero se publicó un contenido con JSON incompleto usando una llave de ruteo válida. RabbitMQ lo aceptó y lo enrutó, porque el broker no valida que ese contenido corresponda al contrato de la aplicación. El fallo apareció al intentar interpretarlo en el consumidor.

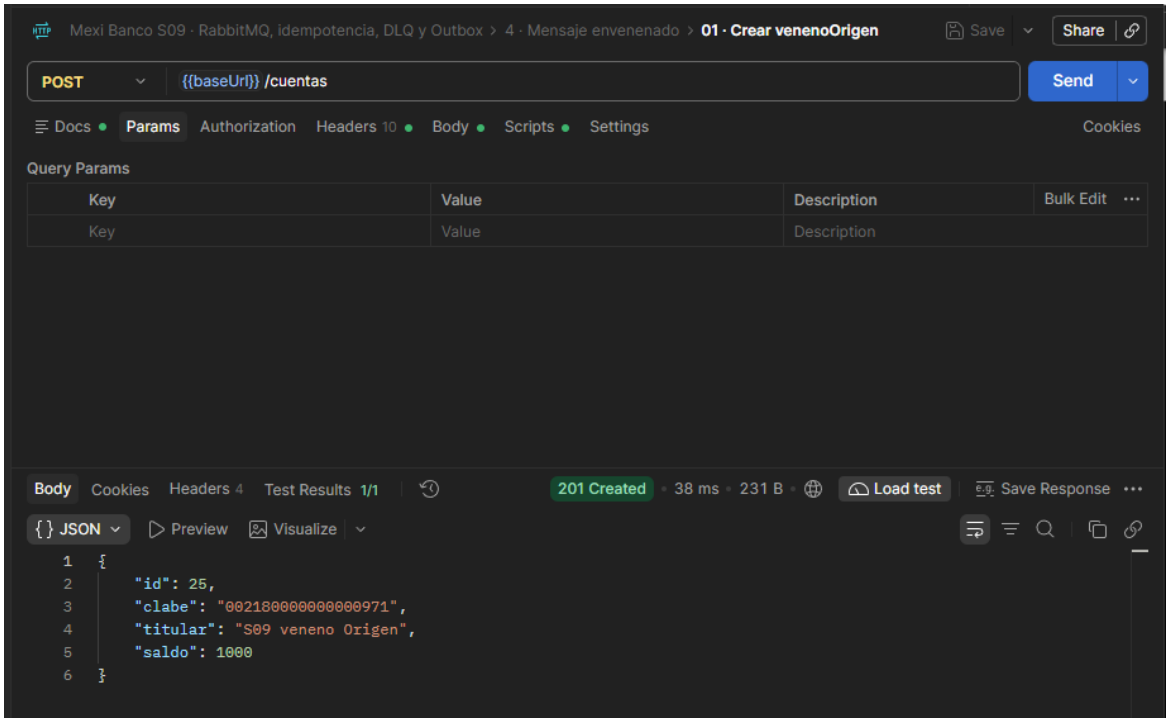


Figura 76: Origen 971 preparado para la prueba final.

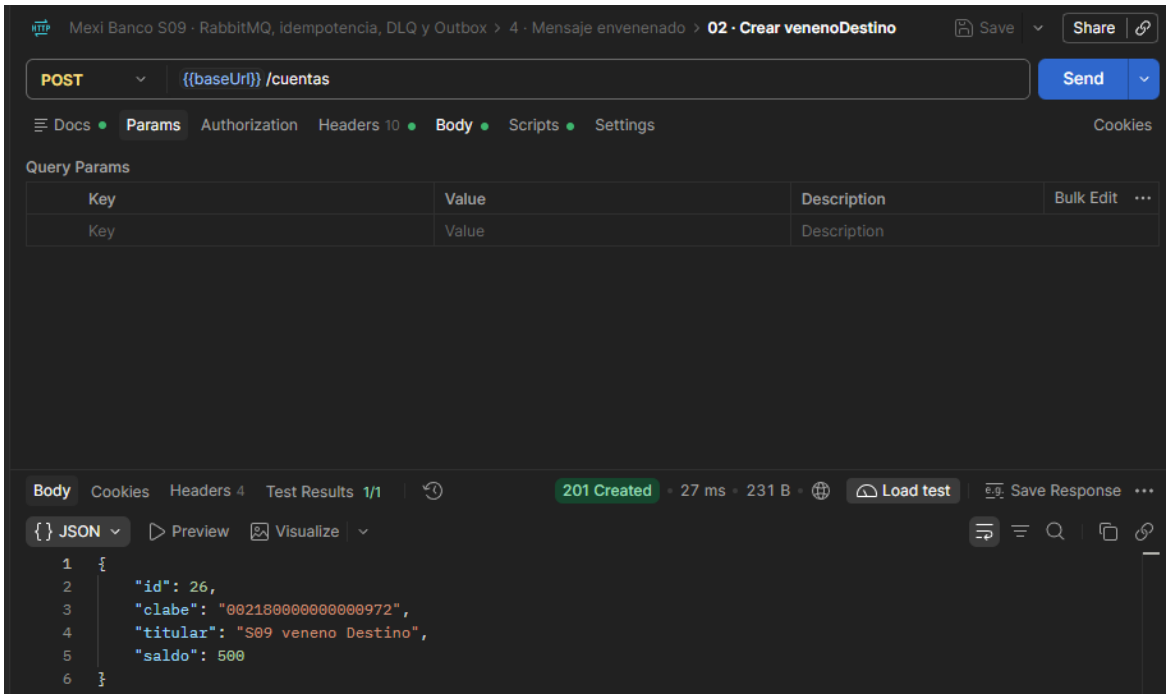


Figura 77: Destino 972 preparado para la prueba final.

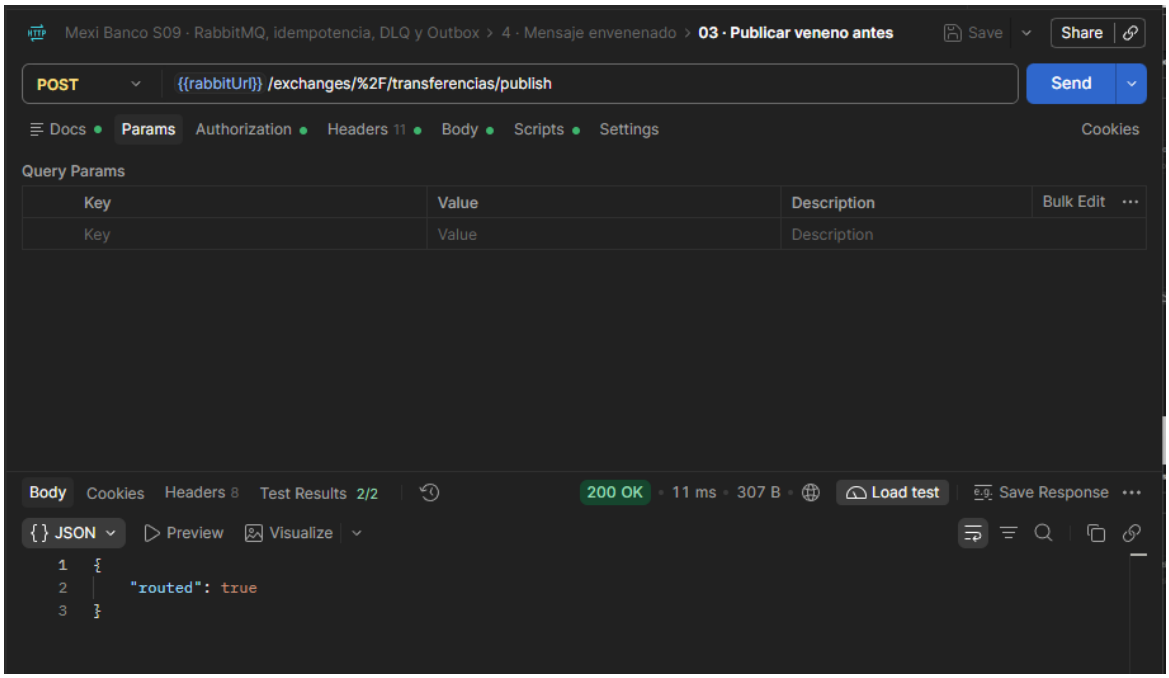


Figura 78: Publicación de un mensaje mal formado con ruteo exitoso.



Figura 79: Error de lectura del JSON incompleto en el consumidor.

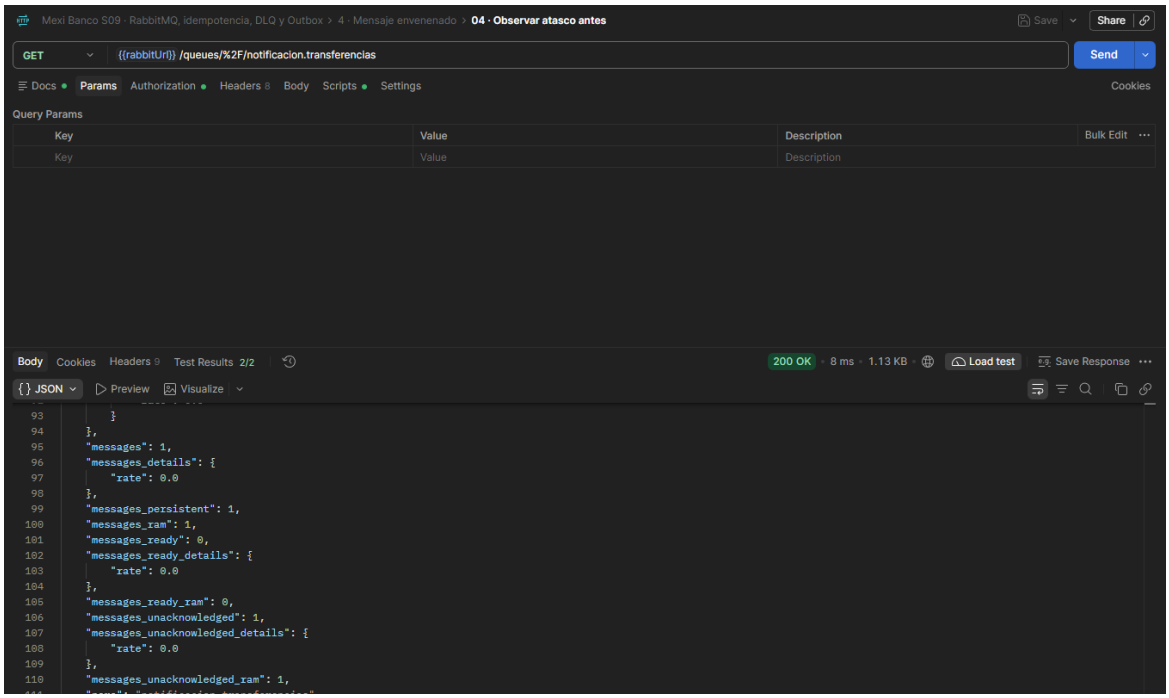


Figura 80: Un mensaje Unacked en la cola durante el problema.

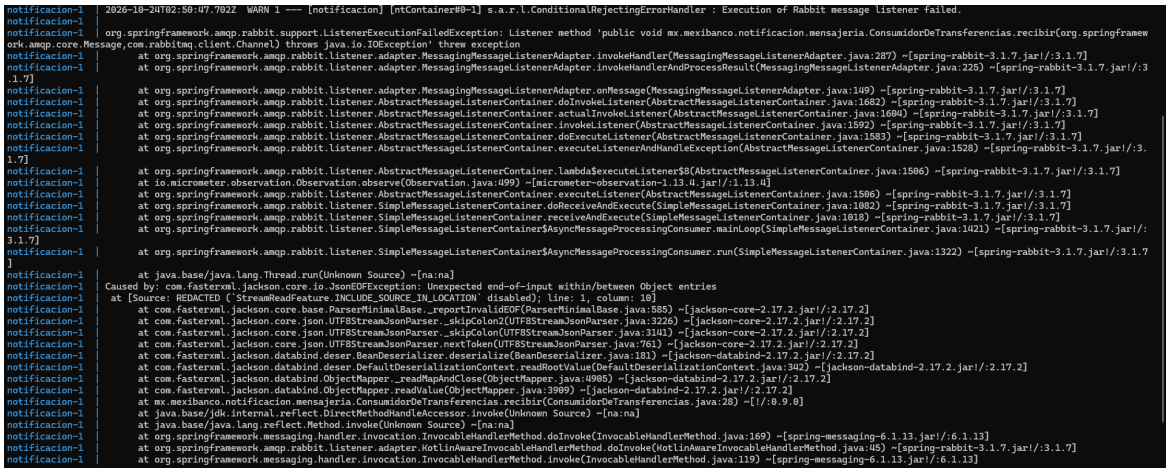


Figura 81: El error de interpretación permanece en los logs.

El error central fue JsonEOFException: el contenido terminó antes de completar un objeto JSON. En la consulta de la cola se observa un mensaje Unacked y cero Ready. Unacked significa entregado pero todavía sin confirmar. Volver a intentar el mismo contenido no corrige un JSON incompleto, por lo que este caso necesita una salida distinta a la de una falla transitoria de base de datos.

Se configuró el exchange directo transferencias.dlx y la cola durable notificacion.transferencias.dlq. La cola principal usa x-dead-letter-exchange con el valor transferencias.dlx y x-dead-letter-routing-key con notificacion.transferencias. El enlace de la DLQ usa esa misma llave. Así, cuando el consumidor rechaza un mensaje sin reencolarlo, RabbitMQ lo dirige a la cola de mensajes muertos.



Figura 82: Reconstrucción del consumidor con manejo del mensaje inválido.

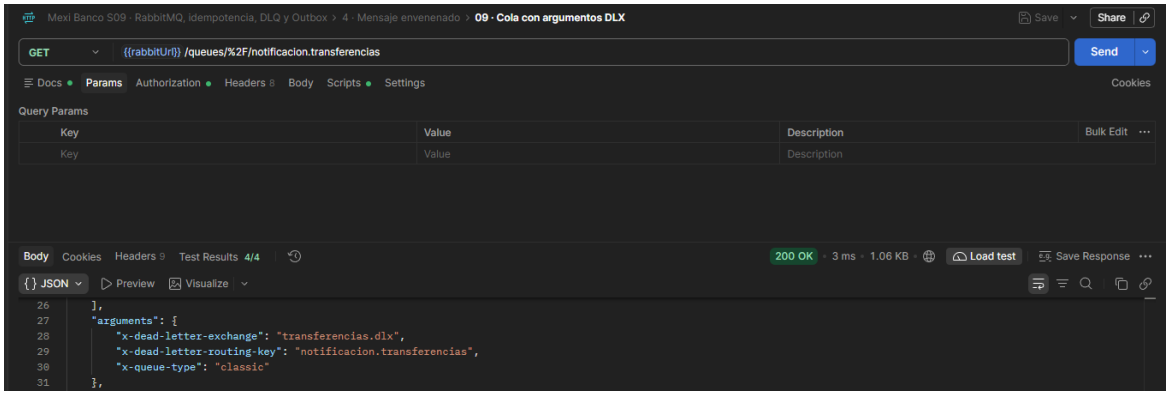


Figura 83: Argumentos de dead lettering configurados en la cola principal.

El listener trabaja con confirmaciones manuales. Primero convierte y valida el evento. Si el contenido es inválido, responde basicNack con requeue en false. Si falla el procesamiento de un evento válido, usa requeue en true para permitir otro intento. Sólo después de que

el servicio transaccional regresa correctamente envía basicAck. Estos son los tres caminos relevantes del consumidor, resumidos para comparar sus confirmaciones:

```
// JSON o contrato invalido:
canal.basicNack(tag, false, false);
// Falla durante el procesamiento de un evento valido:
canal.basicNack(tag, false, true);
// Transaccion terminada correctamente, incluso si era duplicado:
canal.basicAck(tag, false);
```

Se volvió a publicar el mensaje inválido. Ahora la DLQ mostró un mensaje Ready y ninguno Unacked. Al leerlo con requeue aparecieron los encabezados x-death con reason rejected, la cola original y la llave transferencia.realizada. Esto permite saber de dónde vino y por qué terminó ahí, además de conservar el contenido para revisarlo.

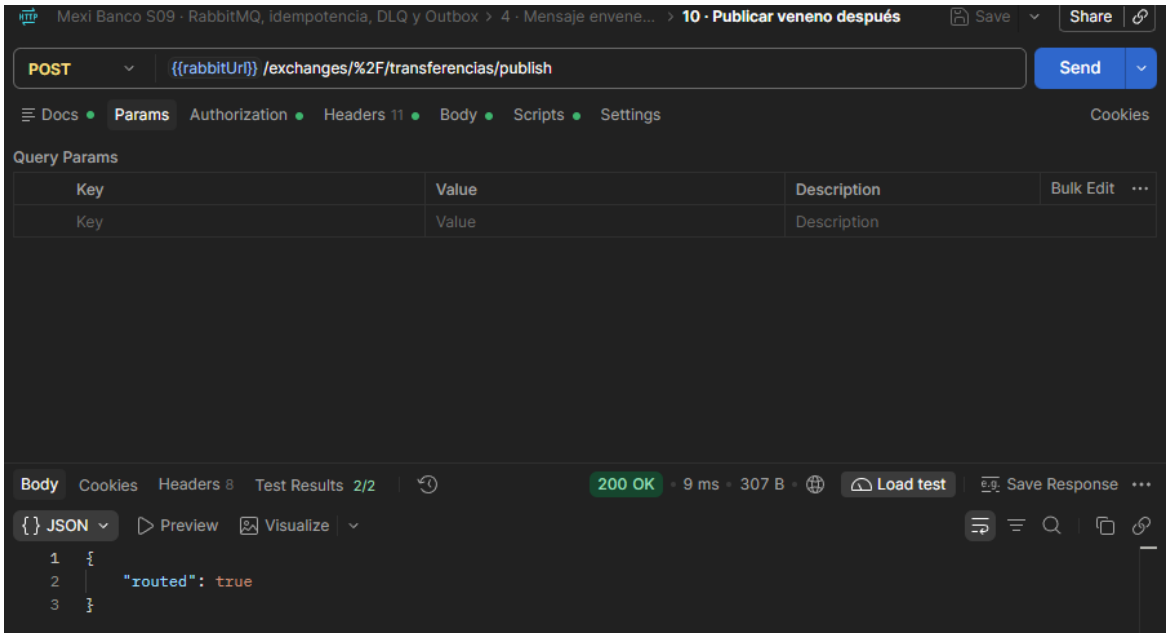


Figura 84: Nueva publicación del mensaje inválido.

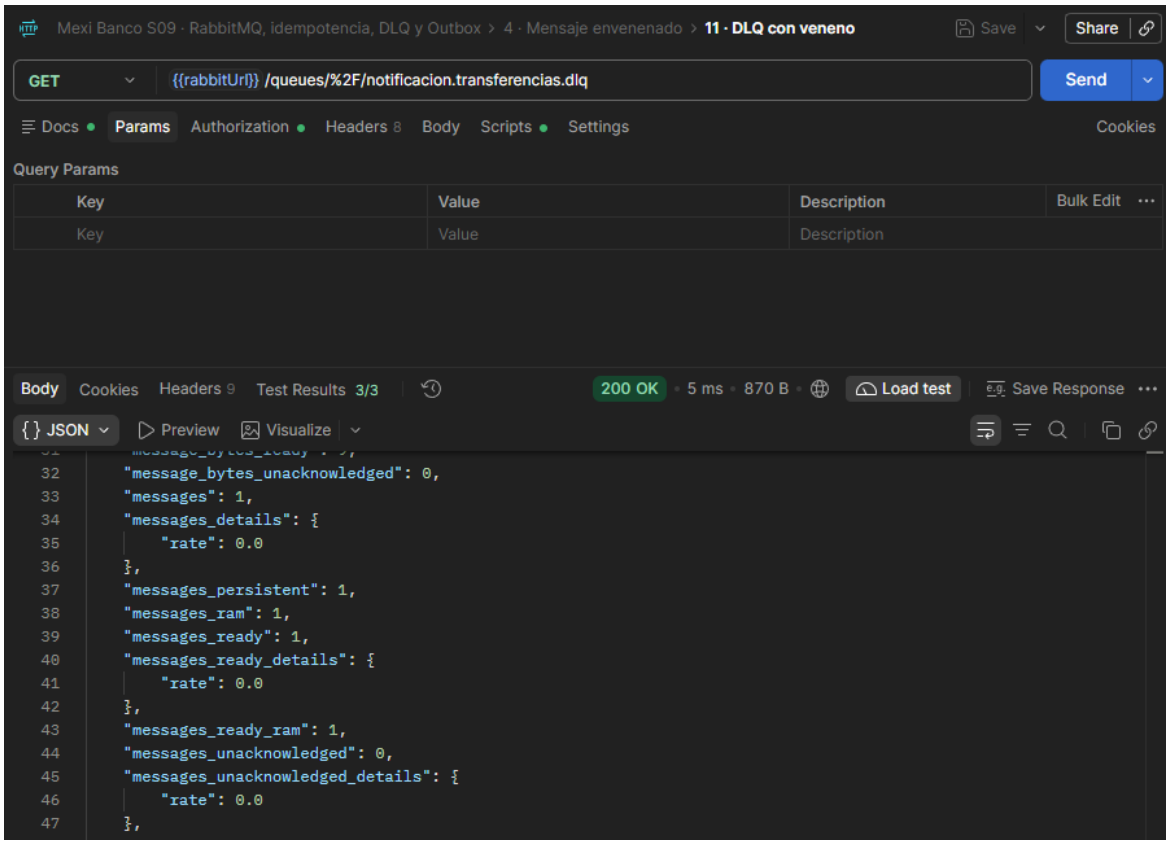


Figura 85: La DLQ contiene un mensaje Ready y cero Unacked.

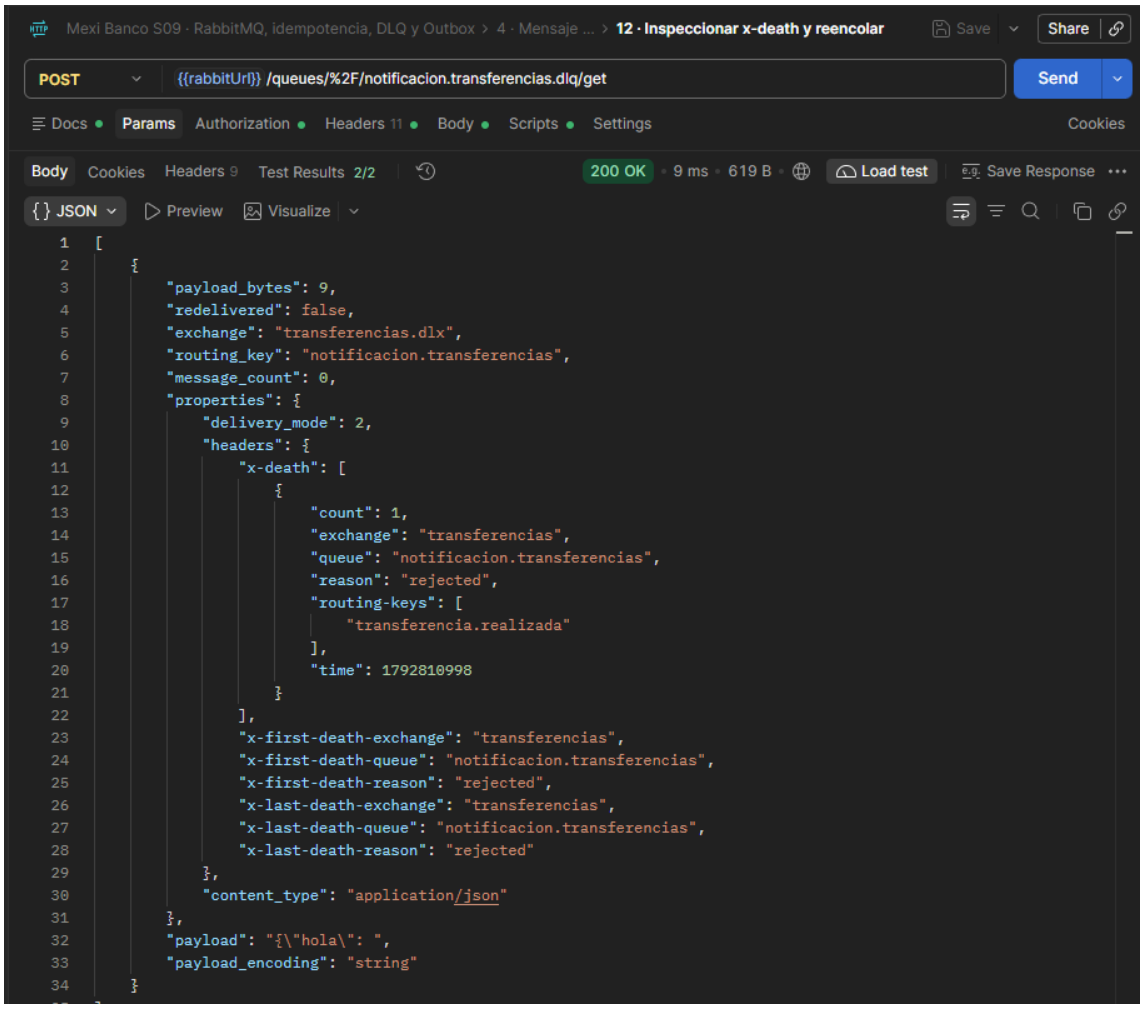


Figura 86: Contenido de la DLQ con x-death y motivo rejected.

Por último se envió una transferencia válida de 19 entre 971 y 972. La transferencia 34 respondió 201 y terminó COMPLETADA. Se creó un aviso por cuenta y los saldos quedaron en 981 y 519. El mensaje inválido se aisló y el consumidor pudo continuar con un evento correcto. La DLQ no repara ni vuelve a publicar automáticamente el contenido; lo separa para que no se trate como una operación normal.

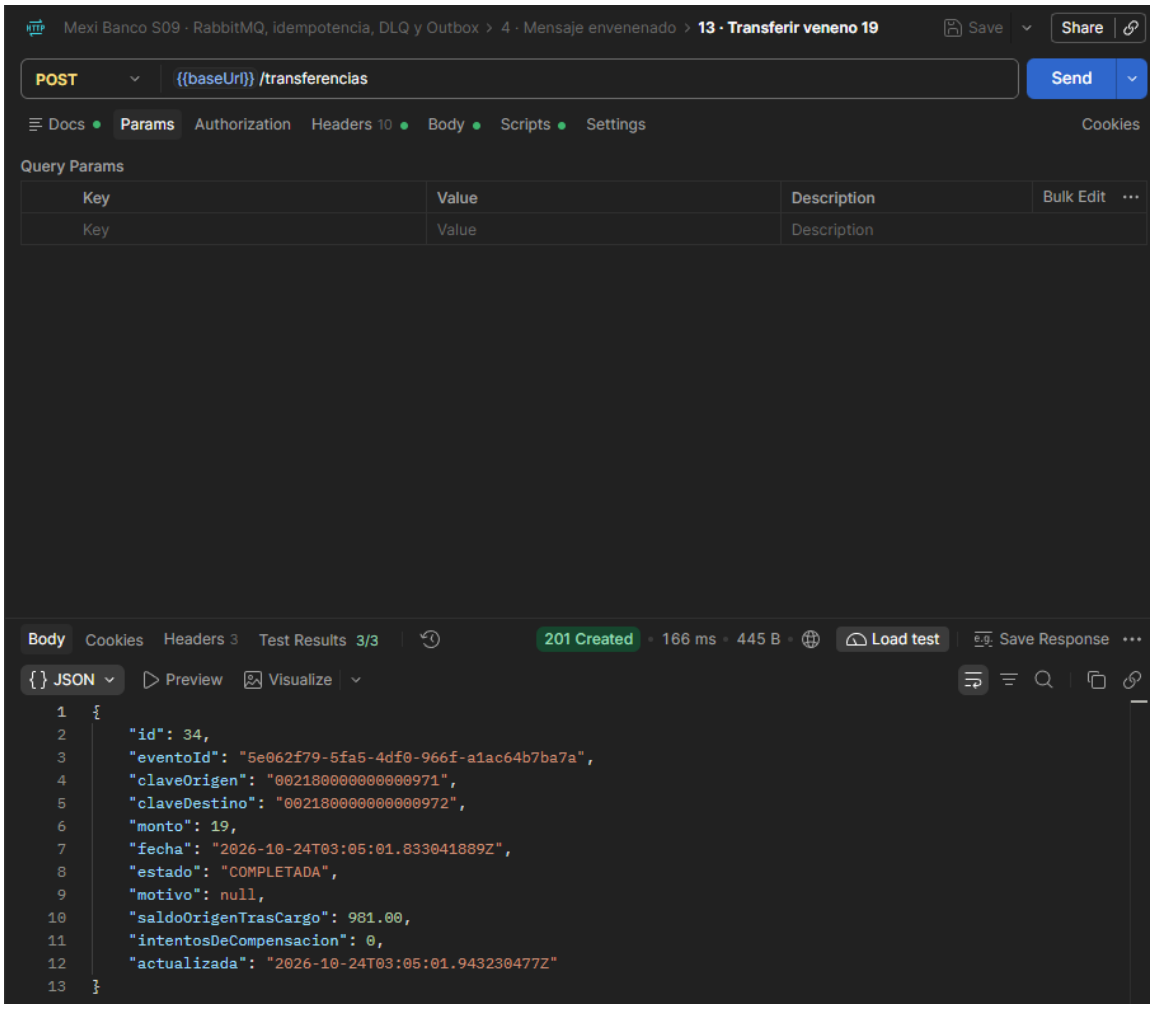


Figura 87: Transferencia válida de 19 después de aislar el mensaje inválido.

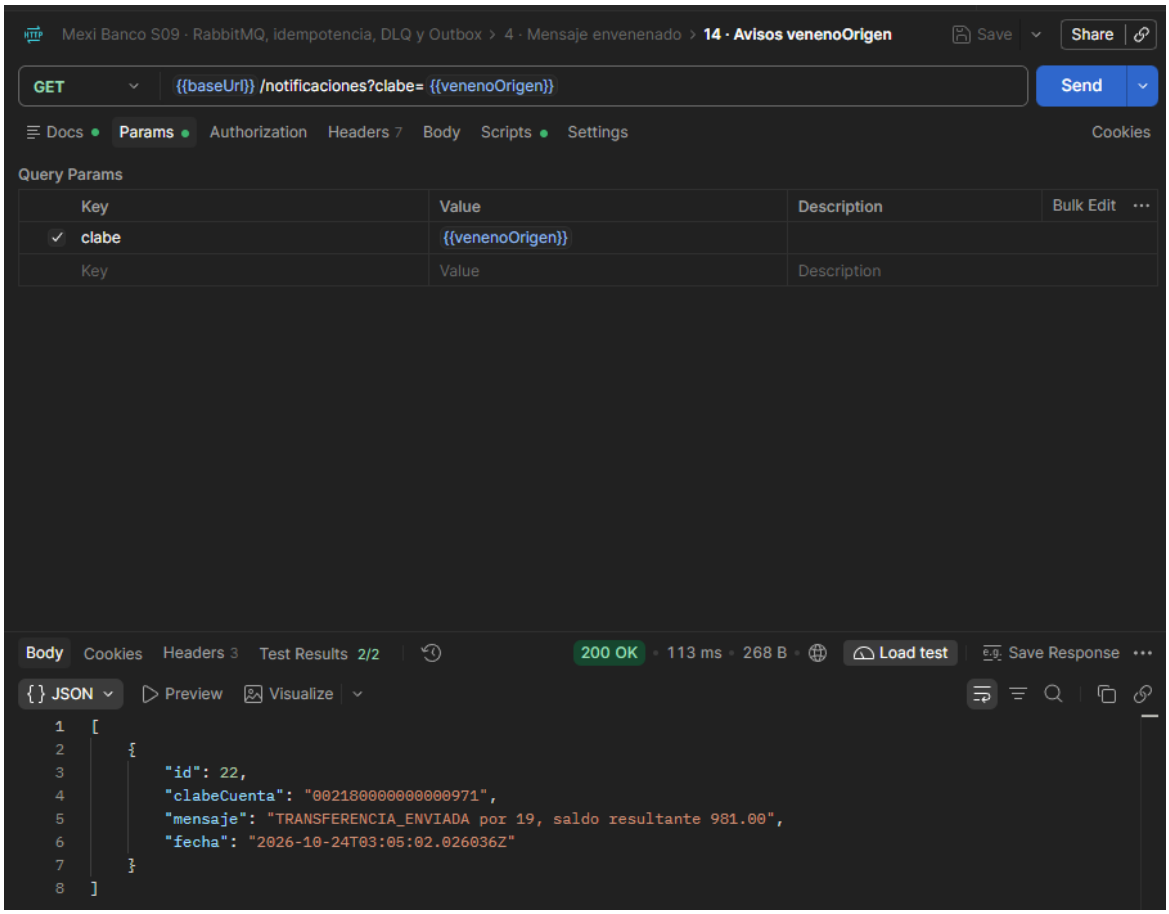


Figura 88: Aviso único del origen por la transferencia válida.

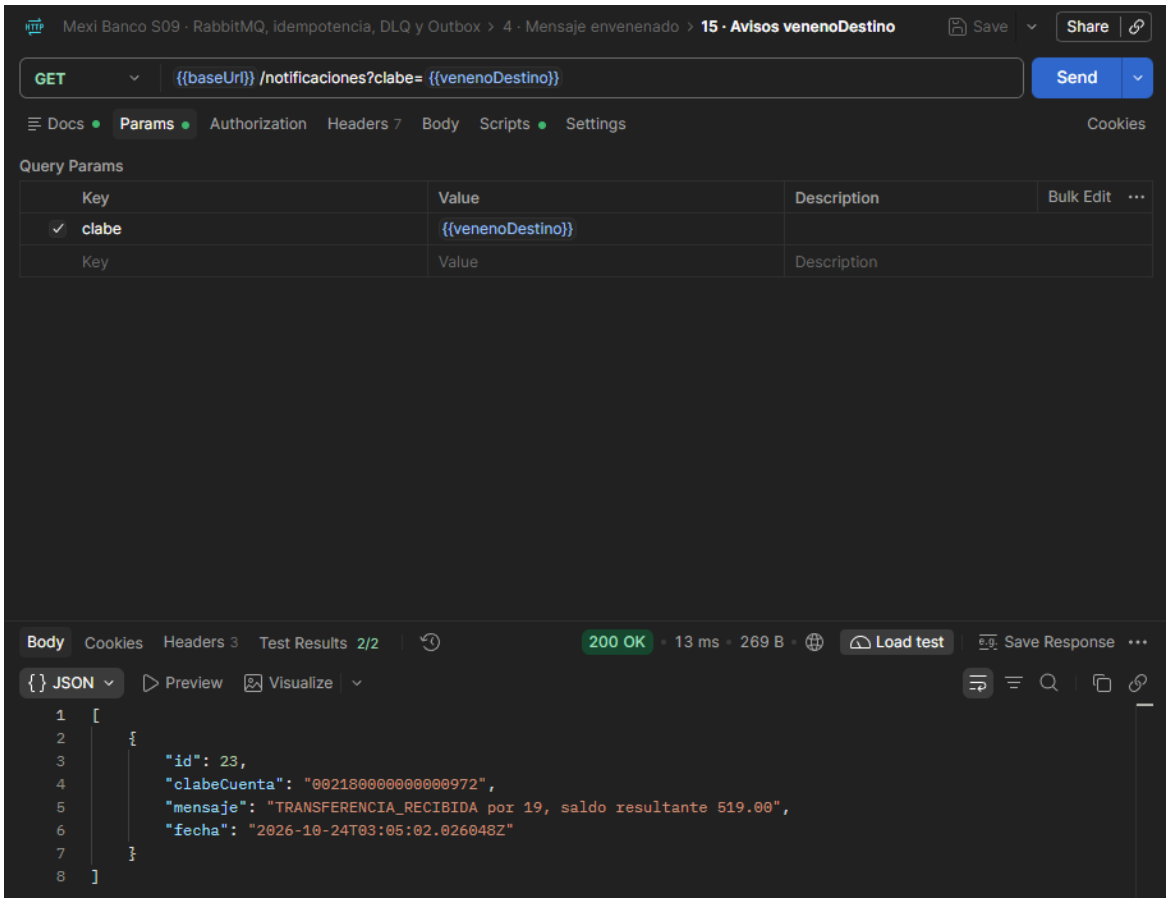


Figura 89: Aviso único del destino por la transferencia válida.

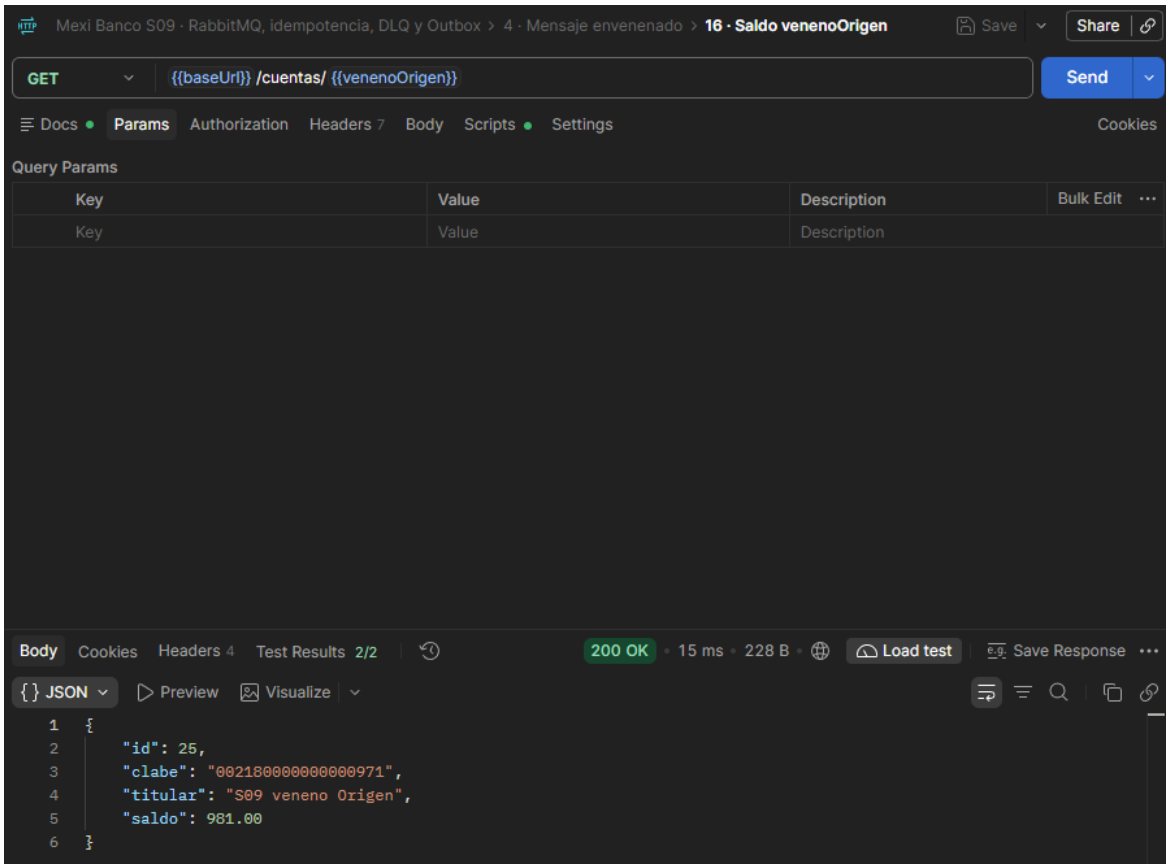


Figura 90: Saldo final del origen: 981.

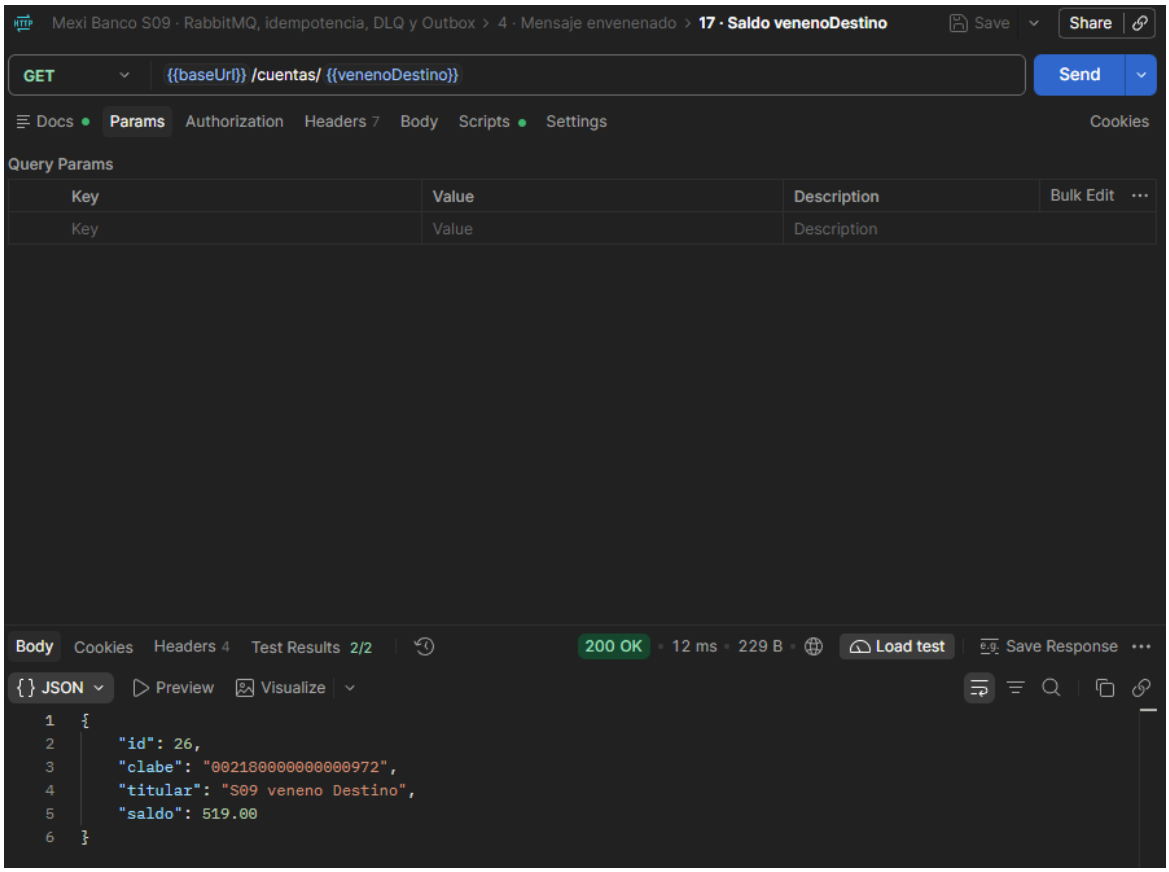


Figura 91: Saldo final del destino: 519.

5 Cierre y preguntas

¿Qué pasa si RabbitMQ cae cuando la transferencia ya está COMPLETADA? En el código actual se guarda el estado final y después se intenta publicar. El método avisar captura el error del publicador y lo registra, por lo que la caída del broker no revierte los saldos ni vuelve pendiente una transferencia ya completada. Los publisher confirms permiten detectar un envío no confirmado, pero no garantizan por sí solos que ese evento se publique más adelante. No se debe confundir que la transferencia esté terminada con que el aviso ya haya llegado a la cola.

¿Por qué el ACK va después del commit? Porque el ACK le permite a RabbitMQ retirar el mensaje. Si se envía primero y luego falla la base de datos, el trabajo puede desaparecer sin dejar el aviso. Al confirmar la base primero queda la posibilidad contraria: que el proceso caiga antes del ACK y el broker reentregue. El registro único de eventoId hace que esa reentrega no cree otra notificación. Son entregas que pueden repetirse, con un efecto local deduplicado.

¿Qué se sacrifica al desacoplar los avisos? Ya no se puede prometer que, al responder la transferencia, la notificación sea inmediatamente visible. Se acepta consistencia eventual entre el traslado del dinero y sus avisos. A cambio, detener notificacion no impide completar transferencias mientras el broker y la cola estén disponibles. En la prueba, los tres eventos esperaron y se procesaron al recuperar el consumidor.

Resultados y conclusión

La diferencia con la llamada directa se observó al detener notificacion: primero quedó un aviso pendiente aunque el dinero ya se había movido; con la cola declarada, las tres transferencias terminaron y sus eventos esperaron al consumidor. Después, duplicar un evento produjo dos avisos por cuenta antes de la protección y uno por cuenta después de registrar eventoId de forma transaccional. Finalmente, el mensaje inválido terminó en la DLQ y una transferencia válida siguió procesándose.

Con esto se comprobó que publicar, consumir y confirmar son pasos distintos. La cola resuelve la espera del consumidor, la idempotencia controla el efecto de los repetidos y la DLQ separa los mensajes que no pueden procesarse tal como llegaron. También quedó claro que la evidencia debe revisarse por completo: en la compensación se obtuvo el estado y el aviso de devolución, pero el saldo de 6400 no permite concluir que la devolución económica haya sido correcta.